



EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

ADVANCED OBJECT ORIENTED PROGRAMMING

2024–2025 SPRING SEMESTER

PROJECT REPORT

DELIVERY DATE

20/06/2025

Project Topic

CLABTicket 

PREPARED BY

05220001067-Aykut HACIOĞLU

05230000350-Ceylin TOR

05220000366-Lidya DELİAĞA

05230000983-Ali Batıkan ÖCAL

Table of Contents

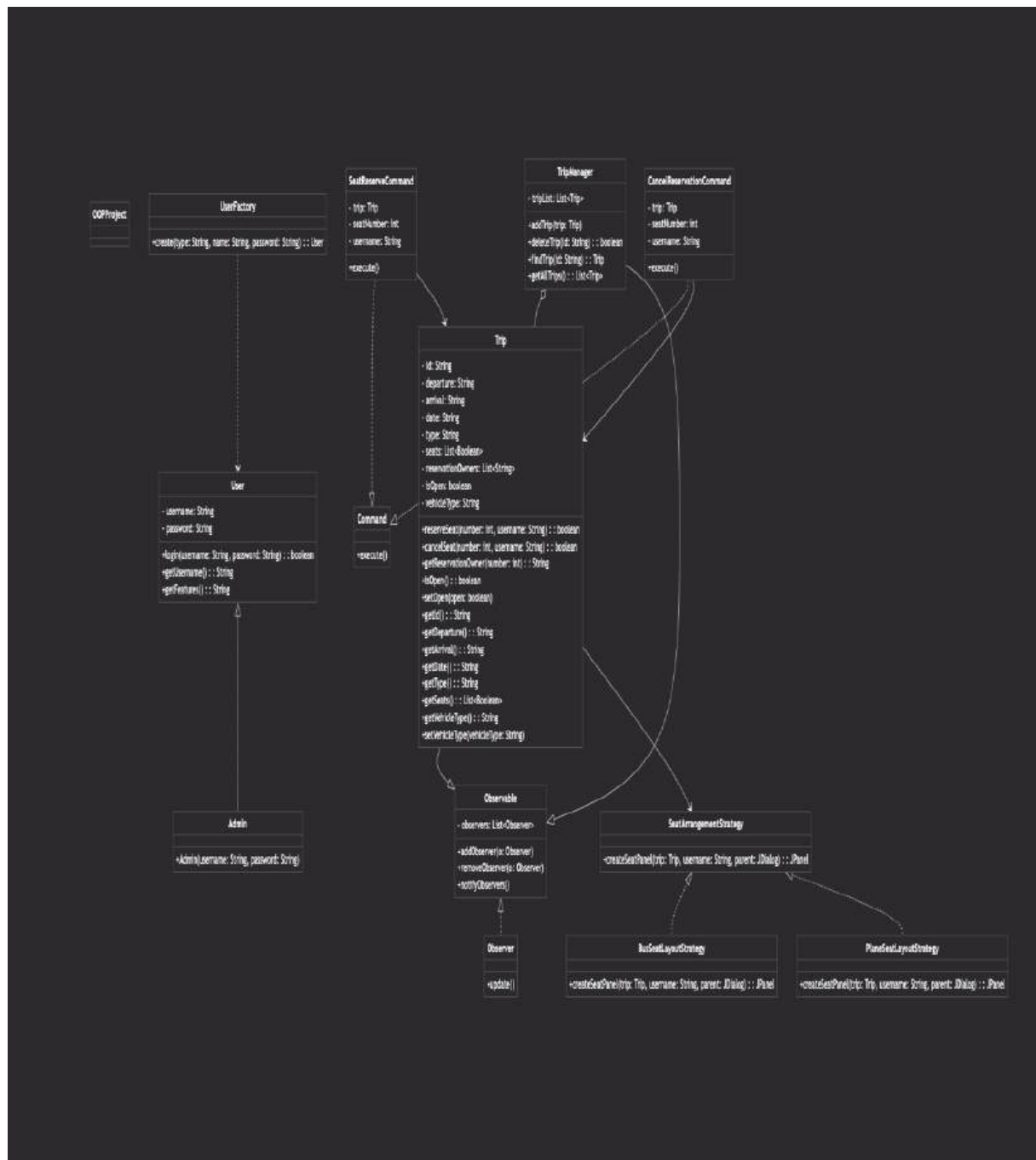
1-PROJECT DESCRIPTION.....	4
2-UML DIAGRAM.....	5
3-DESIGN PATTERNS.....	6
3.1-Factory Pattern	6
3.2-Strategy Pattern	6
3.3-Command Pattern	9
3.4-Observer Pattern	10
4-SCREENSHOTS AND DESCRIPTION OF CLASSES.....	18
4.1-Admin Class	18
4.2-AdminFrame Class	19
4.3-BusSeatLayoutStrategy Class	22
4.4-CancelReservationCommand Class	23
4.5-Command Class.....	24
4.6-LoginFrame Class	25
4.7-OOPProject Class	28
4.8-Observable Class	29
4.9-Observer Class.....	30
4.10-PlaneSeatLayoutStrategy Class	30
4.11-RegisterDialog Class	31
4.12-SeatArrangementStrategy Class	33
4.13-SeatButtonFactory Class	34
4.14-SeatReserveCommand Class	35
4.15-SeatSelectionDialog Class.....	37
4.16-Trip Class.....	40
4.17-TripListDialog Class	41
4.18-TripManager Class.....	44
4.19-User Class	45
4.20-UserFactory Class	46
4.21-UserFrame Class	47
5-SCREENSHOTS FROM APPLICATION	51
5.1-Login	51

5.2-Sign Up	54
5.3-Select Transportation Type	56
5.4-User Panel-Bus Trips	57
5.5-Seat Selection (Bus).....	60
5.6-User Panel-Plane Trips.....	62
5.7-Seat Selection(Plane).....	65
5.8-Admin Sign Up.....	67
5.9-Admin Panel	71
5.10-Trip List (Admin Panel)	72

1-PROJECT DESCRIPTION

This project is a desktop-based graphical user interface application designed for managing seat reservations on bus and airplane trips. Users can log in to the system, view available trips, reserve seats, and cancel their existing reservations. Admin users have additional privileges, such as adding new trips and deleting existing ones. The project employs key object-oriented design patterns including Strategy, Command, Observer and Factory to ensure modularity and maintainability. The user interface is developed using Java Swing, and seat layouts dynamically adjust based on the selected transportation type (bus or plane).

2-UML DIAGRAM



3-DESIGN PATTERNS

3.1-Factory Pattern

```
1 package aoopproject1;
2
3 public class UserFactory {
4     public static User create(String type, String name, String password) {
5         if (type.equalsIgnoreCase("admin")) {
6             return new Admin(name, password); // Create Admin user
7         } else {
8             return new User(name, password); // Create standard User
9         }
10    }
11 }
12
```

-The Factory Pattern is a creational design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created.

-The Factory Pattern is used in the **UserFactory** class to create user objects dynamically based on their type. This abstracts the instantiation logic away from the main application, promoting cleaner and more modular code. It returns either a **User** or an **Admin** object depending on the given role string.

3.2-Strategy Pattern

```
SeatArrangementStrategy.java x PlaneSeatLayoutStrategy.java x BusSeatLayoutStrategy.java x
Source History
1 package aoopproject1;
2
3 import javax.swing.*;
4
5 // Strategy interface for creating seat layout for bus and plane
6 public interface SeatArrangementStrategy {
7     public JPanel createSeatPanel(Trip trip, String username, JDialog parent); // Returns seat layout panel
8 }
9
```

```

SeatArrangementStrategy.java x PlaneSeatLayoutStrategy.java x BusSeatLayoutStrategy.java x
Source History
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.List;
6
7 public class PlaneSeatLayoutStrategy implements SeatArrangementStrategy {
8
9     @Override
10    public JPanel createSeatPanel(Trip trip, String username, JDialog parent) {
11        JPanel panel = new JPanel(); // Main panel for seat layout
12        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS)); // Arrange rows vertically
13        panel.setBackground(new Color(239, 250, 231)); // Set background color
14
15        List<Boolean> seats = trip.getSeats(); // Get seat availability list
16        int seatIndex = 0; // Index for seat position
17        int rowNumber = 1; // Row counter
18
19        while (seatIndex < seats.size()) {
20            JPanel row = new JPanel(new FlowLayout(FlowLayout.LEFT, 20, 5)); // Row panel with spacing
21            row.setBackground(panel.getBackground());
22
23            JLabel rowLabelStart = new JLabel(String.valueOf(rowNumber)); // Row number label (start)
24            rowLabelStart.setPreferredSize(new Dimension(30, 48));
25            rowLabelStart.setFont(new Font("Segoe UI", Font.PLAIN, 14));
26            row.add(rowLabelStart);
27
28            for (int i = 0; i < 4 && seatIndex < seats.size(); i++, seatIndex++) {
29                row.add(SeatButtonFactory.create(trip, seatIndex, username, parent)); // Left side seats
30            }
31
32            row.add(Box.createRigidArea(new Dimension(40, 0))); // Aisle space
33
34            for (int i = 0; i < 4 && seatIndex < seats.size(); i++, seatIndex++) {
35                row.add(SeatButtonFactory.create(trip, seatIndex, username, parent)); // Right side seats
36            }
37
38            JLabel rowLabelEnd = new JLabel(String.valueOf(rowNumber)); // Row number label (end)
39            rowLabelEnd.setPreferredSize(new Dimension(30, 48));
40            rowLabelEnd.setFont(new Font("Segoe UI", Font.PLAIN, 14));
41            row.add(rowLabelEnd);
42
43            panel.add(row); // Add row to the panel
44            rowNumber++;
45        }
46
47        return panel; // Return complete seat layout
48    }
49 }
50

```

```
SeatArrangementStrategy.java x PlaneSeatLayoutStrategy.java x BusSeatLayoutStrategy.java x
Source History
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.List;
6
7 public class BusSeatLayoutStrategy implements SeatArrangementStrategy {
8
9     @Override
10    public JPanel createSeatPanel(Trip trip, String username, JDialog parent) {
11        JPanel panel = new JPanel();
12        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS)); // Vertical layout for rows
13        panel.setBackground(new Color(239, 250, 231)); // Set background color
14
15        List<Boolean> seats = trip.getSeats(); // Get seat occupancy list
16        int seatIndex = 0; // Current seat index
17        int rowNumber = 1; // Row counter
18
19        while (seatIndex < seats.size()) {
20            JPanel row = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 5)); // Row with spacing
21            row.setBackground(panel.getBackground());
22
23            JLabel rowLabelStart = new JLabel(String.valueOf(rowNumber)); // Row number on left
24            rowLabelStart.setPreferredSize(new Dimension(30, 48));
25            rowLabelStart.setFont(new Font("Segoe UI", Font.PLAIN, 14));
26            row.add(rowLabelStart);
27
28            for (int i = 0; i < 2 && seatIndex < seats.size(); i++, seatIndex++) {
29                row.add(SeatButtonFactory.create(trip, seatIndex, username, parent)); // Left side seats
30            }
31
32            row.add(Box.createRigidArea(new Dimension(40, 0))); // Aisle space
33
34            for (int i = 0; i < 2 && seatIndex < seats.size(); i++, seatIndex++) {
35                row.add(SeatButtonFactory.create(trip, seatIndex, username, parent)); // Right side seats
36            }
37
38            JLabel rowLabelEnd = new JLabel(String.valueOf(rowNumber)); // Row number on right
39            rowLabelEnd.setPreferredSize(new Dimension(30, 48));
40            rowLabelEnd.setFont(new Font("Segoe UI", Font.PLAIN, 14));
41            row.add(rowLabelEnd);
42
43            panel.add(row); // Add row to panel
44            rowNumber++;
45        }
46
47        return panel; // Return the full seat panel
48    }
49 }
50
```

-The Strategy Pattern is a behavioral pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. This allows the algorithm to vary independently from clients that use it.

-The Strategy Pattern is implemented through the **SeatArrangementStrategy** interface and its concrete classes **BusSeatLayoutStrategy** and **PlaneSeatLayoutStrategy**. This allows the program to select the appropriate seat layout algorithm at runtime based on the type of trip (bus or plane), promoting flexibility and code reuse.

3.3-Command Pattern

```
Command.java x SeatReserveCommand.java x CancelReservationCommand.java x
Source History
1 package aoopproject1;
2
3 // Command interface for implementing command pattern
4 public interface Command {
5     void execute(); // Method to execute the command
6 }
7
```

```
Command.java x SeatReserveCommand.java x CancelReservationCommand.java x
Source History
1 package aoopproject1;
2
3 public class SeatReserveCommand implements Command {
4     private Trip trip; // Trip where the seat is being reserved
5     private int seatNumber; // Seat number to reserve
6     private String username; // User making the reservation
7
8     public SeatReserveCommand(Trip trip, int seatNumber, String username) {
9         this.trip = trip; // Assign trip
10        this.seatNumber = seatNumber; // Assign seat number
11        this.username = username; // Assign username
12    }
13
14    @Override
15    public void execute() {
16        boolean success = trip.reserveSeat(seatNumber, username); // Try to reserve the seat
17        if (success) {
18            System.out.println("Seat successfully reserved: " + seatNumber); // Reservation success
19        } else {
20            System.out.println("Seat could not be reserved: " + seatNumber); // Reservation failed
21        }
22    }
23 }
24
```

```
Command.java x SeatReserveCommand.java x CancelReservationCommand.java x
Source History
1 package aoopproject1;
2
3 public class CancelReservationCommand implements Command {
4     private Trip trip; // The trip where the reservation exists
5     private int seatNumber; // Seat number to cancel
6     private String username; // Username of the person who made the reservation
7
8     public CancelReservationCommand(Trip trip, int seatNumber, String username) {
9         this.trip = trip; // Assign trip
10        this.seatNumber = seatNumber; // Assign seat number
11        this.username = username; // Assign username
12    }
13
14    @Override
15    public void execute() {
16        boolean success = trip.cancelSeat(seatNumber, username); // Attempt to cancel the seat
17        if (success) {
18            System.out.println("Seat cancellation successful: " + seatNumber); // Success message
19        } else {
20            System.out.println("Cancellation failed (either owner mismatch or already empty): " + seatNumber); // Failure mes:
21        }
22    }
23 }
24
```

-The Command Pattern turns a request into a stand-alone object that contains all the information about the request. It allows you to parameterize methods with different requests, delay execution, or queue a request.

-The Command Pattern encapsulates reservation and cancellation actions into command objects. **SeatReserveCommand** and **CancelReservationCommand** implement the **Command** interface and execute specific actions on the **Trip** object. This decouples the invoker (UI components) from the actual operation logic, improving maintainability and extensibility.

3.4-Observer Pattern

```
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
1 package aoopproject1;
2
3 // Observer interface for implementing the Observer pattern
4 public interface Observer {
5     void update(); // Called when Observable notifies observers
6 }
7
```

```
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
1 package aoopproject1;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public abstract class Observable {
7     private List<Observer> observers = new ArrayList<>(); // List of observers
8
9     public void addObserver(Observer o) {
10         observers.add(o); // Add observer to the list
11     }
12
13     public void removeObserver(Observer o) {
14         observers.remove(o); // Remove observer from the list
15     }
16
17     public void notifyObservers() {
18         for (Observer o : observers) {
19             o.update(); // Notify each observer
20         }
21     }
22 }
23
```

```
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
1 package aoopproject1;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Trip extends Observable {
7     private String id, departure, arrival, date, type; // Trip details
8     private List<Boolean> seats; // true = reserved, false = available
9     private List<String> reservationOwners; // Username per seat
10     private boolean isOpen; // Is trip open for reservation
11     private String vehicleType; // Bus or plane
12
13     public Trip(String id, String departure, String arrival, String date, int seatCount, String type) {
14         this.id = id;
15         this.departure = departure;
16         this.arrival = arrival;
17         this.date = date;
18         this.type = type;
19         this.seats = new ArrayList<>();
20         this.reservationOwners = new ArrayList<>();
21         this.isOpen = true; // By default, trip is open
22
23         for (int i = 0; i < seatCount; i++) {
24             seats.add(false); // All seats initially empty
25             reservationOwners.add(""); // No owners at start
26         }
27     }
28
29     public boolean reserveSeat(int number, String username) {
30         if (!isOpen) return false; // Cannot reserve if trip is closed
31         if (!seats.get(number)) {
32             seats.set(number, true); // Mark seat as taken
33             reservationOwners.set(number, username); // Set owner
34             notifyObservers(); // Update UI
35             return true;
36         }
37         return false;
38     }
39
40     public boolean cancelSeat(int number, String username) {
41         if (!isOpen) return false; // Cannot cancel if trip is closed
42         if (seats.get(number) && reservationOwners.get(number).equals(username)) {
43             seats.set(number, false); // Mark seat as available
44             reservationOwners.set(number, ""); // Clear owner
45             notifyObservers(); // Update UI
46             return true;
47         }
48         return false;
49     }
50 }
```

```

49     }
50
51     public String getReservationOwner(int number) {
52         return reservationOwners.get(number); // Get username for a seat
53     }
54
55     public boolean isOpen() {
56         return isOpen; // Check if trip is open
57     }
58
59     public void setOpen(boolean open) {
60         this.isOpen = open; // Set trip status
61     }
62
63     public String getId() { return id; } // Trip ID
64     public String getDeparture() { return departure; } // Departure location
65     public String getArrival() { return arrival; } // Arrival location
66     public String getDate() { return date; } // Date of trip
67     public String getType() { return type; } // "bus" or "plane"
68     public List<Boolean> getSeats() { return seats; } // Seat status list
69
70     public String getVehicleType() {
71         return vehicleType; // Get vehicle type
72     }
73
74     public void setVehicleType(String vehicleType) {
75         this.vehicleType = vehicleType; // Set vehicle type
76     }
77 }
78

```

```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x aoopproject1 x
Source History
1 package aoopproject1;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class TripManager extends Observable {
7     private List<Trip> tripList = new ArrayList<>(); // List of all trips
8
9     public void addTrip(Trip trip) {
10         tripList.add(trip); // Add new trip to the list
11         notifyObservers(); // Notify observers (e.g., UI)
12     }
13
14     public boolean deleteTrip(String id) {
15         Trip target = null;
16         for (Trip t : tripList) {
17             if (t.getId().equalsIgnoreCase(id)) {
18                 target = t; // Find trip by ID (case-insensitive)
19                 break;
20             }
21         }
22         if (target != null) {
23             tripList.remove(target); // Remove trip
24             notifyObservers(); // Notify UI
25             return true;
26         }
27         return false; // Trip not found
28     }
29
30     public Trip findTrip(String id) {
31         for (Trip t : tripList) {
32             if (t.getId().equalsIgnoreCase(id)) {
33                 return t; // Return trip if found
34             }
35         }
36         return null; // Trip not found
37     }
38
39     public List<Trip> getAllTrips() {
40         return new ArrayList<>(tripList); // Return copy of trip list
41     }
42 }
43

```

```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x aoopproject1 x
Source History
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.*;
6
7 public class SeatSelectionDialog extends JDialog implements Observer {
8     private int emojiX = -50; // Animation position for emoji
9     private Timer animationTimer; // Timer for animation
10    private Trip trip; // Current trip
11    private String username; // Logged-in user
12
13    public SeatSelectionDialog(JFrame parent, Trip trip, String username) {
14        super(parent, "Seat Selection - Trip ID: " + trip.getId(), true);
15        this.trip = trip;
16        this.username = username;
17        trip.addObserver(this); // Subscribe to trip updates
18        setSize(800, 600);
19        setLocationRelativeTo(parent); // Center the dialog
20
21        Color background = new Color(239, 250, 231);
22        JPanel panel = new JPanel(new BorderLayout());
23        panel.setBackground(background);
24        panel.setBorder(BorderFactory.createEmptyBorder(0, 20, 20, 20));
25
26        JPanel animationPanel = new JPanel() {
27            @Override
28            protected void paintComponent(Graphics g) {
29                super.paintComponent(g);
30                if (trip.getType().equalsIgnoreCase("plane")) {
31                    setBackground(new Color(200, 230, 255)); // Sky color
32                } else {
33                    setBackground(new Color(245, 235, 200)); // Road color
34                }
35                g.setFont(new Font("Segoe UI", Font.PLAIN, 28));
36                String emoji = trip.getType().equalsIgnoreCase("plane") ? "✈️" : "🚌"; // Emoji based on type
37
38                g.setColor(new Color(0, 0, 0, 40));
39                g.drawString(emoji, emojiX + 2, 37);
40                g.setColor(Color.BLACK);
41                g.drawString(emoji, emojiX, 35);
42            }
43        };
44        animationPanel.setPreferredSize(new Dimension(800, 50));
45
46        animationTimer = new Timer(20, e -> {
47            emojiX += 2;
48            if (emojiX > getWidth()) emojiX = -50; // Reset emoji position
49        });
50        animationPanel.repaint();
51        animationTimer.start(); // Start animation
52
53        JPanel northPanel = new JPanel(new BorderLayout());
54        northPanel.setBackground(background);
55        northPanel.add(animationPanel, BorderLayout.CENTER);
56
57        JLabel title = new JLabel("Please select a seat", JLabel.CENTER); // Title
58        title.setFont(new Font("Segoe UI", Font.BOLD, 18));
59        title.setForeground(new Color(30, 30, 30));
60        northPanel.add(title, BorderLayout.SOUTH);
61        panel.add(northPanel, BorderLayout.NORTH);
62
63        // Select strategy based on trip type (only plane or bus exist)
64        SeatArrangementStrategy strategy = trip.getType().equalsIgnoreCase("plane")
65            ? new PlaneSeatLayoutStrategy()
66            : new BusSeatLayoutStrategy();
67
68        JPanel gridWrapper = strategy.createSeatPanel(trip, username, this); // Create seat layout
69
70        JScrollPane scroll = new JScrollPane(gridWrapper); // Scrollable seat view
71        scroll.setBorder(BorderFactory.createEmptyBorder());
72        scroll.getViewport().setBackground(background);
73        scroll.setBackground(background);
74        scroll.getVerticalScrollBar().setUnitIncrement(16);
75
76        panel.add(scroll, BorderLayout.CENTER);
77        add(panel); // Add main panel to dialog
78
79        addWindowListener(new WindowAdapter() {
80            @Override
81            public void windowClosing(WindowEvent e) {
82                if (animationTimer != null) animationTimer.stop(); // Stop animation on close
83                trip.removeObserver(SeatSelectionDialog.this); // Unsubscribe from trip
84            }
85        });
86
87        setVisible(true); // Show dialog
88        initUI(); // Initialize UI
89    }
90
91    @Override
92    public void update() {
93        getContentPane().removeAll(); // Clear UI
94        initUI(); // Rebuild UI
95        revalidate();
96        repaint();
97    }
98

```



```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x aoopproject1 x
Source History
97
98
99 public void initView() {
100     getContentPane().removeAll(); // Clear previous content
101     Color background = new Color(239, 250, 231);
102
103     JPanel panel = new JPanel(new BorderLayout()); // Main container
104     panel.setBackground(background);
105     panel.setBorder(BorderFactory.createEmptyBorder(0, 20, 20, 20));
106
107     JPanel animationPanel = new JPanel() {
108         @Override
109         protected void paintComponent(Graphics g) {
110             super.paintComponent(g);
111             if (trip.getType().equalsIgnoreCase("plane")) {
112                 setBackground(new Color(200, 230, 255)); // Sky color
113             } else {
114                 setBackground(new Color(245, 235, 200)); // Road color
115             }
116             g.setFont(new Font("Segoe UI", Font.PLAIN, 28));
117             String emoji = trip.getType().equalsIgnoreCase("plane") ? "✈️" : "🚗"; // Emoji
118             g.setColor(new Color(0, 0, 0, 40));
119             g.drawString(emoji, emojiX + 2, 37);
120             g.setColor(Color.BLACK);
121             g.drawString(emoji, emojiX, 35);
122         }
123     };
124     animationPanel.setPreferredSize(new Dimension(800, 50));
125
126     animationTimer = new Timer(20, e -> {
127         emojiX += 2;
128         if (emojiX > getWidth()) emojiX = -50;
129         animationPanel.repaint();
130     });
131     animationTimer.start(); // Restart animation
132
133     JPanel northPanel = new JPanel(new BorderLayout());
134     northPanel.setBackground(background);
135     northPanel.add(animationPanel, BorderLayout.CENTER);
136
137     JLabel title = new JLabel("Please select a seat", JLabel.CENTER); // Title label
138     title.setFont(new Font("Segoe UI", Font.BOLD, 18));
139     title.setForeground(new Color(30, 30, 30));
140     northPanel.add(title, BorderLayout.SOUTH);
141     panel.add(northPanel, BorderLayout.NORTH);
142
143     SeatArrangementStrategy strategy = trip.getType().equalsIgnoreCase("plane")
144         ? new PlaneSeatLayoutStrategy()
145         : new BusSeatLayoutStrategy();
146
147     JPanel gridWrapper = strategy.createSeatPanel(trip, username, this); // Seat layout
148     JScrollPane scroll = new JScrollPane(gridWrapper); // Scroll view
149     scroll.setBorder(BorderFactory.createEmptyBorder());
150     scroll.getViewport().setBackground(background);
151     scroll.setBackground(background);
152     scroll.getVerticalScrollBar().setUnitIncrement(16);
153
154     panel.add(scroll, BorderLayout.CENTER);
155     setContentPane(panel); // Apply new content
156     revalidate();
157     repaint();
158 }
159
160

```

```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.MouseAdapter;
6 import java.awt.event.MouseEvent;
7 import java.text.SimpleDateFormat;
8 import java.util.Comparator;
9 import java.util.List;
10 import java.util.stream.Collectors;
11
12 public class UserFrame extends JFrame implements Observer {
13     private final TripManager tripManager; // Central trip manager (Observable)
14     private final List<User> users; // List of all users in the system
15     private final String[] selectedType = new String[1]; // Transport type filter (bus or plane)
16     private final String loggedInUsername; // Currently logged-in user
17
18     private final JPanel tripListPanel = new JPanel(); // Container for trip cards
19     private final JScrollPane scrollPane = new JScrollPane(tripListPanel); // Scrollable wrapper
20
21     public UserFrame(TripManager tripManager, List<User> users, String initialType, String username) {
22         this.tripManager = tripManager;
23         this.users = users;
24         this.selectedType[0] = initialType;
25         this.loggedInUsername = username;
26
27         // Register this frame as an observer to TripManager
28         this.tripManager.addObserver(this);
29
30         // Frame settings
31         setTitle("User Panel - " + initialType.toUpperCase() + " Trips");
32         setSize(1000, 600);
33         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34         setLocationRelativeTo(null);
35         setLayout(new BorderLayout());
36
37         // UI colors
38         Color background = new Color(239, 250, 231);
39         Color green = new Color(107, 163, 104);
40         Color greenHover = new Color(87, 143, 84);
41         Color textColor = new Color(40, 40, 40);
42
43         // Header title
44         JLabel header = new JLabel("🚌 Available Trips", JLabel.CENTER);
45         header.setFont(new Font("Segoe UI", Font.BOLD, 22));
46         header.setForeground(textColor);
47         header.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
48         header.setBackground(background);
49
50         header.setOpaque(true);
51         add(header, BorderLayout.NORTH);
52
53         // Main panel for trips
54         tripListPanel.setLayout(new BoxLayout(tripListPanel, BoxLayout.Y_AXIS));
55         tripListPanel.setBackground(background);
56         tripListPanel.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));
57
58         // Scroll wrapper
59         scrollPane.setBorder(BorderFactory.createEmptyBorder());
60         scrollPane.setBackground(background);
61         scrollPane.getViewport().setBackground(background);
62         scrollPane.getVerticalScrollBar().setUnitIncrement(16);
63         scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
64         scrollPane.setPreferredSize(new Dimension(960, 450));
65         add(scrollPane, BorderLayout.CENTER);
66
67         // Main action buttons
68         JButton reserveBtn = new JButton("Reserve / Cancel");
69         JButton searchBtn = new JButton("Search Trip by ID");
70         JButton changeTypeBtn = new JButton("Change Transport Type");
71         JButton logoutBtn = new JButton("Logout");
72
73         // Set visual and interaction styles for buttons
74         JButton[] buttons = { reserveBtn, searchBtn, changeTypeBtn, logoutBtn };
75         for (JButton btn : buttons) {
76             btn.setFont(new Font("Segoe UI", Font.BOLD, 14));
77             btn.setBackground(green);
78             btn.setForeground(Color.WHITE);
79             btn.setOpaque(true);
80             btn.setBorderPainted(false);
81             btn.setFocusPainted(false);
82             btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
83             btn.addMouseListener(new MouseAdapter() {
84                 public void mouseEntered(MouseEvent e) { btn.setBackground(greenHover); }
85                 public void mouseExited(MouseEvent e) { btn.setBackground(green); }
86             });
87         }
88
89         // Action: reservation guidance
90         reserveBtn.addActionListener(e -> {
91             JOptionPane.showMessageDialog(this, "Please click on a trip card.");
92         });
93
94         // Action: trip search by ID
95         searchBtn.addActionListener(e -> {
96             String input = JOptionPane.showInputDialog(this, "Enter Trip ID:");
97             if (input != null && !input.trim().isEmpty()) {
98                 Trip t = tripManager.findTrip(input.trim());
99             }
100         });
101     }
102
103     // Observer methods
104     public void update() {
105         // Refresh trip list
106     }
107
108     public void transportTypeChanged(String type) {
109         // Refresh trip list
110     }
111
112     public void userLoggedIn(String username) {
113         // Refresh trip list
114     }
115
116     public void userLoggedOut() {
117         // Refresh trip list
118     }
119 }

```

```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
97 if (t != null && t.getType().equalsIgnoreCase(selectedType[0])) {
98     // Show trip details and allow seat reservation
99     JOptionPane.showMessageDialog(this,
100         "Trip Detail:\n" +
101         "ID: " + t.getId() + "\n" +
102         "Departure: " + t.getDeparture() + "\n" +
103         "Arrival: " + t.getArrival() + "\n" +
104         "Date: " + t.getDate(),
105         "Trip Info", JOptionPane.INFORMATION_MESSAGE);
106     int confirm = JOptionPane.showConfirmDialog(this,
107         "Would you like to reserve a seat for this trip?",
108         "Confirm", JOptionPane.YES_NO_OPTION);
109     if (confirm == JOptionPane.YES_OPTION) {
110         new SeatSelectionDialog(this, t, loggedInUsername);
111         refreshTripList();
112     }
113     } else {
114         JOptionPane.showMessageDialog(this, "X Trip not found or type mismatch.");
115     }
116 }
117 }
118
119 // Action: transport type switching
120 changeTypeBtn.addActionListener(e -> {
121     String newType = showTypeSelectionDialog();
122     if (newType != null && !newType.equalsIgnoreCase(selectedType[0])) {
123         selectedType[0] = newType;
124         setTitle("User Panel - " + newType.toUpperCase() + " Trips");
125         header.setText("Available " + newType.toUpperCase() + " Trips");
126         refreshTripList();
127     }
128 }
129
130 // Action: logout
131 logoutBtn.addActionListener(e -> {
132     dispose(); // Close current frame
133     new LoginFrame(users, tripManager); // Return to login screen
134 });
135
136 // Add buttons to panel
137 JPanel buttonPanel = new JPanel();
138 buttonPanel.setBackground(background);
139 buttonPanel.add(reserveBtn);
140 buttonPanel.add(searchBtn);
141 buttonPanel.add(changeTypeBtn);
142 buttonPanel.add(logoutBtn);
143 add(buttonPanel, BorderLayout.SOUTH);
144
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
145 refreshTripList(); // Initial UI load
146 setVisible(true);
147
148 // Remove observer on close
149 addWindowListener(new java.awt.event.WindowAdapter() {
150     @Override
151     public void windowClosing(java.awt.event.WindowEvent e) {
152         tripManager.removeObserver(UserFrame.this);
153     }
154 });
155
156 @Override
157 public void update() {
158     refreshTripList();
159 }
160
161 private String showTypeSelectionDialog() {
162     JDialog dialog = new JDialog(this, "Select Transport Type", true);
163     dialog.setSize(320, 200);
164     dialog.setLocationRelativeTo(this);
165     dialog.setLayout(new BorderLayout());
166
167     // Style and colors
168     Color background = new Color(239, 250, 231);
169     Color pastelYellow = new Color(255, 236, 179);
170     Color pastelBlue = new Color(179, 229, 252);
171     dialog.getContentPane().setBackground(background);
172
173     // Title
174     JLabel label = new JLabel("Please select transport type:", JLabel.CENTER);
175     label.setFont(new Font("Segoe UI", Font.BOLD, 16));
176     label.setForeground(new Color(40, 40, 40));
177     label.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
178     dialog.add(label, BorderLayout.NORTH);
179
180     // Buttons for type selection
181     JPanel buttonPanel = new JPanel(new GridLayout(1, 2, 20, 10));
182     buttonPanel.setBorder(BorderFactory.createEmptyBorder(20, 30, 20, 30));
183     buttonPanel.setBackground(background);
184
185     JButton busBtn = new JButton("Bus");
186     JButton planeBtn = new JButton("Plane");
187     busBtn.setBackground(pastelYellow);
188     planeBtn.setBackground(pastelBlue);
189     busBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
190     planeBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
191
192

```



```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
193 final String[] selected = {null};
194
195 busBtn.addActionListener(e -> {
196     selected[0] = "bus";
197     dialog.dispose();
198 });
199
200 planeBtn.addActionListener(e -> {
201     selected[0] = "plane";
202     dialog.dispose();
203 });
204
205 buttonPanel.add(busBtn);
206 buttonPanel.add(planeBtn);
207 dialog.add(buttonPanel, BorderLayout.CENTER);
208 dialog.setVisible(true);
209
210 return selected[0];
211 }
212
213 private void refreshTripList() {
214     tripListPanel.removeAll(); // Clear existing cards
215     SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
216
217     // Filter and sort trips by date
218     List<Trip> sortedTrips = tripManager.getAllTrips().stream()
219         .filter(t -> t.getType().equalsIgnoreCase(selectedType[0]))
220         .sorted(Comparator.comparing(t -> {
221             try {
222                 return sdf.parse(t.getDate());
223             } catch (Exception e) {
224                 return new java.util.Date();
225             }
226         }))
227         .collect(Collectors.toList());
228
229     // Display each trip as a card
230     for (Trip t : sortedTrips) {
231         String emoji = t.getType().equalsIgnoreCase("plane") ? "✈️" : "🚌";
232         int total = t.getSeats().size();
233         long occupied = t.getSeats().stream().filter(k -> k).count();
234         long available = total - occupied;
235
236         JPanel card = new JPanel(new GridLayout(2, 1));
237         card.setPreferredSize(new Dimension(920, 80));
238         card.setBackground(Color.WHITE);
239         card.setBorder(BorderFactory.createCompoundBorder(
240             BorderFactory.createLineBorder(new Color(200, 200, 200)),
241             BorderFactory.createEmptyBorder(10, 15, 10, 15)
242         ));
243
244         JLabel title = new JLabel(emoji + " " + t.getDeparture() + " → " + t.getArrival() + " | " + t.getDate());
245         JLabel info = new JLabel("Available: " + available + " | Occupied: " + occupied + " | ID: " + t.getId());
246         title.setFont(new Font("Segoe UI", Font.BOLD, 16));
247         info.setFont(new Font("Segoe UI", Font.PLAIN, 14));
248         info.setForeground(new Color(70, 70, 70));
249
250         card.add(title);
251         card.add(info);
252         card.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
253
254         // Click to open seat selection
255         card.addMouseListener(new MouseAdapter() {
256             public void mouseClicked(MouseEvent evt) {
257                 new SeatSelectionDialog(UserFrame.this, t, loggedInUsername);
258                 refreshTripList(); // Refresh UI after reservation
259             }
260         });
261
262         tripListPanel.add(card);
263         tripListPanel.add(Box.createRigidArea(new Dimension(0, 12))); // Space
264     }
265
266     // Section for showing user's reserved seats
267     JLabel rLabel = new JLabel("✳️ Your Reserved Seats:");
268     rLabel.setFont(new Font("Segoe UI", Font.BOLD, 16));
269     rLabel.setBorder(BorderFactory.createEmptyBorder(10, 0, 5, 0));
270     tripListPanel.add(rLabel);
271
272     // Loop through all trips and find user's reservations
273     for (Trip t : tripManager.getAllTrips()) {
274         List<Boolean> seats = t.getSeats();
275         for (int i = 0; i < seats.size(); i++) {
276             if (seats.get(i) && loggedInUsername.equals(t.getReservationOwner(i))) {
277                 int finalI = i;
278                 JPanel reservationRow = new JPanel(new FlowLayout(FlowLayout.LEFT));
279                 reservationRow.setBackground(new Color(250, 250, 240));
280
281                 JLabel info = new JLabel("Trip ID: " + t.getId() + ", Date: " + t.getDate() + ", Seat No: " + (i + 1));
282                 info.setFont(new Font("Segoe UI", Font.PLAIN, 13));
283
284                 JButton cancelBtn = new JButton("Cancel");
285                 cancelBtn.setBackground(new Color(204, 77, 77));
286                 cancelBtn.setForeground(Color.WHITE);
287                 cancelBtn.setFont(new Font("Segoe UI", Font.PLAIN, 12));
288

```



```

289 // Cancel reservation with command
290 cancelBtn.addActionListener(e -> {
291     Command cancelCommand = new CancelReservationCommand(t, finalI, loggedInUsername);
292     cancelCommand.execute();
293     JOptionPane.showMessageDialog(this, "Reservation canceled.");
294     refreshTripList();
295 });
296
297 reservationRow.add(info);
298 reservationRow.add(cancelBtn);
299 tripListPanel.add(reservationRow);
300
301 }
302
303 }
304
305 tripListPanel.revalidate();
306 tripListPanel.repaint();
307
308 }

```

-The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

-The Observer Pattern is used to notify UI components when data changes occur in trip or seat reservations. Classes like **Trip** and **TripManager** extend **Observable**, and GUI components such as **SeatSelectionDialog** and **UserFrame** implement **Observer**. Whenever a seat is reserved or canceled, observers are automatically updated to reflect the latest state in the interface.

4-SCREENSHOTS AND DESCRIPTION OF CLASSES

4.1-Admin Class

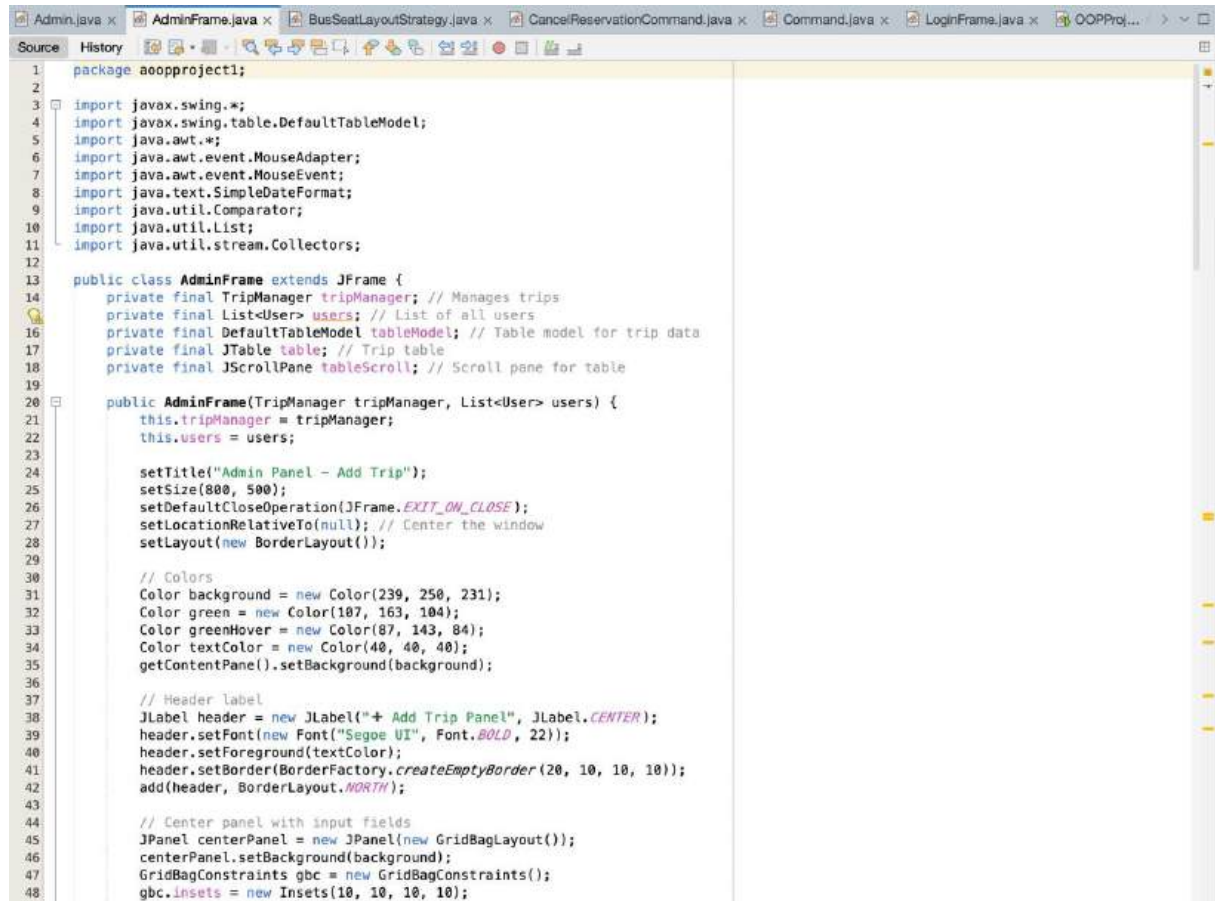


```
1 package aoopproject1;
2
3 public class Admin extends User {
4
5     public Admin(String username, String password) {
6         super(username, password); // Call parent constructor to set username and password
7     }
8
9 }
```

The `Admin` class extends `User` to represent admin users. It reuses the `User` constructor and can support admin-specific features later.

- **Purpose:** Represents admin role.
- **OOP Principle:** Inheritance.
- **Design Pattern:** Used with the **Factory Pattern** via `UserFactory`.
- **Used in:** `RegisterDialog`, `UserFactory`.

4.2-AdminFrame Class

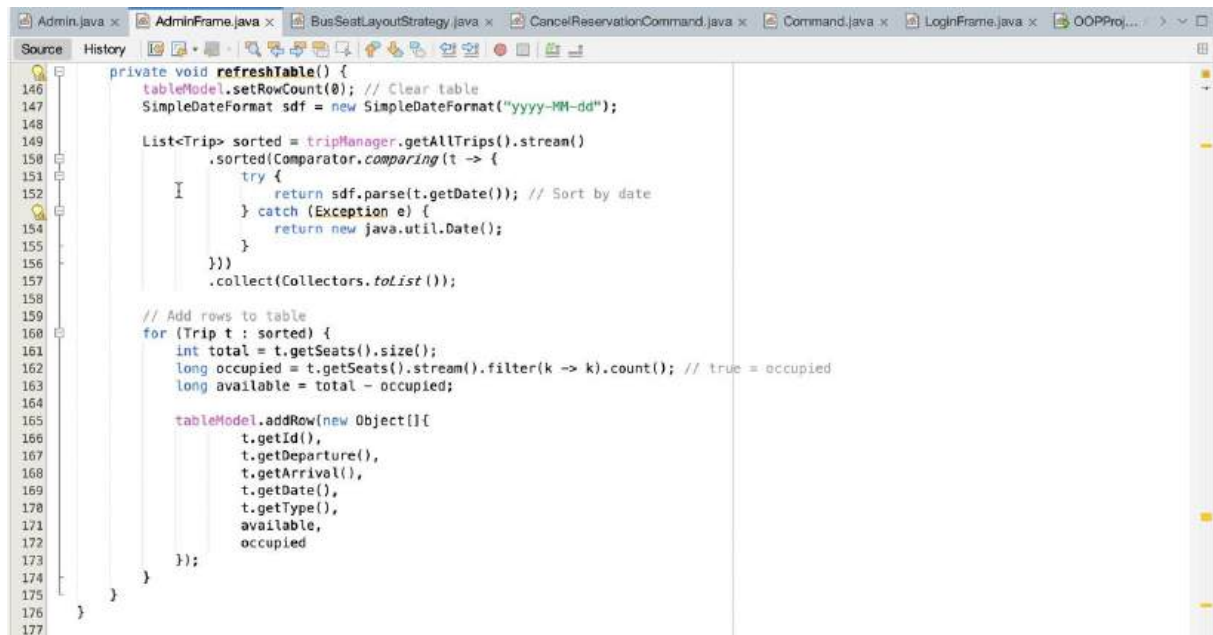


```
1 package aoopproject1;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import java.awt.*;
6 import java.awt.event.MouseAdapter;
7 import java.awt.event.MouseEvent;
8 import java.text.SimpleDateFormat;
9 import java.util.Comparator;
10 import java.util.List;
11 import java.util.stream.Collectors;
12
13 public class AdminFrame extends JFrame {
14     private final TripManager tripManager; // Manages trips
15     private final List<User> users; // List of all users
16     private final DefaultTableModel tableModel; // Table model for trip data
17     private final JTable table; // Trip table
18     private final JScrollPane tableScroll; // Scroll pane for table
19
20     public AdminFrame(TripManager tripManager, List<User> users) {
21         this.tripManager = tripManager;
22         this.users = users;
23
24         setTitle("Admin Panel - Add Trip");
25         setSize(800, 500);
26         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
27         setLocationRelativeTo(null); // Center the window
28         setLayout(new BorderLayout());
29
30         // Colors
31         Color background = new Color(239, 250, 231);
32         Color green = new Color(107, 163, 104);
33         Color greenHover = new Color(87, 143, 84);
34         Color textColor = new Color(40, 40, 40);
35         getContentPane().setBackground(background);
36
37         // Header Label
38         JLabel header = new JLabel("+ Add Trip Panel", JLabel.CENTER);
39         header.setFont(new Font("Segoe UI", Font.BOLD, 22));
40         header.setForeground(textColor);
41         header.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
42         add(header, BorderLayout.NORTH);
43
44         // Center panel with input fields
45         JPanel centerPanel = new JPanel(new GridBagLayout());
46         centerPanel.setBackground(background);
47         GridBagConstraints gbc = new GridBagConstraints();
48         gbc.insets = new Insets(10, 10, 10, 10);
```

```

Admin.java x AdminFrame.java x BusSeatLayoutStrategy.java x CancelReservationCommand.java x Command.java x LoginFrame.java x OOPProj...
Source History
49 gbc.fill = GridBagConstraints.HORIZONTAL;
50 gbc.gridx = 0;
51
52 // Input fields
53 JTextField idField = new JTextField(15);
54 JTextField departureField = new JTextField(15);
55 JTextField arrivalField = new JTextField(15);
56 JTextField dateField = new JTextField(15);
57 JTextField seatCountField = new JTextField(15);
58 JComboBox<String> typeCombo = new JComboBox<>{new String[] {"bus", "plane"}};
59
60 // Labels and fields
61 String[] labels = { "Trip ID:", "Departure:", "Arrival:", "Date (yyyy-MM-dd):", "Seat Count:", "Type:" };
62 Component[] fields = { idField, departureField, arrivalField, dateField, seatCountField, typeCombo };
63
64 for (int i = 0; i < labels.length; i++) {
65     gbc.gridy = i;
66     centerPanel.add(new JLabel(labels[i]), gbc); // Label
67     gbc.gridy = i;
68     gbc.gridx = 1;
69     centerPanel.add(fields[i], gbc); // Field
70     gbc.gridx = 0;
71 }
72
73 add(centerPanel, BorderLayout.CENTER);
74
75 // Trip table
76 String[] columns = { "ID", "Departure", "Arrival", "Date", "Type", "Available", "Occupied" };
77 tableModel = new DefaultTableModel(columns, 0);
78 table = new JTable(tableModel);
79 tableScroll = new JScrollPane(table);
80 tableScroll.setPreferredSize(new Dimension(780, 200));
81 tableScroll.setVisible(false); // Hidden initially
82 add(tableScroll, BorderLayout.EAST);
83
84 // Bottom panel with buttons
85 JPanel bottomPanel = new JPanel();
86 bottomPanel.setBackground(background);
87 JButton addBtn = new JButton("Add Trip");
88 JButton logoutBtn = new JButton("Logout");
89 JButton listBtn = new JButton("View Trips");
90
91 JButton[] buttons = { addBtn, logoutBtn, listBtn };
92 for (JButton btn : buttons) {
93     btn.setFont(new Font("Segoe UI", Font.BOLD, 13));
94     btn.setBackground(green);
95     btn.setForeground(Color.WHITE);
96     btn.setOpaque(true);
97
98     btn.setBorderPainted(false);
99     btn.setFocusPainted(false);
100     btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
101     // Hover effect
102     btn.addMouseListener(new MouseAdapter() {
103         public void mouseEntered(MouseEvent e) { btn.setBackground(greenHover); }
104         public void mouseExited(MouseEvent e) { btn.setBackground(green); }
105     });
106     bottomPanel.add(btn);
107 }
108
109 // Add Trip button action
110 addBtn.addActionListener(e -> {
111     try {
112         Trip newTrip = new Trip(
113             idField.getText().trim(),
114             departureField.getText().trim(),
115             arrivalField.getText().trim(),
116             dateField.getText().trim(),
117             Integer.parseInt(seatCountField.getText().trim()),
118             (String) typeCombo.getSelectedItem()
119         );
120         tripManager.addTrip(newTrip); // Add trip to manager
121         JOptionPane.showMessageDialog(this, "Trip added.");
122         // Clear fields
123         idField.setText(""); departureField.setText(""); arrivalField.setText("");
124         dateField.setText(""); seatCountField.setText("");
125     } catch (Exception ex) {
126         JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
127     }
128 });
129
130 // Logout button action
131 logoutBtn.addActionListener(e -> {
132     dispose(); // Close current window
133     new LoginFrame(users, tripManager); // Open login screen
134 });
135
136 // View trips button action
137 listBtn.addActionListener(e -> {
138     TripListDialog dialog = new TripListDialog(this, tripManager); // Show trip list dialog
139     dialog.setVisible(true);
140 });
141
142 add(bottomPanel, BorderLayout.SOUTH);
143 setVisible(true); // Show frame
144 }

```



```
146 private void refreshTable() {
147     tableModel.setRowCount(0); // Clear table
148     SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
149
150     List<Trip> sorted = tripManager.getAllTrips().stream()
151         .sorted(Comparator.comparing(t -> {
152             try {
153                 return sdf.parse(t.getDate()); // Sort by date
154             } catch (Exception e) {
155                 return new java.util.Date();
156             }
157         }))
158         .collect(Collectors.toList());
159
160     // Add rows to table
161     for (Trip t : sorted) {
162         int total = t.getSeats().size();
163         long occupied = t.getSeats().stream().filter(k -> k).count(); // true = occupied
164         long available = total - occupied;
165
166         tableModel.addRow(new Object[]{
167             t.getId(),
168             t.getDeparture(),
169             t.getArrival(),
170             t.getDate(),
171             t.getType(),
172             available,
173             occupied
174         });
175     }
176 }
177 }
```

Purpose:

Provides a graphical user interface for administrators to add new trips, view existing ones, and manage trip data.

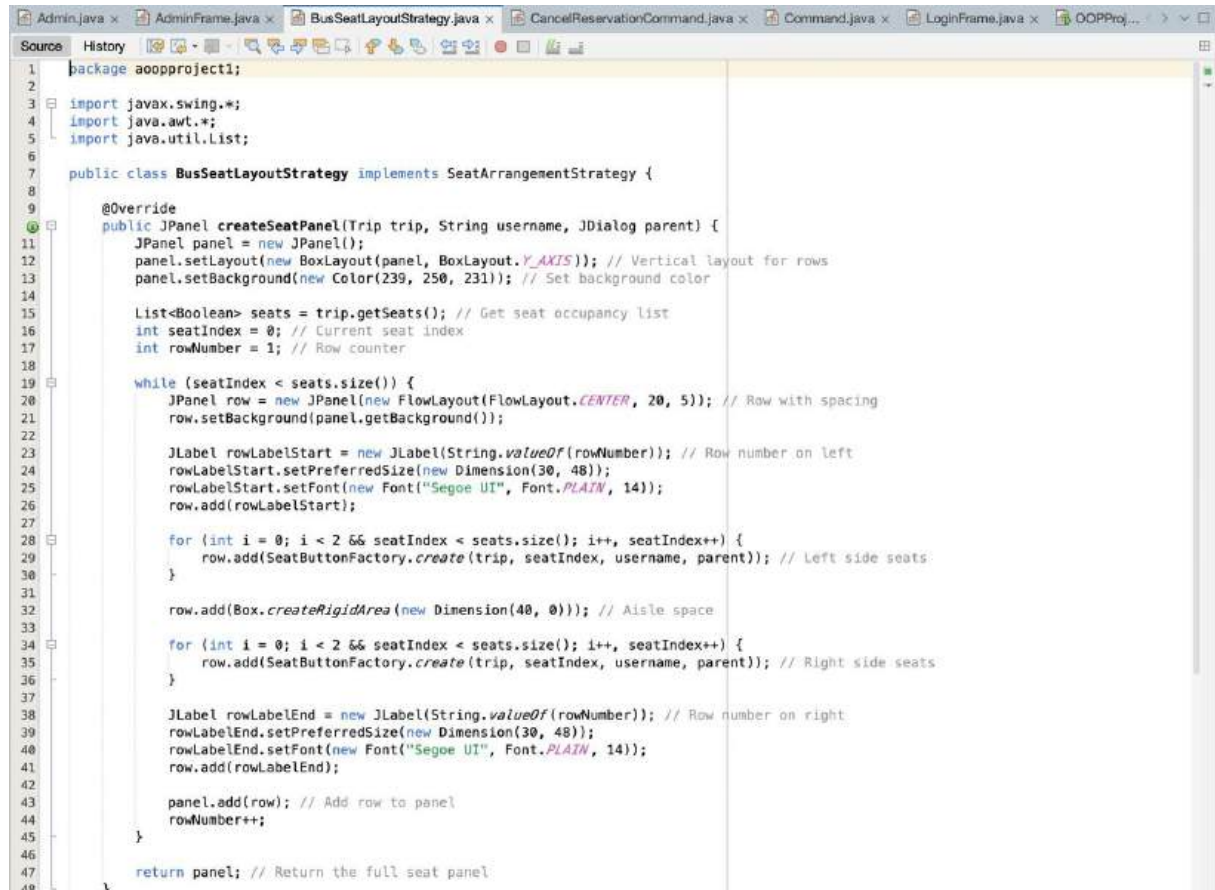
Key Responsibilities:

- Allows admins to input trip details (ID, route, date, seat count, type) and add them to the system.
- Displays a table of current trips using a `JTable` with filtering and sorting features.
- Enables access to a full trip list dialog (`TripListDialog`).
- Offers logout functionality to return to the login screen.

Usage:

Instantiated when an admin logs in. Serves as the central interface for trip management tasks.

4.3-BusSeatLayoutStrategy Class



```
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.List;
6
7 public class BusSeatLayoutStrategy implements SeatArrangementStrategy {
8
9     @Override
10    public JPanel createSeatPanel(Trip trip, String username, JDialog parent) {
11        JPanel panel = new JPanel();
12        panel.setLayout(new BorderLayout()); // Vertical layout for rows
13        panel.setBackground(new Color(239, 250, 231)); // Set background color
14
15        List<Boolean> seats = trip.getSeats(); // Get seat occupancy list
16        int seatIndex = 0; // Current seat index
17        int rowNumber = 1; // Row counter
18
19        while (seatIndex < seats.size()) {
20            JPanel row = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 5)); // Row with spacing
21            row.setBackground(panel.getBackground());
22
23            JLabel rowLabelStart = new JLabel(String.valueOf(rowNumber)); // Row number on left
24            rowLabelStart.setPreferredSize(new Dimension(30, 48));
25            rowLabelStart.setFont(new Font("Segoe UI", Font.PLAIN, 14));
26            row.add(rowLabelStart);
27
28            for (int i = 0; i < 2 && seatIndex < seats.size(); i++, seatIndex++) {
29                row.add(SeatButtonFactory.create(trip, seatIndex, username, parent)); // Left side seats
30            }
31
32            row.add(Box.createRigidArea(new Dimension(40, 0))); // Aisle space
33
34            for (int i = 0; i < 2 && seatIndex < seats.size(); i++, seatIndex++) {
35                row.add(SeatButtonFactory.create(trip, seatIndex, username, parent)); // Right side seats
36            }
37
38            JLabel rowLabelEnd = new JLabel(String.valueOf(rowNumber)); // Row number on right
39            rowLabelEnd.setPreferredSize(new Dimension(30, 48));
40            rowLabelEnd.setFont(new Font("Segoe UI", Font.PLAIN, 14));
41            row.add(rowLabelEnd);
42
43            panel.add(row); // Add row to panel
44            rowNumber++;
45        }
46
47        return panel; // Return the full seat panel
48    }
49 }
```

Design Pattern Used: Strategy Pattern

Implements: SeatArrangementStrategy

Purpose:

Defines the seat arrangement layout for bus trips. It dynamically generates a visual seat layout in a 2+2 row format with an aisle in between.

Key Responsibilities:

- Creates a vertical layout (BoxLayout) where each row has 4 seats and row numbers on both ends.
- Uses SeatButtonFactory to generate seat buttons for interaction.
- Highlights bus-specific seating logic separated from other vehicle types (e.g., planes).

Usage:

Called during seat selection to display the appropriate layout for bus-type trips.

4.4-CancelReservationCommand Class



```
1 package aoopproject1;
2
3 public class CancelReservationCommand implements Command {
4     private Trip trip; // The trip where the reservation exists
5     private int seatNumber; // Seat number to cancel
6     private String username; // Username of the person who made the reservation
7
8     public CancelReservationCommand(Trip trip, int seatNumber, String username) {
9         this.trip = trip; // Assign trip
10        this.seatNumber = seatNumber; // Assign seat number
11        this.username = username; // Assign username
12    }
13
14    @Override
15    public void execute() {
16        boolean success = trip.cancelSeat(seatNumber, username); // Attempt to cancel the seat
17        if (success) {
18            System.out.println("Seat cancellation successful: " + seatNumber); // Success message
19        } else {
20            System.out.println("Cancellation failed (either owner mismatch or already empty): " + seatNumber); // Failure message
21        }
22    }
23 }
24
```

Design Pattern Used: Command Pattern

Implements: Command interface

Purpose:

Encapsulates the logic to cancel a specific seat reservation made by a user on a trip.

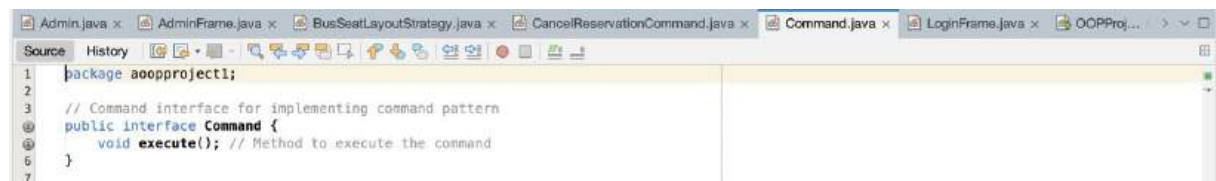
Key Responsibilities:

- Stores trip, seat number, and user information.
- Executes the cancellation action using the Trip class's cancelSeat method.
- Decouples the action trigger from the logic execution.

Usage:

Called when a user chooses to cancel their reserved seat via the UI.

4.5-Command Class



Design Pattern Used: Command Pattern

Purpose:

Defines a common interface for executing operations. This enables encapsulating requests as objects, allowing operations like undo/redo, logging, or queueing.

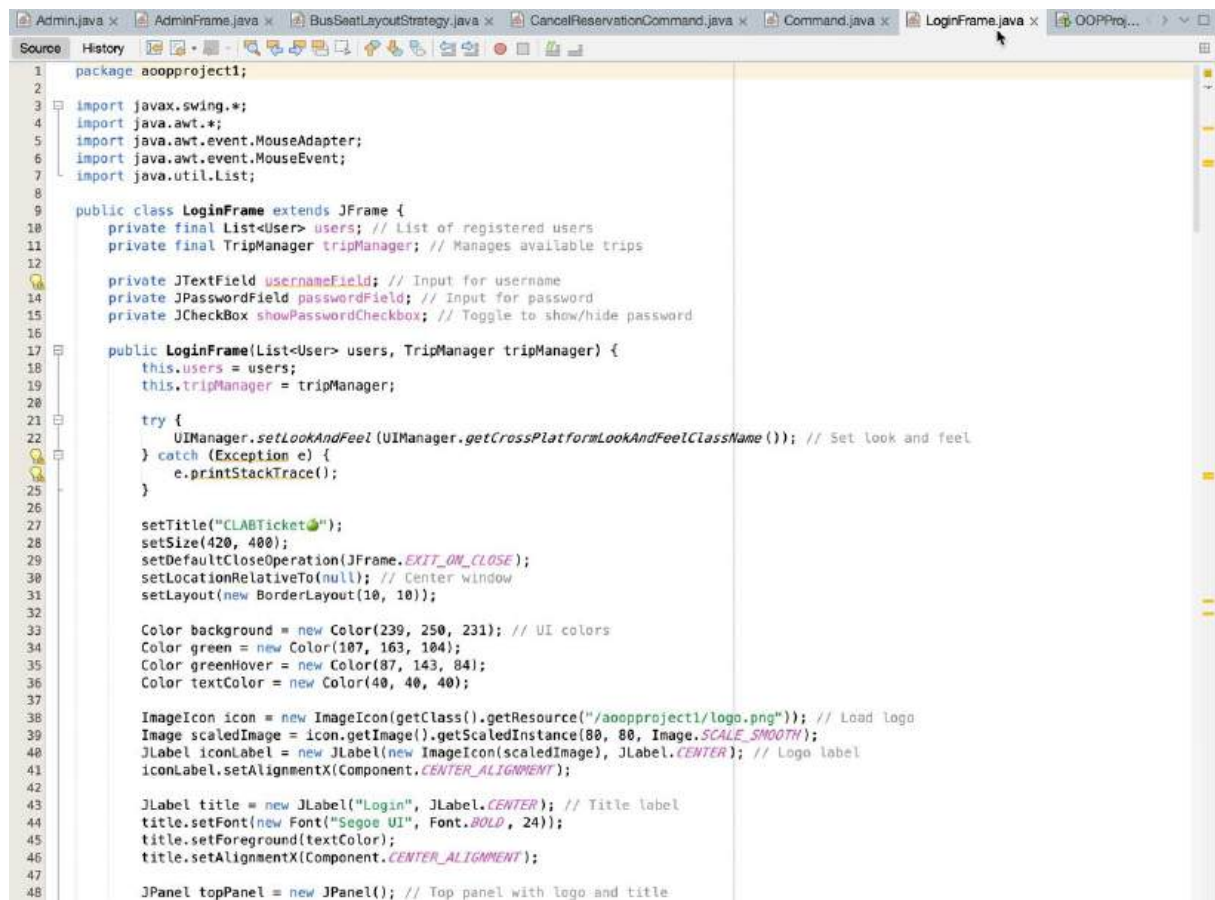
Key Responsibility:

- Declares the `execute()` method, which all concrete command classes (e.g., `SeatReserveCommand`, `CancelReservationCommand`) must implement.

Usage:

Implemented by command classes to perform specific actions, such as reserving or canceling a seat.

4.6-LoginFrame Class



```
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.MouseAdapter;
6 import java.awt.event.MouseEvent;
7 import java.util.List;
8
9 public class LoginFrame extends JFrame {
10     private final List<User> users; // List of registered users
11     private final TripManager tripManager; // Manages available trips
12
13     private JTextField usernameField; // Input for username
14     private JPasswordField passwordField; // Input for password
15     private JCheckBox showPasswordCheckbox; // Toggle to show/hide password
16
17     public LoginFrame(List<User> users, TripManager tripManager) {
18         this.users = users;
19         this.tripManager = tripManager;
20
21         try {
22             UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName()); // Set look and feel
23         } catch (Exception e) {
24             e.printStackTrace();
25         }
26
27         setTitle("CLABTicket🎫");
28         setSize(420, 400);
29         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
30         setLocationRelativeTo(null); // Center window
31         setLayout(new BorderLayout(10, 10));
32
33         Color background = new Color(239, 250, 231); // UI colors
34         Color green = new Color(107, 163, 104);
35         Color greenHover = new Color(87, 143, 84);
36         Color textColor = new Color(40, 40, 40);
37
38         ImageIcon icon = new ImageIcon(getClass().getResource("/aoopproject1/logo.png")); // Load logo
39         Image scaledImage = icon.getImage().getScaledInstance(80, 80, Image.SCALE_SMOOTH);
40         JLabel iconLabel = new JLabel(new ImageIcon(scaledImage), JLabel.CENTER); // Logo label
41         iconLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
42
43         JLabel title = new JLabel("Login", JLabel.CENTER); // Title label
44         title.setFont(new Font("Segoe UI", Font.BOLD, 24));
45         title.setForeground(textColor);
46         title.setAlignmentX(Component.CENTER_ALIGNMENT);
47
48         JPanel topPanel = new JPanel(); // Top panel with logo and title
```

```

Admin.java x AdminFrame.java x BusSeatLayoutStrategy.java x CancelReservationCommand.java x Command.java x LoginFrame.java x OOPProj...
Source History
49 topPanel.setLayout(new BorderLayout(topPanel, BorderLayout.P_AXIS));
50 topPanel.setBackground(background);
51 topPanel.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
52 topPanel.add(iconLabel);
53 topPanel.add(Box.createRigidArea(new Dimension(0, 10)));
54 topPanel.add(title);
55
56 add(topPanel, BorderLayout.NORTH);
57
58 JPanel form = new JPanel(new GridBagLayout()); // Form panel
59 form.setBackground(background);
60 GridBagConstraints gbc = new GridBagConstraints();
61 form.setBorder(BorderFactory.createEmptyBorder(0, 30, 10, 30));
62 gbc.insets = new Insets(8, 8, 8, 8);
63 gbc.fill = GridBagConstraints.HORIZONTAL;
64
65 JLabel emailLabel = new JLabel("username:"); // Username label
66 emailLabel.setForeground(textColor);
67 gbc.gridx = 0; gbc.gridy = 0;
68 form.add(emailLabel, gbc);
69
70 JTextField usernameField = new JTextField(); // Username input
71 usernameField.setForeground(Color.BLACK);
72 usernameField.setBackground(Color.WHITE);
73 gbc.gridx = 1; gbc.gridy = 0;
74 form.add(usernameField, gbc);
75
76 JLabel passwordLabel = new JLabel("password:"); // Password label
77 passwordLabel.setForeground(textColor);
78 gbc.gridx = 0; gbc.gridy = 1;
79 form.add(passwordLabel, gbc);
80
81 JPasswordField passwordField = new JPasswordField(); // Password input
82 passwordField.setForeground(Color.BLACK);
83 passwordField.setBackground(Color.WHITE);
84 gbc.gridx = 1; gbc.gridy = 1;
85 form.add(passwordField, gbc);
86
87 JCheckBox showPasswordCheckbox = new JCheckBox("Show password"); // Checkbox to toggle password visibility
88 showPasswordCheckbox.setFont(new Font("Segoe UI", Font.PLAIN, 12));
89 showPasswordCheckbox.setBackground(background);
90 showPasswordCheckbox.setForeground(textColor);
91 showPasswordCheckbox.addActionListener(e -> {
92     passwordField.setEchoChar(showPasswordCheckbox.isSelected() ? (char) 0 : '*');
93 });
94 gbc.gridx = 1; gbc.gridy = 2;
95 form.add(showPasswordCheckbox, gbc);
96
97 add(form, BorderLayout.CENTER);
98
99 JPanel buttonPanel = new JPanel(new GridLayout(2, 1, 10, 10)); // Login and register buttons
100 buttonPanel.setBorder(BorderFactory.createEmptyBorder(0, 30, 20, 30));
101 buttonPanel.setBackground(background);
102
103 JButton loginBtn = new JButton("Login"); // Login button
104 loginBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
105 loginBtn.setBackground(green);
106 loginBtn.setForeground(textColor);
107 loginBtn.setOpaque(true);
108 loginBtn.setBorderPainted(false);
109 loginBtn.setFocusPainted(false);
110
111 loginBtn.addActionListener(e -> login()); // Trigger login
112 loginBtn.addMouseListener(new MouseAdapter() {
113     public void mouseEntered(MouseEvent e) { loginBtn.setBackground(greenHover); }
114     public void mouseExited(MouseEvent e) { loginBtn.setBackground(green); }
115 });
116 buttonPanel.add(loginBtn);
117
118 JButton registerBtn = new JButton("Don't have an account? Sign up"); // Register button
119 registerBtn.setFont(new Font("Segoe UI", Font.BOLD, 13));
120 registerBtn.setForeground(textColor);
121 registerBtn.setContentAreaFilled(false);
122 registerBtn.setBorderPainted(false);
123 registerBtn.setFocusPainted(false);
124 registerBtn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
125 registerBtn.addActionListener(e -> new RegisterDialog(this, users).setVisible(true)); // Open register dialog
126
127 buttonPanel.add(registerBtn);
128 add(buttonPanel, BorderLayout.SOUTH);
129
130 getContentPane().setBackground(background);
131 setVisible(true); // Show frame
132
133
134 private void login() {
135     String username = usernameField.getText().trim(); // Get entered username
136     String password = new String(passwordField.getPassword()).trim(); // Get entered password
137
138     if (username.isEmpty() || password.isEmpty()) {
139         JOptionPane.showMessageDialog(this, "Please fill in all fields.", "Error", JOptionPane.ERROR_MESSAGE);
140         return;
141     }
142
143     for (User u : users) {
144         if (u.login(username, password)) { // Check credentials

```



```

Admin.java x AdminFrame.java x BusSeatLayoutStrategy.java x CancelReservationCommand.java x Command.java x LoginFrame.java x OOPProj...
Source History
145 JOptionPane.showMessageDialog(this, "✔ Login successful: " + username);
146 dispose(); // Close login window
147
148 if (u instanceof Admin) {
149     new AdminFrame(tripManager, users); // Open admin panel
150 } else {
151     String selectedType = showTypeSelectionDialog(); // Ask for transportation type
152     if (selectedType != null) {
153         new UserFrame(tripManager, users, selectedType, username); // Open user panel
154     }
155 }
156 return;
157 }
158 }
159
160 JOptionPane.showMessageDialog(this, "✖ Login failed. Please check your credentials.", "Error", JOptionPane.ERROR_MESSAGE);
161 }
162
163 private String showTypeSelectionDialog() {
164     JDialog dialog = new JDialog(this, "Select Transportation Type", true); // Modal dialog
165     dialog.setSize(320, 200);
166     dialog.setLocationRelativeTo(this);
167     dialog.setLayout(new BorderLayout());
168
169     Color background = new Color(239, 250, 231);
170     Color pastelYellow = new Color(255, 236, 179);
171     Color pastelBlue = new Color(179, 229, 252);
172
173     dialog.getContentPane().setBackground(background);
174
175     JLabel label = new JLabel("Please select the transportation type:", JLabel.CENTER); // Instruction
176     label.setFont(new Font("Segoe UI", Font.BOLD, 16));
177     label.setForeground(new Color(40, 40, 40));
178     label.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
179     label.setBackground(background);
180     label.setOpaque(true);
181     dialog.add(label, BorderLayout.NORTH);
182
183     JPanel buttonPanel = new JPanel(new GridLayout(1, 2, 20, 10)); // Bus/Plane buttons
184     buttonPanel.setBorder(BorderFactory.createEmptyBorder(20, 30, 20, 30));
185     buttonPanel.setBackground(background);
186
187     JButton busBtn = new JButton("🚌 Bus"); // Bus option
188     busBtn.setBackground(pastelYellow);
189     busBtn.setForeground(Color.BLACK);
190     busBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
191     busBtn.setFocusPainted(false);
192
193     JButton planeBtn = new JButton("✈️ Plane"); // Plane option
194     planeBtn.setBackground(pastelBlue);
195     planeBtn.setForeground(Color.BLACK);
196     planeBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
197     planeBtn.setFocusPainted(false);
198
199     final String[] selected = {null}; // Selected type container
200
201     busBtn.addActionListener(e -> {
202         selected[0] = "bus";
203         dialog.dispose(); // Close dialog
204     });
205
206     planeBtn.addActionListener(e -> {
207         selected[0] = "plane";
208         dialog.dispose(); // Close dialog
209     });
210
211     buttonPanel.add(busBtn);
212     buttonPanel.add(planeBtn);
213     dialog.add(buttonPanel, BorderLayout.CENTER);
214
215     dialog.setVisible(true); // Show dialog
216     return selected[0]; // Return selected type
217 }
218
219 }

```

Purpose:

Handles user login for both Admin and standard User accounts through a graphical interface.

Key Responsibilities:

- Displays input fields for username and password.
- Validates user credentials against the registered user list.
- Opens the corresponding panel (AdminFrame or UserFrame) upon successful login.
- Provides access to registration via RegisterDialog.
- Allows users to select a transportation type (bus or plane) before proceeding to the UserFrame.

Notable GUI Features:

- Custom Swing-based design with styled buttons, password toggle, and responsive feedback.
- Transport type selection dialog after login for standard users.

Related Design Patterns:

- **Factory Pattern:** The `UserFactory` (used during registration) creates either `User` or `Admin` objects.
- **Observer Pattern:** Although not directly in this class, the `TripManager` used here is observed by `UserFrame`.

4.7-OOPProject Class

```
1 package aoopproject1;
2
3 import java.util.*;
4
5 public class OOPProject {
6     public static void main(String[] args) {
7
8         List<User> users = new ArrayList<>(); // List to store users
9         users.add(UserFactory.create("admin", "admin", "1234")); // Create admin user
10        users.add(UserFactory.create("user", "ali", "pass")); // Create normal user
11
12        TripManager tripManager = new TripManager(); // Create trip manager
13
14        // Add sample trips (bus and plane)
15        tripManager.addTrip(new Trip("B1", "Istanbul", "Ankara", "2025-06-01", 12, "bus"));
16        tripManager.addTrip(new Trip("B2", "Izmir", "Bursa", "2025-06-05", 16, "bus"));
17        tripManager.addTrip(new Trip("U1", "Istanbul", "Berlin", "2025-06-15", 40, "plane"));
18        tripManager.addTrip(new Trip("U2", "Antalya", "London", "2025-07-01", 36, "plane"));
19        tripManager.addTrip(new Trip("B3", "Dinar", "Sandikli", "2025-12-01", 20, "bus"));
20        tripManager.addTrip(new Trip("B4", "Urla", "Menemen", "2026-03-23", 32, "bus"));
21        tripManager.addTrip(new Trip("U3", "Paris", "London", "2025-06-23", 48, "plane"));
22        tripManager.addTrip(new Trip("U4", "Moscow", "Antalya", "2026-02-01", 56, "plane"));
23        tripManager.addTrip(new Trip("U5", "Moscow", "Antalya", "2026-02-01", 24, "plane"));
24
25        javax.swing.SwingUtilities.invokeLater(() -> {
26            new LoginFrame(users, tripManager); // Launch login screen
27        });
28    }
29 }
30
```

Purpose:

Acts as the entry point of the application. Initializes core data structures and launches the login interface.

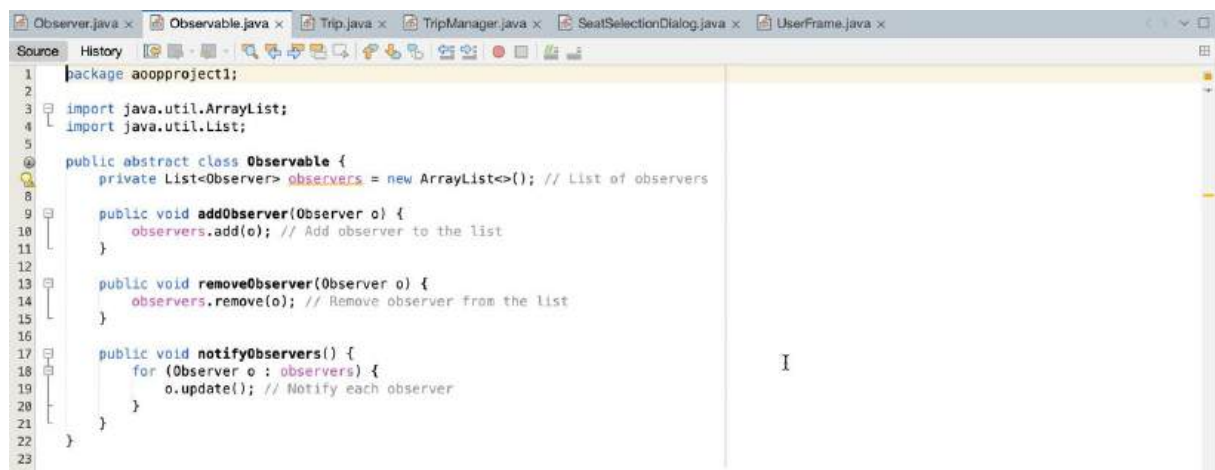
Key Responsibilities:

- Creates a list of users using the **Factory Pattern** (`UserFactory.create()`), including both an admin and a regular user.
- Initializes the `TripManager`, which manages all trips and implements the **Observer Pattern**.
- Populates the application with sample `Trip` data for both buses and planes.
- Launches the GUI by starting the `LoginFrame` on the Event Dispatch Thread (EDT) using `SwingUtilities.invokeLater`.

Related Design Patterns:

- **Factory Pattern:** Used to create user objects (Admin or User) based on role.
- **Observer Pattern:** TripManager allows UI components like UserFrame to observe changes in trip data.

4.8-Observable Class



Purpose:

Serves as the base class for any component that needs to be observed, enabling the **Observer Design Pattern**.

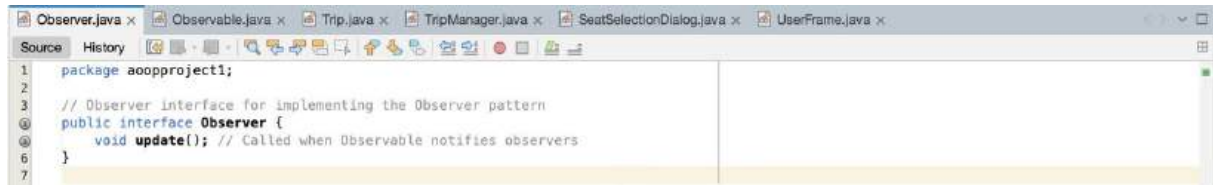
Key Responsibilities:

- Maintains a list of `Observer` objects.
- Provides methods to add or remove observers.
- Notifies all observers by calling their `update()` method when a change occurs.

Related Design Pattern:

- **Observer Pattern:**
This class implements the *Subject* role. It allows dependent components (observers) to stay updated with its changes — for example, UI components like `UserFrame` are notified when trip data changes in `TripManager`.

4.9-Observer Class



Purpose:

Defines the contract for any class that wants to receive updates from an `Observable` subject.

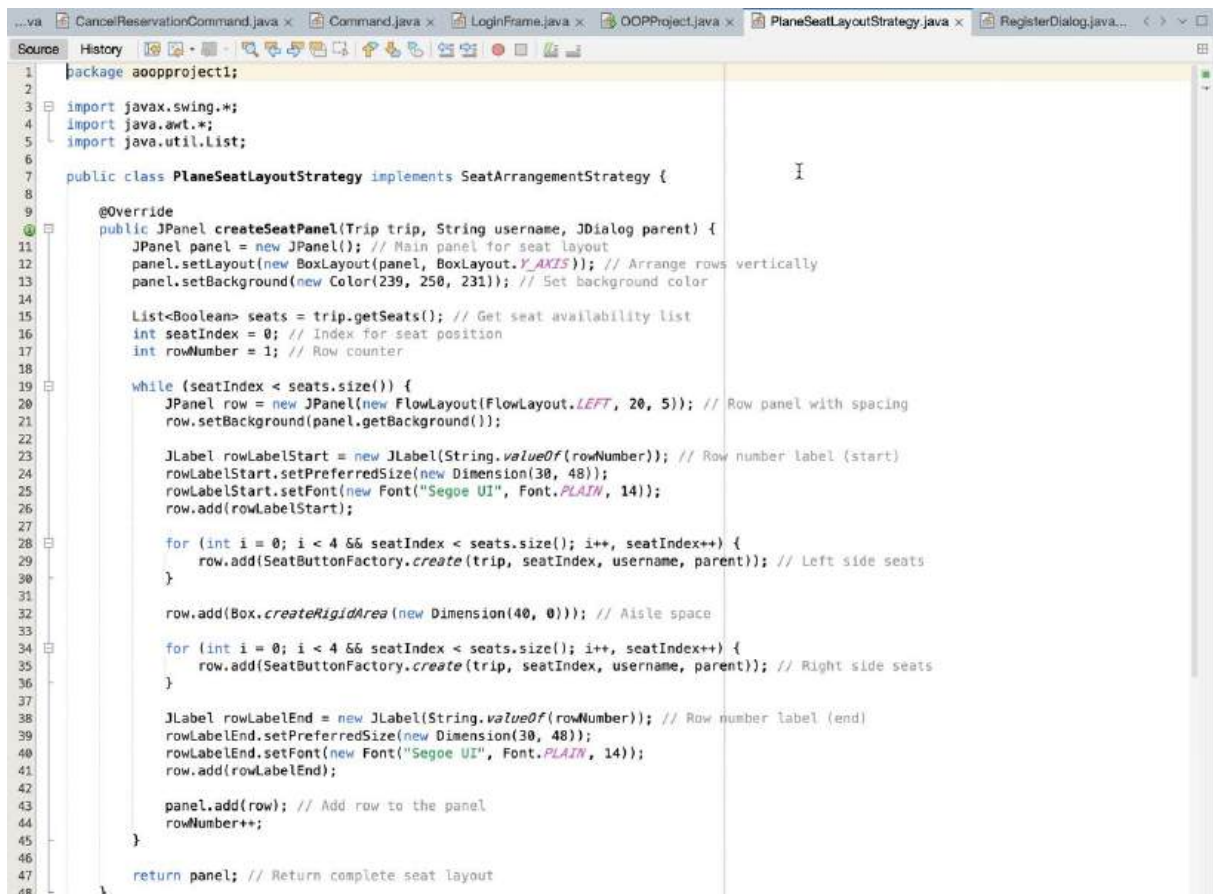
Key Responsibilities:

- Contains a single method `update()`, which is called when the observed subject changes.

Related Design Pattern:

- Observer Pattern:**
This interface plays the *Observer* role. Classes that implement it (e.g., `UserFrame`) are notified automatically by the `Observable` class (e.g., `TripManager`) when data such as trip availability changes.

4.10-PlaneSeatLayoutStrategy Class



Purpose:

Defines the layout of airplane seats using the **Strategy Design Pattern**.

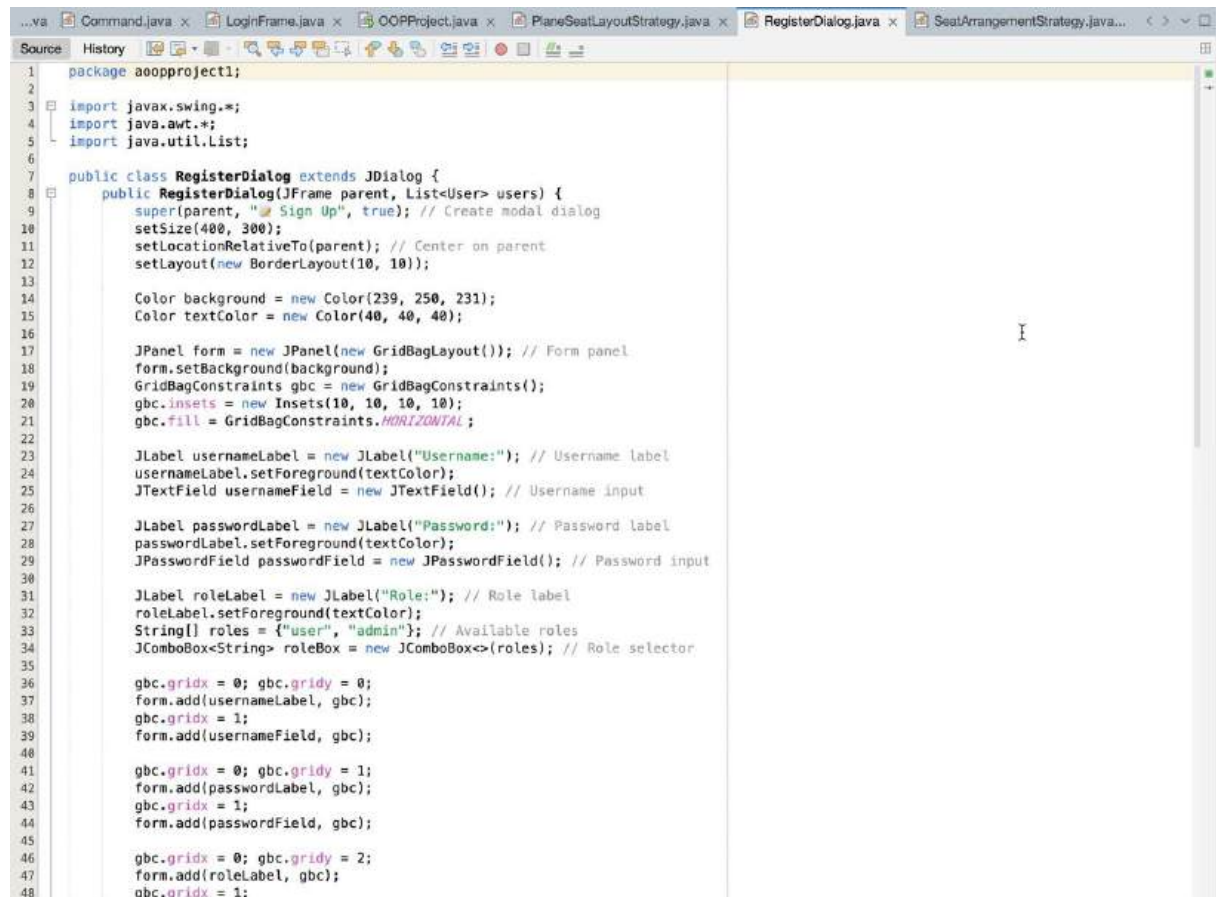
Key Responsibilities:

- Implements the `SeatArrangementStrategy` interface.
- Dynamically generates a seat layout panel with rows containing 8 seats each (4 on each side of an aisle).
- Applies consistent row numbering and visual styling for plane seating.

Related Design Pattern:

- **Strategy Pattern:**
This class is a *concrete strategy* in the Strategy pattern. It allows the system to switch between different seat arrangement strategies (`PlaneSeatLayoutStrategy` for planes and `BusSeatLayoutStrategy` for buses) at runtime based on the vehicle type.

4.11-RegisterDialog Class



```
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.List;
6
7 public class RegisterDialog extends JDialog {
8     public RegisterDialog(JFrame parent, List<User> users) {
9         super(parent, "Sign Up", true); // Create modal dialog
10        setSize(400, 300);
11        setLocationRelativeTo(parent); // Center on parent
12        setLayout(new BorderLayout(10, 10));
13
14        Color background = new Color(239, 250, 231);
15        Color textColor = new Color(40, 40, 40);
16
17        JPanel form = new JPanel(new GridBagLayout()); // Form panel
18        form.setBackground(background);
19        GridBagConstraints gbc = new GridBagConstraints();
20        gbc.insets = new Insets(10, 10, 10, 10);
21        gbc.fill = GridBagConstraints.HORIZONTAL;
22
23        JLabel usernameLabel = new JLabel("Username:"); // Username label
24        usernameLabel.setForeground(textColor);
25        JTextField usernameField = new JTextField(); // Username input
26
27        JLabel passwordLabel = new JLabel("Password:"); // Password label
28        passwordLabel.setForeground(textColor);
29        JPasswordField passwordField = new JPasswordField(); // Password input
30
31        JLabel roleLabel = new JLabel("Role:"); // Role label
32        roleLabel.setForeground(textColor);
33        String[] roles = {"user", "admin"}; // Available roles
34        JComboBox<String> roleBox = new JComboBox<>(roles); // Role selector
35
36        gbc.gridx = 0; gbc.gridy = 0;
37        form.add(usernameLabel, gbc);
38        gbc.gridx = 1;
39        form.add(usernameField, gbc);
40
41        gbc.gridx = 0; gbc.gridy = 1;
42        form.add(passwordLabel, gbc);
43        gbc.gridx = 1;
44        form.add(passwordField, gbc);
45
46        gbc.gridx = 0; gbc.gridy = 2;
47        form.add(roleLabel, gbc);
48        gbc.gridx = 1;
```

```

...va Command.java x LoginFrame.java x OOPProject.java x PlaneSeatLayoutStrategy.java x RegisterDialog.java x SeatArrangementStrategy.java...
Source History
49 form.add(roleBox, gbc);
50
51 add(form, BorderLayout.CENTER);
52
53 JButton signUpBtn = new JButton("Sign Up"); // Register button
54 signUpBtn.setBackground(new Color(107, 163, 184));
55 signUpBtn.setForeground(Color.WHITE);
56 signUpBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
57 signUpBtn.setFocusPainted(false);
58 signUpBtn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
59
60 signUpBtn.addActionListener(e -> {
61     String name = usernameField.getText().trim(); // Get username
62     String password = new String(passwordField.getPassword()).trim(); // Get password
63     String role = roleBox.getSelectedItem().toString(); // Get selected role
64
65     if (name.isEmpty() || password.length() < 6) {
66         JOptionPane.showMessageDialog(this, "Please enter a valid username and at least a 6-character password.");
67         return;
68     }
69
70     if (role.equals("admin")) {
71         String code = JOptionPane.showInputDialog(this, "🔑 Enter admin registration code:"); // Prompt for admin code
72         if (code == null || !code.equals("secret123")) {
73             JOptionPane.showMessageDialog(this, "Invalid admin code!"); // Reject invalid code
74             return;
75         }
76     }
77
78     users.add(UserFactory.create(role, name, password)); // Add user to list
79     JOptionPane.showMessageDialog(this, "✅ Registration successful!");
80     dispose(); // Close dialog
81 });
82
83 JPanel bottom = new JPanel(); // Bottom panel with button
84 bottom.setBackground(background);
85 bottom.add(signUpBtn);
86 add(bottom, BorderLayout.SOUTH);
87
88 }
89

```

Purpose:

Displays a modal dialog window that allows users to register either as a standard user or an admin.

Key Responsibilities:

- Collects username, password, and role selection.
- Validates input (password length, empty fields).
- If the "admin" role is selected, prompts for a registration code.
- Uses the **Factory Design Pattern** to create user objects through `UserFactory.create(...)`.
- Adds the new user to the application's user list.

Related Design Pattern:

- **Factory Pattern:**
The class uses `UserFactory` to create either a `User` or an `Admin` object depending on the selected role, thus hiding object creation logic and promoting flexibility.

4.12-SeatArrangementStrategy Class



```
1 package aoooproject1;
2
3 import javax.swing.*;
4
5 // Strategy interface for creating seat layout for bus and plane
6 public interface SeatArrangementStrategy {
7     public JPanel createSeatPanel(Trip trip, String username, JDialog parent); // Returns seat layout panel
8 }
9
```

Purpose:

Defines a common strategy for generating seat layouts for different types of transportation vehicles (bus or plane).

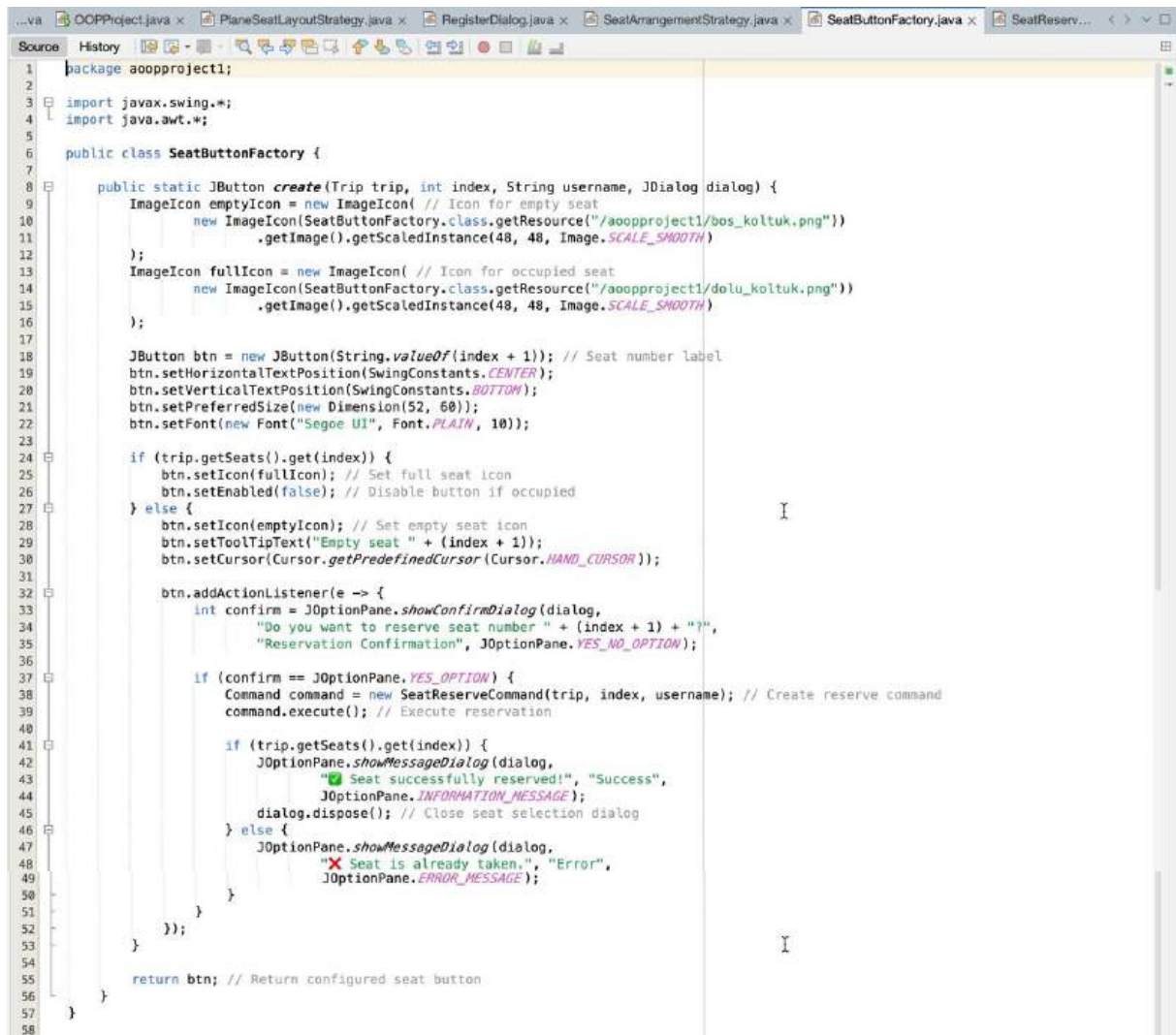
Key Responsibilities:

- Declares the method `createSeatPanel(...)` which returns a custom seat layout (`JPanel`) based on the vehicle type and trip data.
- Enables flexibility and scalability when adding new seat layout types.

Related Design Pattern:

- **Strategy Pattern:**
This interface is part of the **Strategy Design Pattern**, allowing the application to define a family of seat layout algorithms and make them interchangeable (e.g., `BusSeatLayoutStrategy`, `PlaneSeatLayoutStrategy`), promoting open/closed principle and separation of concerns.

4.13-SeatButtonFactory Class



Purpose:

This class implements the **Factory Design Pattern** to create and return customized JButton components that visually represent seat buttons in the UI.

Key Responsibilities:

- Generates a seat button with appropriate seat number, icon (occupied or empty), and interaction logic.
- Uses icons (bos_koltuk.png for empty, dolu_koltuk.png for occupied) to visually represent seat status.
- If the seat is available, the button includes a confirmation prompt and executes the seat reservation through a Command.

Design Pattern Used:

- **Factory Pattern:**
Provides a centralized method (create) to encapsulate the logic for creating seat buttons. This promotes code reuse, encapsulation, and separation of concerns.

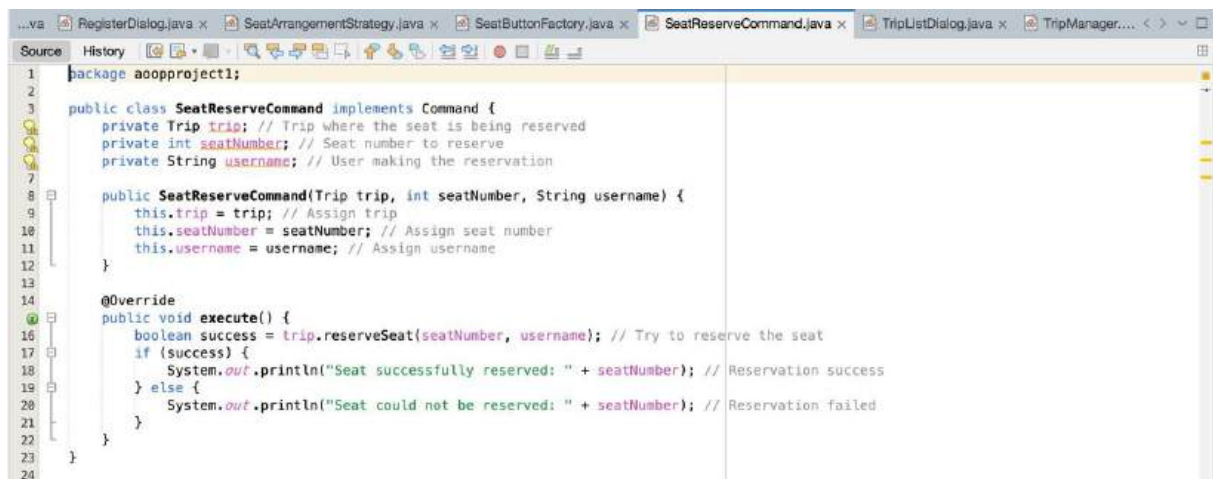
- **Command Pattern (in collaboration):**

When a user confirms a seat reservation, a `SeatReserveCommand` is created and executed. This decouples the button's UI logic from the seat reservation logic.

Related Classes:

- `SeatReserveCommand`: Handles the reservation logic.
- `Trip`: Provides the seat availability and reservation data.

4.14-SeatReserveCommand Class



```

1 package aoopproject1;
2
3 public class SeatReserveCommand implements Command {
4     private Trip trip; // Trip where the seat is being reserved
5     private int seatNumber; // Seat number to reserve
6     private String username; // User making the reservation
7
8     public SeatReserveCommand(Trip trip, int seatNumber, String username) {
9         this.trip = trip; // Assign trip
10        this.seatNumber = seatNumber; // Assign seat number
11        this.username = username; // Assign username
12    }
13
14    @Override
15    public void execute() {
16        boolean success = trip.reserveSeat(seatNumber, username); // Try to reserve the seat
17        if (success) {
18            System.out.println("Seat successfully reserved: " + seatNumber); // Reservation success
19        } else {
20            System.out.println("Seat could not be reserved: " + seatNumber); // Reservation failed
21        }
22    }
23 }
24

```

Purpose:

Implements the **Command Design Pattern** to encapsulate the action of reserving a seat on a trip. This allows the seat reservation to be executed, stored, or passed around as an object.

Key Responsibilities:

- Stores all necessary data (`Trip`, `seatNumber`, and `username`) required to perform a seat reservation.
- Defines the `execute()` method which attempts to reserve a seat and outputs the result.

Design Pattern Used:

- **Command Pattern:**

This class is a concrete command that implements the `Command` interface. It decouples the request for a seat reservation from its execution. This is useful for adding flexibility (e.g., undo/redo, logging, queueing commands).

Related Components:

- `Trip`: Contains the logic to perform the reservation.
- `Command`: Interface that standardizes the `execute()` method.
- `SeatButtonFactory`: Triggers this command upon user interaction.

4.15-SeatSelectionDialog Class

```
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x aoopproject1 x
Source History
1 package aoopproject1;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.*;
6
7 public class SeatSelectionDialog extends JDialog implements Observer {
8     private int emojiX = -50; // Animation position for emoji
9     private Timer animationTimer; // Timer for animation
10    private Trip trip; // Current trip
11    private String username; // Logged-in user
12
13    public SeatSelectionDialog(JFrame parent, Trip trip, String username) {
14        super(parent, "Seat Selection - Trip ID: " + trip.getId(), true);
15        this.trip = trip;
16        this.username = username;
17        trip.addObserver(this); // Subscribe to trip updates
18        setSize(800, 600);
19        setLocationRelativeTo(parent); // Center the dialog
20
21        Color background = new Color(239, 250, 231);
22        JPanel panel = new JPanel(new BorderLayout());
23        panel.setBackground(background);
24        panel.setBorder(BorderFactory.createEmptyBorder(0, 20, 20, 20));
25
26        JPanel animationPanel = new JPanel() {
27            @Override
28            protected void paintComponent(Graphics g) {
29                super.paintComponent(g);
30                if (trip.getType().equalsIgnoreCase("plane")) {
31                    setBackground(new Color(200, 230, 255)); // Sky color
32                } else {
33                    setBackground(new Color(245, 235, 200)); // Road color
34                }
35                g.setFont(new Font("Segoe UI", Font.PLAIN, 28));
36                String emoji = trip.getType().equalsIgnoreCase("plane") ? "✈️" : "🚗"; // Emoji based on type
37
38                g.setColor(new Color(0, 0, 0, 40));
39                g.drawString(emoji, emojiX + 2, 37);
40                g.setColor(Color.BLACK);
41                g.drawString(emoji, emojiX, 35);
42            }
43        };
44        animationPanel.setPreferredSize(new Dimension(800, 50));
45
46        animationTimer = new Timer(20, e -> {
47            emojiX += 2;
48            if (emojiX > getWidth()) emojiX = -50; // Reset emoji position
49        });
50        animationPanel.repaint();
51        animationTimer.start(); // Start animation
52
53        JPanel northPanel = new JPanel(new BorderLayout());
54        northPanel.setBackground(background);
55        northPanel.add(animationPanel, BorderLayout.CENTER);
56
57        JLabel title = new JLabel("Please select a seat", JLabel.CENTER); // Title
58        title.setFont(new Font("Segoe UI", Font.BOLD, 18));
59        title.setForeground(new Color(30, 30, 30));
60        northPanel.add(title, BorderLayout.SOUTH);
61        panel.add(northPanel, BorderLayout.NORTH);
62
63        // Select strategy based on trip type (only plane or bus exist)
64        SeatArrangementStrategy strategy = trip.getType().equalsIgnoreCase("plane")
65            ? new PlaneSeatLayoutStrategy()
66            : new BusSeatLayoutStrategy();
67
68        JPanel gridWrapper = strategy.createSeatPanel(trip, username, this); // Create seat layout
69
70        JScrollPane scroll = new JScrollPane(gridWrapper); // Scrollable seat view
71        scroll.setBorder(BorderFactory.createEmptyBorder());
72        scroll.getViewPort().setBackground(background);
73        scroll.setBackground(background);
74        scroll.getVerticalScrollBar().setUnitIncrement(16);
75
76        panel.add(scroll, BorderLayout.CENTER);
77        add(panel); // Add main panel to dialog
78
79        addWindowListener(new WindowAdapter() {
80            @Override
81            public void windowClosing(WindowEvent e) {
82                if (animationTimer != null) animationTimer.stop(); // Stop animation on close
83                trip.removeObserver(SeatSelectionDialog.this); // Unsubscribe from trip
84            }
85        });
86
87        setVisible(true); // Show dialog
88        initUI(); // Initialize UI
89    }
90
91    @Override
92    public void update() {
93        getContentPane().removeAll(); // Clear UI
94        initUI(); // Rebuild UI
95        revalidate();
96        repaint();
97    }
98
99 }
```



```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x aoopproject1 x
Source History
97
98
99 public void initView() {
100     getContentPane().removeAll(); // Clear previous content
101     Color background = new Color(239, 250, 231);
102
103     JPanel panel = new JPanel(new BorderLayout()); // Main container
104     panel.setBackground(background);
105     panel.setBorder(BorderFactory.createEmptyBorder(0, 20, 20, 20));
106
107     JPanel animationPanel = new JPanel() {
108         @Override
109         protected void paintComponent(Graphics g) {
110             super.paintComponent(g);
111             if (trip.getType().equalsIgnoreCase("plane")) {
112                 setBackground(new Color(200, 230, 255)); // Sky color
113             } else {
114                 setBackground(new Color(245, 235, 200)); // Road color
115             }
116             g.setFont(new Font("Segoe UI", Font.PLAIN, 28));
117             String emoji = trip.getType().equalsIgnoreCase("plane") ? "✈️" : "🚗"; // Emoji
118             g.setColor(new Color(0, 0, 0, 40));
119             g.drawString(emoji, emojiX + 2, 37);
120             g.setColor(Color.BLACK);
121             g.drawString(emoji, emojiX, 35);
122         }
123     };
124     animationPanel.setPreferredSize(new Dimension(800, 50));
125
126     animationTimer = new Timer(20, e -> {
127         emojiX += 2;
128         if (emojiX > getWidth()) emojiX = -50;
129         animationPanel.repaint();
130     });
131     animationTimer.start(); // Restart animation
132
133     JPanel northPanel = new JPanel(new BorderLayout());
134     northPanel.setBackground(background);
135     northPanel.add(animationPanel, BorderLayout.CENTER);
136
137     JLabel title = new JLabel("Please select a seat", JLabel.CENTER); // Title label
138     title.setFont(new Font("Segoe UI", Font.BOLD, 18));
139     title.setForeground(new Color(30, 30, 30));
140     northPanel.add(title, BorderLayout.SOUTH);
141     panel.add(northPanel, BorderLayout.NORTH);
142
143     SeatArrangementStrategy strategy = trip.getType().equalsIgnoreCase("plane")
144         ? new PlaneSeatLayoutStrategy()
145         : new BusSeatLayoutStrategy();
146
147     JPanel gridWrapper = strategy.createSeatPanel(trip, username, this); // Seat layout
148     JScrollPane scroll = new JScrollPane(gridWrapper); // Scroll view
149     scroll.setBorder(BorderFactory.createEmptyBorder());
150     scroll.getViewport().setBackground(background);
151     scroll.setBackground(background);
152     scroll.getVerticalScrollBar().setUnitIncrement(16);
153
154     panel.add(scroll, BorderLayout.CENTER);
155     setContentPane(panel); // Apply new content
156     revalidate();
157     repaint();
158 }
159
160

```

Purpose:

This class creates a graphical dialog window where users can select their seats for a specific trip (bus or plane). It adapts the seat layout dynamically based on the trip type and provides real-time updates using the **Observer Pattern**.

Key Responsibilities:

- Visually displays seat layout using a strategy depending on the vehicle type (BusSeatLayoutStrategy or PlaneSeatLayoutStrategy).
- Animates a moving emoji ("✈️" or "🚗") for visual appeal.
- Subscribes to the trip (Observable) and updates its interface when changes (like seat reservations) occur.

Design Patterns Used:

- **Strategy Pattern**
 - Dynamically selects between `BusSeatLayoutStrategy` and `PlaneSeatLayoutStrategy` based on trip type.
 - Allows flexibility and separation of seat arrangement logic.
- **Observer Pattern**
 - Implements `Observer` interface to listen for changes in the `Trip` object (an `Observable`).
 - Automatically updates the UI when seat data changes.

Related Components:

- `Trip`: The observable trip object that manages seat state.
- `SeatArrangementStrategy`: Strategy interface for different layout implementations.
- `BusSeatLayoutStrategy`, `PlaneSeatLayoutStrategy`: Concrete strategies used to render layout.

4.16-Trip Class

```
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
1 package aoopproject1;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Trip extends Observable {
7     private String id, departure, arrival, date, type; // Trip details
8     private List<Boolean> seats; // true = reserved, false = available
9     private List<String> reservationOwners; // Username per seat
10    private boolean isOpen; // Is trip open for reservation
11    private String vehicleType; // Bus or plane
12
13    public Trip(String id, String departure, String arrival, String date, int seatCount, String type) {
14        this.id = id;
15        this.departure = departure;
16        this.arrival = arrival;
17        this.date = date;
18        this.type = type;
19        this.seats = new ArrayList<>();
20        this.reservationOwners = new ArrayList<>();
21        this.isOpen = true; // By default, trip is open
22
23        for (int i = 0; i < seatCount; i++) {
24            seats.add(false); // All seats initially empty
25            reservationOwners.add(""); // No owners at start
26        }
27    }
28
29    public boolean reserveSeat(int number, String username) {
30        if (!isOpen) return false; // Cannot reserve if trip is closed
31        if (!seats.get(number)) {
32            seats.set(number, true); // Mark seat as taken
33            reservationOwners.set(number, username); // Set owner
34            notifyObservers(); // Update UI
35            return true;
36        }
37        return false;
38    }
39
40    public boolean cancelSeat(int number, String username) {
41        if (!isOpen) return false; // Cannot cancel if trip is closed
42        if (seats.get(number) && reservationOwners.get(number).equals(username)) {
43            seats.set(number, false); // Mark seat as available
44            reservationOwners.set(number, ""); // Clear owner
45            notifyObservers(); // Update UI
46            return true;
47        }
48        return false;
49    }
50
51    public String getReservationOwner(int number) {
52        return reservationOwners.get(number); // Get username for a seat
53    }
54
55    public boolean isOpen() {
56        return isOpen; // Check if trip is open
57    }
58
59    public void setOpen(boolean open) {
60        this.isOpen = open; // Set trip status
61    }
62
63    public String getId() { return id; } // Trip ID
64    public String getDeparture() { return departure; } // Departure location
65    public String getArrival() { return arrival; } // Arrival location
66    public String getDate() { return date; } // Date of trip
67    public String getType() { return type; } // "bus" or "plane"
68    public List<Boolean> getSeats() { return seats; } // Seat status list
69
70    public String getVehicleType() {
71        return vehicleType; // Get vehicle type
72    }
73
74    public void setVehicleType(String vehicleType) {
75        this.vehicleType = vehicleType; // Set vehicle type
76    }
77 }
78
```

Purpose:

The `Trip` class models a transportation trip (bus or plane), including its route, date, seat availability, and reservation ownership. It extends `Observable` to support UI updates when seat reservations change.

Key Responsibilities:

- Manages trip details (ID, route, date, type).
- Tracks seat availability and the user who reserved each seat.

- Allows seat reservation and cancellation while notifying observers of changes.
- Controls whether a trip is open for booking.

Design Patterns Used:

- **Observer Pattern**
 - Inherits from `Observable`, enabling this class to notify subscribed observers (e.g., `SeatSelectionDialog`) when seat reservations are updated.
 - `notifyObservers()` is triggered after a seat is reserved or canceled, ensuring the UI reflects real-time seat status.

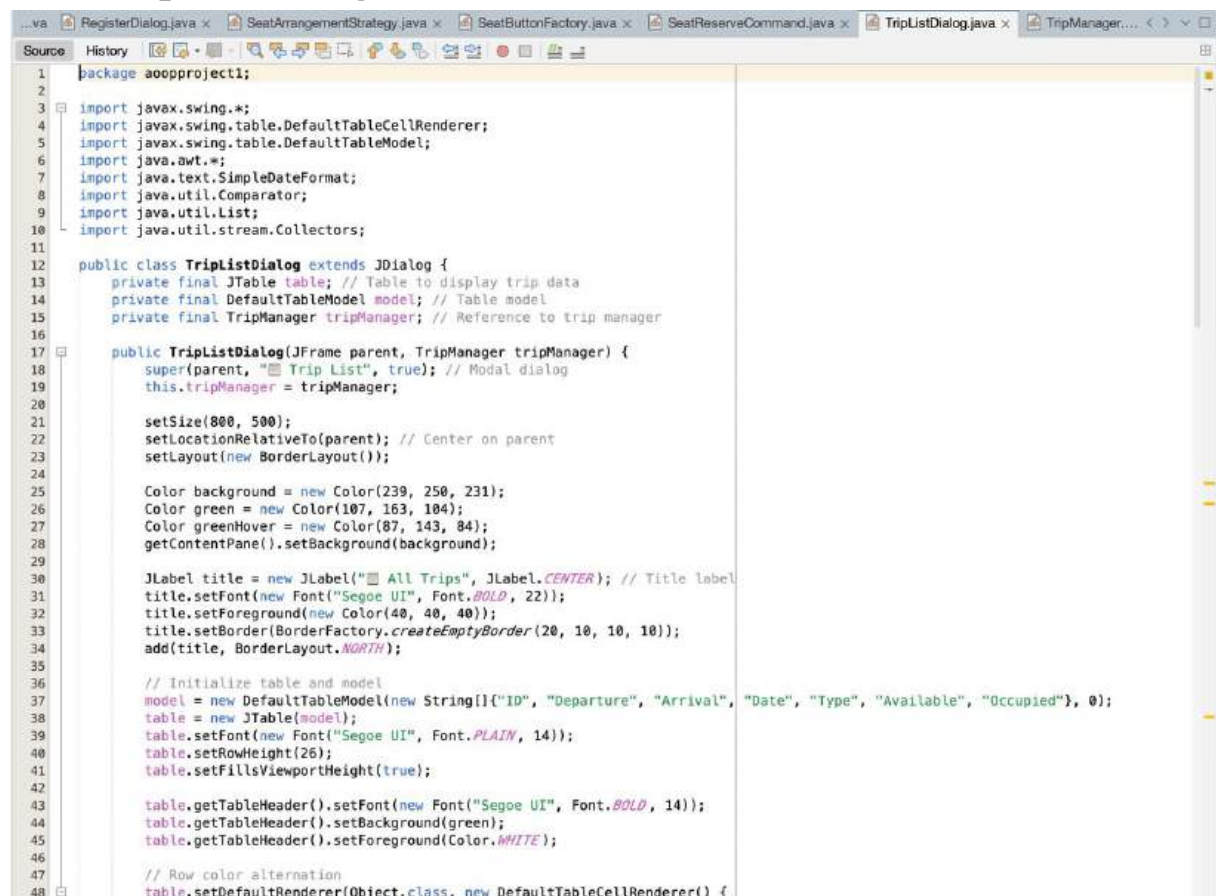
Related Components:

- `SeatSelectionDialog` (observer): Updates UI when the trip changes.
- `TripManager`: Manages a collection of `Trip` objects.

Core Methods:

- `reserveSeat(int, String)`: Attempts to reserve a seat for a user.
- `cancelSeat(int, String)`: Allows a user to cancel their reservation.
- `getReservationOwner(int)`: Returns who reserved a specific seat.
- Getters and setters for trip metadata and vehicle type.

4.17-TripListDialog Class



```

1 package aocpproject1;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableCellRenderer;
5 import javax.swing.table.DefaultTableModel;
6 import java.awt.*;
7 import java.text.SimpleDateFormat;
8 import java.util.Comparator;
9 import java.util.List;
10 import java.util.stream.Collectors;
11
12 public class TripListDialog extends JDialog {
13     private final JTable table; // Table to display trip data
14     private final DefaultTableModel model; // Table model
15     private final TripManager tripManager; // Reference to trip manager
16
17     public TripListDialog(JFrame parent, TripManager tripManager) {
18         super(parent, "Trip List", true); // Modal dialog
19         this.tripManager = tripManager;
20
21         setSize(800, 500);
22         setLocationRelativeTo(parent); // Center on parent
23         setLayout(new BorderLayout());
24
25         Color background = new Color(239, 250, 231);
26         Color green = new Color(107, 163, 104);
27         Color greenHover = new Color(87, 143, 84);
28         getContentPane().setBackground(background);
29
30         JLabel title = new JLabel("All Trips", JLabel.CENTER); // Title label
31         title.setFont(new Font("Segoe UI", Font.BOLD, 22));
32         title.setForeground(new Color(40, 40, 40));
33         title.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
34         add(title, BorderLayout.NORTH);
35
36         // Initialize table and model
37         model = new DefaultTableModel(new String[]{"ID", "Departure", "Arrival", "Date", "Type", "Available", "Occupied"}, 0);
38         table = new JTable(model);
39         table.setFont(new Font("Segoe UI", Font.PLAIN, 14));
40         table.setRowHeight(26);
41         table.setFillsViewportHeight(true);
42
43         table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 14));
44         table.getTableHeader().setBackground(green);
45         table.getTableHeader().setForeground(Color.WHITE);
46
47         // Row color alternation
48         table.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {

```

```

...va RegisterDialog.java x SeatArrangementStrategy.java x SeatButtonFactory.java x SeatReserveCommand.java x TripListDialog.java x TripManager... < > v
Source History
49 final Color even = new Color(255, 255, 255);
50 final Color odd = new Color(230, 255, 230);
51 @Override
52 public Component getTableCellRendererComponent(JTable tbl, Object val, boolean isSelected, boolean hasFocus, int row, int col) {
53     Component c = super.getTableCellRendererComponent(tbl, val, isSelected, hasFocus, row, col);
54     c.setBackground(row % 2 == 0 ? even : odd); // Alternate row colors
55     return c;
56 }
57 }
58
59 JScrollPane scrollPane = new JScrollPane(table); // Scroll for table
60 scrollPane.getViewport().setBackground(Color.WHITE);
61 add(scrollPane, BorderLayout.CENTER);
62
63 JButton deleteBtn = new JButton("X Delete Trip"); // Delete button
64 deleteBtn.setFont(new Font("Segoe UI", Font.BOLD, 14));
65 deleteBtn.setBackground(green);
66 deleteBtn.setForeground(Color.WHITE);
67 deleteBtn.setOpaque(true);
68 deleteBtn.setBorderPainted(false);
69 deleteBtn.setFocusPainted(false);
70 deleteBtn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
71 deleteBtn.setPreferredSize(new Dimension(150, 35));
72
73 // Hover effect for delete button
74 deleteBtn.addMouseListener(new java.awt.event.MouseAdapter() {
75     public void mouseEntered(java.awt.event.MouseEvent evt) {
76         deleteBtn.setBackground(greenHover);
77     }
78     public void mouseExited(java.awt.event.MouseEvent evt) {
79         deleteBtn.setBackground(green);
80     }
81 });
82
83 // Delete button logic
84 deleteBtn.addActionListener(e -> {
85     int selected = table.getSelectedRow(); // Get selected row
86     if (selected >= 0) {
87         String id = (String) model.getValueAt(selected, 0); // Get trip ID
88         int confirm = JOptionPane.showConfirmDialog(this, "Delete trip " + id + "?", "Confirm", JOptionPane.YES_NO_OPTION);
89         if (confirm == JOptionPane.YES_OPTION) {
90             if (tripManager.deleteTrip(id)) {
91                 JOptionPane.showMessageDialog(this, "Trip deleted.");
92                 refreshTable(); // Refresh list
93             } else {
94                 JOptionPane.showMessageDialog(this, "Trip not found.");
95             }
96         }
97     }
98     else {
99         JOptionPane.showMessageDialog(this, "Please select a trip to delete."); // No selection warning
100     }
101 });
102
103 JPanel bottomPanel = new JPanel(); // Panel for delete button
104 bottomPanel.setBackground(background);
105 bottomPanel.add(deleteBtn);
106 add(bottomPanel, BorderLayout.SOUTH);
107
108 refreshTable(); // Load data initially
109 }
110
111 private void refreshTable() {
112     model.setRowCount(0); // Clear table
113     SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
114
115     List<Trip> sorted = tripManager.getAllTrips().stream()
116         .sorted(Comparator.comparing(t -> {
117             try {
118                 return sdf.parse(t.getDate()); // Sort by date
119             } catch (Exception e) {
120                 return new java.util.Date();
121             }
122         }))
123         .collect(Collectors.toList());
124
125     for (Trip t : sorted) {
126         int total = t.getSeats().size(); // Total seats
127         long occupied = t.getSeats().stream().filter(k -> k.count()); // Count occupied
128         long available = total - occupied;
129
130         model.addRow(new Object[]{
131             t.getId(), t.getDeparture(), t.getArrival(),
132             t.getDate(), t.getType(), available, occupied
133         });
134     }
135 }
136
137 }

```

Purpose:

TripListDialog is a modal window that displays all trips in a table format and allows admins to delete selected trips from the system.

Key Responsibilities:

- Displays all available trips using a table.
- Allows admins to delete trips with confirmation.
- Provides a scrollable and color-enhanced UI for readability.
- Sorts trips by date when listing.

Core Features:

- Table columns: ID, Departure, Arrival, Date, Type, Available seats, Occupied seats.
- Uses alternating row colors for better readability.
- Clicking a trip and pressing "Delete" removes it from the `TripManager` and updates the UI.
- Uses `DefaultTableModel` and custom cell rendering for layout and design.

Associated Components:

- `TripManager`: Used to fetch and delete trips.
- `Trip`: Data source for the table rows.

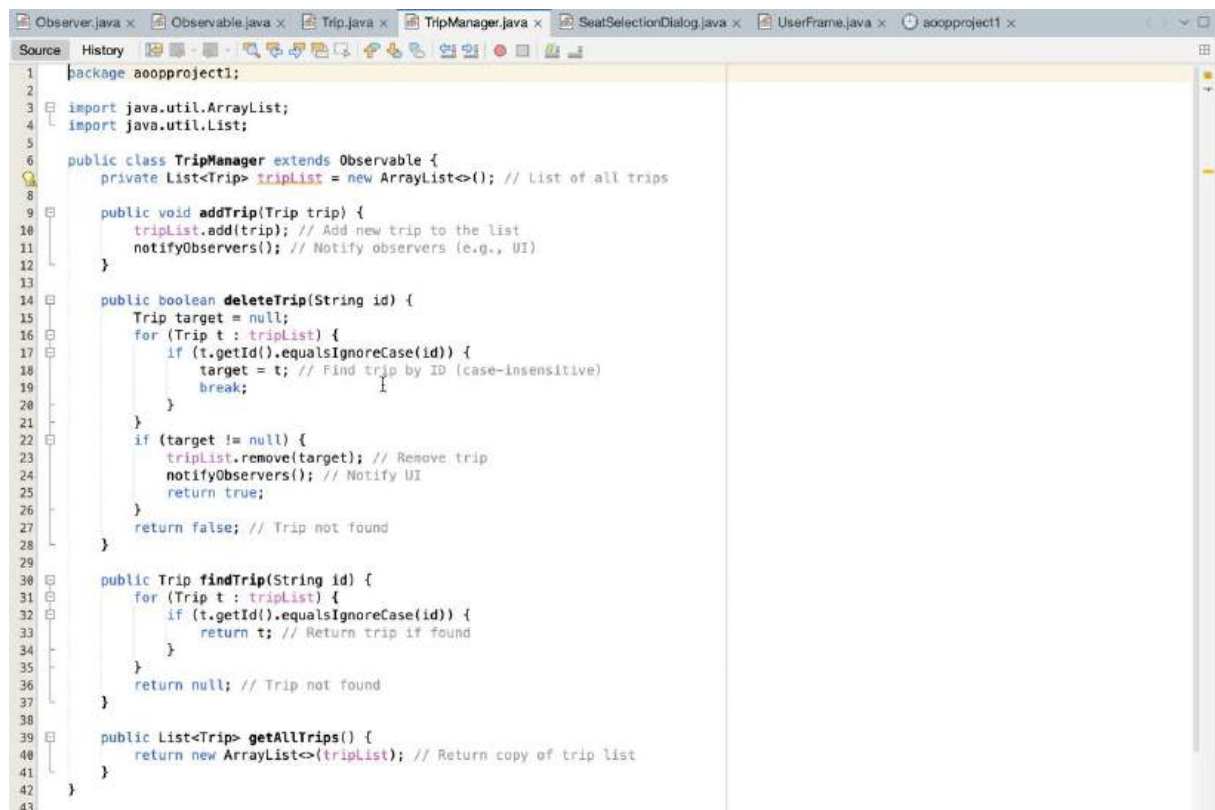
Notable Methods:

- `refreshTable()`: Populates and refreshes the table with trip data, sorted by date.
- Action listener for the delete button handles confirmation and deletion logic.

UI Details:

- Consistent with the project's theme (colors, font).
- Responsive layout and user-friendly interaction.

4.18-TripManager Class



```
1 package aoopproject1;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class TripManager extends Observable {
7     private List<Trip> tripList = new ArrayList<>(); // List of all trips
8
9     public void addTrip(Trip trip) {
10         tripList.add(trip); // Add new trip to the list
11         notifyObservers(); // Notify observers (e.g., UI)
12     }
13
14     public boolean deleteTrip(String id) {
15         Trip target = null;
16         for (Trip t : tripList) {
17             if (t.getId().equalsIgnoreCase(id)) {
18                 target = t; // Find trip by ID (case-insensitive)
19                 break;
20             }
21         }
22         if (target != null) {
23             tripList.remove(target); // Remove trip
24             notifyObservers(); // Notify UI
25             return true;
26         }
27         return false; // Trip not found
28     }
29
30     public Trip findTrip(String id) {
31         for (Trip t : tripList) {
32             if (t.getId().equalsIgnoreCase(id)) {
33                 return t; // Return trip if found
34             }
35         }
36         return null; // Trip not found
37     }
38
39     public List<Trip> getAllTrips() {
40         return new ArrayList<>(tripList); // Return copy of trip list
41     }
42 }
43
```

Purpose:

TripManager manages all trips in the system. It acts as a central controller for adding, deleting, searching, and listing trips.

Design Pattern Used:

Observer Pattern (extends observable) – Notifies registered UI components when trips are added or deleted, so the UI can refresh automatically.

Responsibilities:

- Add a new trip.
- Delete an existing trip by ID.
- Find a trip by ID.
- Provide a full list of all trips.

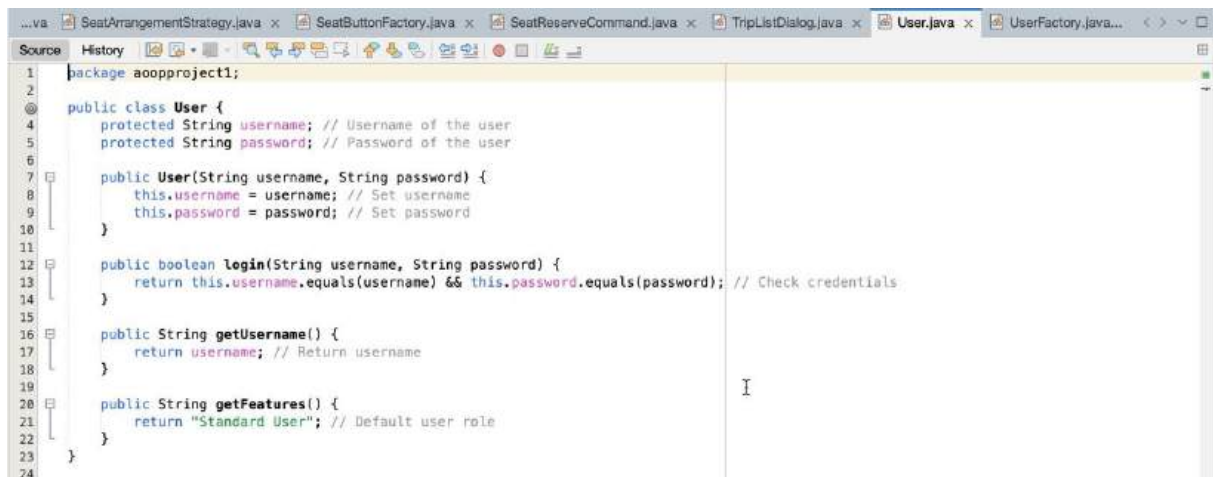
Key Methods:

- `addTrip(Trip trip)`: Adds a new trip and notifies observers (e.g., updates the UI).
- `deleteTrip(String id)`: Deletes a trip by its ID (case-insensitive). Returns `true` if successful.
- `findTrip(String id)`: Searches and returns a trip matching the given ID.
- `getAllTrips()`: Returns a copy of the full list of trips.

Usage Context:

- Used by Admin and User panels to manage and display trips.
- Automatically updates the UI when the trip list changes.

4.19-User Class



```
1 package aoopproject1;
2
3
4 public class User {
5     protected String username; // Username of the user
6     protected String password; // Password of the user
7
8     public User(String username, String password) {
9         this.username = username; // Set username
10        this.password = password; // Set password
11    }
12
13    public boolean login(String username, String password) {
14        return this.username.equals(username) && this.password.equals(password); // Check credentials
15    }
16
17    public String getUsername() {
18        return username; // Return username
19    }
20
21    public String getFeatures() {
22        return "Standard User"; // Default user role
23    }
24 }
```

Purpose:

The `User` class defines a standard user in the system with basic login functionality and identity information.

Responsibilities:

- Stores a username and password.
- Verifies login credentials through the `login()` method.
- Provides default user role information via `getFeatures()`.

Key Methods:

- `login(String username, String password)`: Returns `true` if the provided credentials match the user's stored credentials.
- `getUsername()`: Returns the user's username.
- `getFeatures()`: Returns the user's role or description — "Standard User" by default.

Usage Context:

- Used as a base class for user-related operations, such as login verification.
- Extended by the `Admin` class to support role-based behavior.

Design Pattern:

-Inheritance (Object-Oriented Principle) – This class serves as a superclass for specialized user types (e.g., `Admin`).

4.20-UserFactory Class



Purpose:

The `UserFactory` class simplifies the creation of `User` objects based on their type (e.g., admin or standard user).

Responsibilities:

- Provides a static `create()` method to instantiate either a `User` or `Admin` based on input parameters.

Key Method:

- `create(String type, String name, String password):`
 - If type is "admin" (case-insensitive), returns an `Admin` object.
 - Otherwise, returns a standard `User`.

Usage Context:

- Used during user registration to dynamically create the correct type of user.
- Encapsulates object creation logic, avoiding direct use of `new` in other parts of the code.

Design Pattern:

-**Factory Pattern** – Centralizes and abstracts object creation, allowing flexible instantiation based on input type.

4.21-UserFrame Class

```
Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
package aoopproject1;

import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.text.SimpleDateFormat;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class UserFrame extends JFrame implements Observer {
    private final TripManager tripManager; // Central trip manager (Observable)
    private final List<User> users; // List of all users in the system
    private final String[] selectedType = new String[1]; // Transport type filter (bus or plane)
    private final String loggedInUsername; // Currently logged-in user

    private final JPanel tripListPanel = new JPanel(); // Container for trip cards
    private final JScrollPane scrollPane = new JScrollPane(tripListPanel); // Scrollable wrapper

    public UserFrame(TripManager tripManager, List<User> users, String initialType, String username) {
        this.tripManager = tripManager;
        this.users = users;
        this.selectedType[0] = initialType;
        this.loggedInUsername = username;

        // Register this frame as an observer to TripManager
        this.tripManager.addObserver(this);

        // Frame settings
        setTitle("User Panel - " + initialType.toUpperCase() + " Trips");
        setSize(1000, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // UI colors
        Color background = new Color(239, 250, 231);
        Color green = new Color(107, 163, 104);
        Color greenHover = new Color(87, 143, 84);
        Color textColor = new Color(40, 40, 40);

        // Header title
        JLabel header = new JLabel("Available Trips", JLabel.CENTER);
        header.setFont(new Font("Segoe UI", Font.BOLD, 22));
        header.setForeground(textColor);
        header.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
        header.setBackground(background);

        header.setOpaque(true);
        add(header, BorderLayout.NORTH);

        // Main panel for trips
        tripListPanel.setLayout(new BoxLayout(tripListPanel, BoxLayout.Y_AXIS));
        tripListPanel.setBackground(background);
        tripListPanel.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));

        // Scroll wrapper
        scrollPane.setBorder(BorderFactory.createEmptyBorder());
        scrollPane.setBackground(background);
        scrollPane.getViewport().setBackground(background);
        scrollPane.getVerticalScrollBar().setUnitIncrement(16);
        scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
        scrollPane.setPreferredSize(new Dimension(960, 450));
        add(scrollPane, BorderLayout.CENTER);

        // Main action buttons
        JButton reserveBtn = new JButton("Reserve / Cancel");
        JButton searchBtn = new JButton("Search Trip by ID");
        JButton changeTypeBtn = new JButton("Change Transport Type");
        JButton logoutBtn = new JButton("Logout");

        // Set visual and interaction styles for buttons
        JButton[] buttons = { reserveBtn, searchBtn, changeTypeBtn, logoutBtn };
        for (JButton btn : buttons) {
            btn.setFont(new Font("Segoe UI", Font.BOLD, 14));
            btn.setBackground(green);
            btn.setForeground(Color.WHITE);
            btn.setOpaque(true);
            btn.setBorderPainted(false);
            btn.setFocusPainted(false);
            btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
            btn.addMouseListener(new MouseAdapter() {
                public void mouseEntered(MouseEvent e) { btn.setBackground(greenHover); }
                public void mouseExited(MouseEvent e) { btn.setBackground(green); }
            });
        }

        // Action: reservation guidance
        reserveBtn.addActionListener(e -> {
            JOptionPane.showMessageDialog(this, "Please click on a trip card.");
        });

        // Action: trip search by ID
        searchBtn.addActionListener(e -> {
            String input = JOptionPane.showInputDialog(this, "Enter Trip ID:");
            if (input != null && !input.trim().isEmpty()) {
                Trip t = tripManager.findTrip(input.trim());
            }
        });
    }
}
```

```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144

if (t != null && t.getType().equalsIgnoreCase(selectedType[0])) {
    // Show trip details and allow seat reservation
    JOptionPane.showMessageDialog(this,
        "Trip Detail:\n" +
        "ID: " + t.getId() + "\n" +
        "Departure: " + t.getDeparture() + "\n" +
        "Arrival: " + t.getArrival() + "\n" +
        "Date: " + t.getDate(),
        "Trip Info", JOptionPane.INFORMATION_MESSAGE);
    int confirm = JOptionPane.showConfirmDialog(this,
        "Would you like to reserve a seat for this trip?",
        "Confirm", JOptionPane.YES_NO_OPTION);
    if (confirm == JOptionPane.YES_OPTION) {
        new SeatSelectionDialog(this, t, loggedInUsername);
        refreshTripList();
    }
} else {
    JOptionPane.showMessageDialog(this, "X Trip not found or type mismatch.");
}
});

// Action: transport type switching
changeTypeBtn.addActionListener(e -> {
    String newType = showTypeSelectionDialog();
    if (newType != null && !newType.equalsIgnoreCase(selectedType[0])) {
        selectedType[0] = newType;
        setTitle("User Panel - " + newType.toUpperCase() + " Trips");
        header.setText("Available " + newType.toUpperCase() + " Trips");
        refreshTripList();
    }
});

// Action: logout
logoutBtn.addActionListener(e -> {
    dispose(); // Close current frame
    new LoginFrame(users, tripManager); // Return to login screen
});

// Add buttons to panel
JPanel buttonPanel = new JPanel();
buttonPanel.setBackground(background);
buttonPanel.add(reserveBtn);
buttonPanel.add(searchBtn);
buttonPanel.add(changeTypeBtn);
buttonPanel.add(logoutBtn);
add(buttonPanel, BorderLayout.SOUTH);

```



```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
145 refreshTripList(); // Initial UI load
146 setVisible(true);
147
148 // Remove observer on close
149 addWindowListener(new java.awt.event.WindowAdapter() {
150     @Override
151     public void windowClosing(java.awt.event.WindowEvent e) {
152         tripManager.removeObserver(UserFrame.this);
153     }
154 });
155
156
157 @Override
158 public void update() {
159     refreshTripList();
160 }
161
162 private String showTypeSelectionDialog() {
163     JDialog dialog = new JDialog(this, "Select Transport Type", true);
164     dialog.setSize(320, 200);
165     dialog.setLocationRelativeTo(this);
166     dialog.setLayout(new BorderLayout());
167
168     // Style and colors
169     Color background = new Color(239, 250, 231);
170     Color pastelYellow = new Color(255, 236, 179);
171     Color pastelBlue = new Color(179, 229, 252);
172     dialog.getContentPane().setBackground(background);
173
174     // Title
175     JLabel label = new JLabel("Please select transport type:", JLabel.CENTER);
176     label.setFont(new Font("Segoe UI", Font.BOLD, 16));
177     label.setForeground(new Color(40, 40, 40));
178     label.setBorder(BorderFactory.createEmptyBorder(20, 10, 10, 10));
179     dialog.add(label, BorderLayout.NORTH);
180
181     // Buttons for type selection
182     JPanel buttonPanel = new JPanel(new GridLayout(1, 2, 20, 10));
183     buttonPanel.setBorder(BorderFactory.createEmptyBorder(20, 30, 20, 30));
184     buttonPanel.setBackground(background);
185
186     JButton busBtn = new JButton("Bus");
187     JButton planeBtn = new JButton("Plane");
188     busBtn.setBackground(pastelYellow);
189     planeBtn.setBackground(pastelBlue);
190     busBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
191     planeBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
192
193     final String[] selected = {null};
194
195     busBtn.addActionListener(e -> {
196         selected[0] = "bus";
197         dialog.dispose();
198     });
199
200     planeBtn.addActionListener(e -> {
201         selected[0] = "plane";
202         dialog.dispose();
203     });
204
205     buttonPanel.add(busBtn);
206     buttonPanel.add(planeBtn);
207     dialog.add(buttonPanel, BorderLayout.CENTER);
208     dialog.setVisible(true);
209
210     return selected[0];
211 }
212
213 private void refreshTripList() {
214     tripListPanel.removeAll(); // Clear existing cards
215     SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
216
217     // Filter and sort trips by date
218     List<Trip> sortedTrips = tripManager.getAllTrips().stream()
219         .filter(t -> t.getType().equalsIgnoreCase(selectedType[0]))
220         .sorted(Comparator.comparing(t -> {
221             try {
222                 return sdf.parse(t.getDate());
223             } catch (Exception e) {
224                 return new java.util.Date();
225             }
226         }))
227         .collect(Collectors.toList());
228
229     // Display each trip as a card
230     for (Trip t : sortedTrips) {
231         String emoji = t.getType().equalsIgnoreCase("plane") ? "✈️" : "🚌";
232         int total = t.getSeats().size();
233         long occupied = t.getSeats().stream().filter(k -> k).count();
234         long available = total - occupied;
235
236         JPanel card = new JPanel(new GridLayout(2, 1));
237         card.setPreferredSize(new Dimension(920, 80));
238         card.setBackground(Color.WHITE);
239         card.setBorder(BorderFactory.createCompoundBorder(
240             BorderFactory.createLineBorder(new Color(200, 200, 200)),

```



```

Observer.java x Observable.java x Trip.java x TripManager.java x SeatSelectionDialog.java x UserFrame.java x
Source History
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308

    ));
    BorderFactory.createEmptyBorder(10, 15, 10, 15)

    JLabel title = new JLabel(emoji + " " + t.getDeparture() + " → " + t.getArrival() + " | " + t.getDate());
    JLabel info = new JLabel("Available: " + available + " | Occupied: " + occupied + " | ID: " + t.getId());
    title.setFont(new Font("Segoe UI", Font.BOLD, 16));
    info.setFont(new Font("Segoe UI", Font.PLAIN, 14));
    info.setForeground(new Color(70, 70, 70));

    card.add(title);
    card.add(info);
    card.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

    // Click to open seat selection
    card.addMouseListener(new MouseAdapter() {
        public void mouseClicked(MouseEvent evt) {
            new SeatSelectionDialog(UserFrame.this, t, loggedInUsername);
            refreshTripList(); // Refresh UI after reservation
        }
    });

    tripListPanel.add(card);
    tripListPanel.add(Box.createRigidArea(new Dimension(0, 12))); // Space

}

// Section for showing user's reserved seats
JLabel rLabel = new JLabel("♥ Your Reserved Seats:");
rLabel.setFont(new Font("Segoe UI", Font.BOLD, 16));
rLabel.setBorder(BorderFactory.createEmptyBorder(10, 0, 5, 0));
tripListPanel.add(rLabel);

// Loop through all trips and find user's reservations
for (Trip t : tripManager.getAllTrips()) {
    List<Boolean> seats = t.getSeats();
    for (int i = 0; i < seats.size(); i++) {
        if (seats.get(i) && loggedInUsername.equals(t.getReservationOwner(i))) {
            int finalI = i;
            JPanel reservationRow = new JPanel(new FlowLayout(FlowLayout.LEFT));
            reservationRow.setBackground(new Color(250, 250, 240));

            JLabel info = new JLabel("Trip ID: " + t.getId() + ", Date: " + t.getDate() + ", Seat No: " + (i + 1));
            info.setFont(new Font("Segoe UI", Font.PLAIN, 13));

            JButton cancelBtn = new JButton("Cancel");
            cancelBtn.setBackground(new Color(204, 77, 77));
            cancelBtn.setForeground(Color.WHITE);
            cancelBtn.setFont(new Font("Segoe UI", Font.PLAIN, 12));

            // Cancel reservation with command
            cancelBtn.addActionListener(e -> {
                Command cancelCommand = new CancelReservationCommand(t, finalI, loggedInUsername);
                cancelCommand.execute();
                JOptionPane.showMessageDialog(this, "Reservation canceled.");
                refreshTripList();
            });

            reservationRow.add(info);
            reservationRow.add(cancelBtn);
            tripListPanel.add(reservationRow);
        }
    }
}

tripListPanel.revalidate();
tripListPanel.repaint();
}

```

Purpose:

The `UserFrame` class represents the main interface for standard users to interact with the system. It allows users to view available trips, search by trip ID, reserve or cancel seats, and switch between transport types (bus or plane).

Responsibilities:

- Observes the `TripManager` for changes and updates the UI accordingly.
- Filters trips based on the selected vehicle type.
- Displays trip details in a scrollable, card-based layout.
- Enables users to initiate seat reservations and cancellations.
- Manages user logout and transport type switching.

Design Patterns Used:

- **Observer Pattern:**
Implements the `Observer` interface to receive updates from `TripManager` when trips are added, deleted, or modified. This ensures that the trip list in the user interface stays up to date automatically.
- **Command Pattern:**
The cancellation of a seat reservation is encapsulated within the `CancelReservationCommand` class. This allows the action to be decoupled from the UI logic and promotes reusability and maintainability.

5-SCREENSHOTS FROM APPLICATION

5.1-Login

CLABTicket 



Login

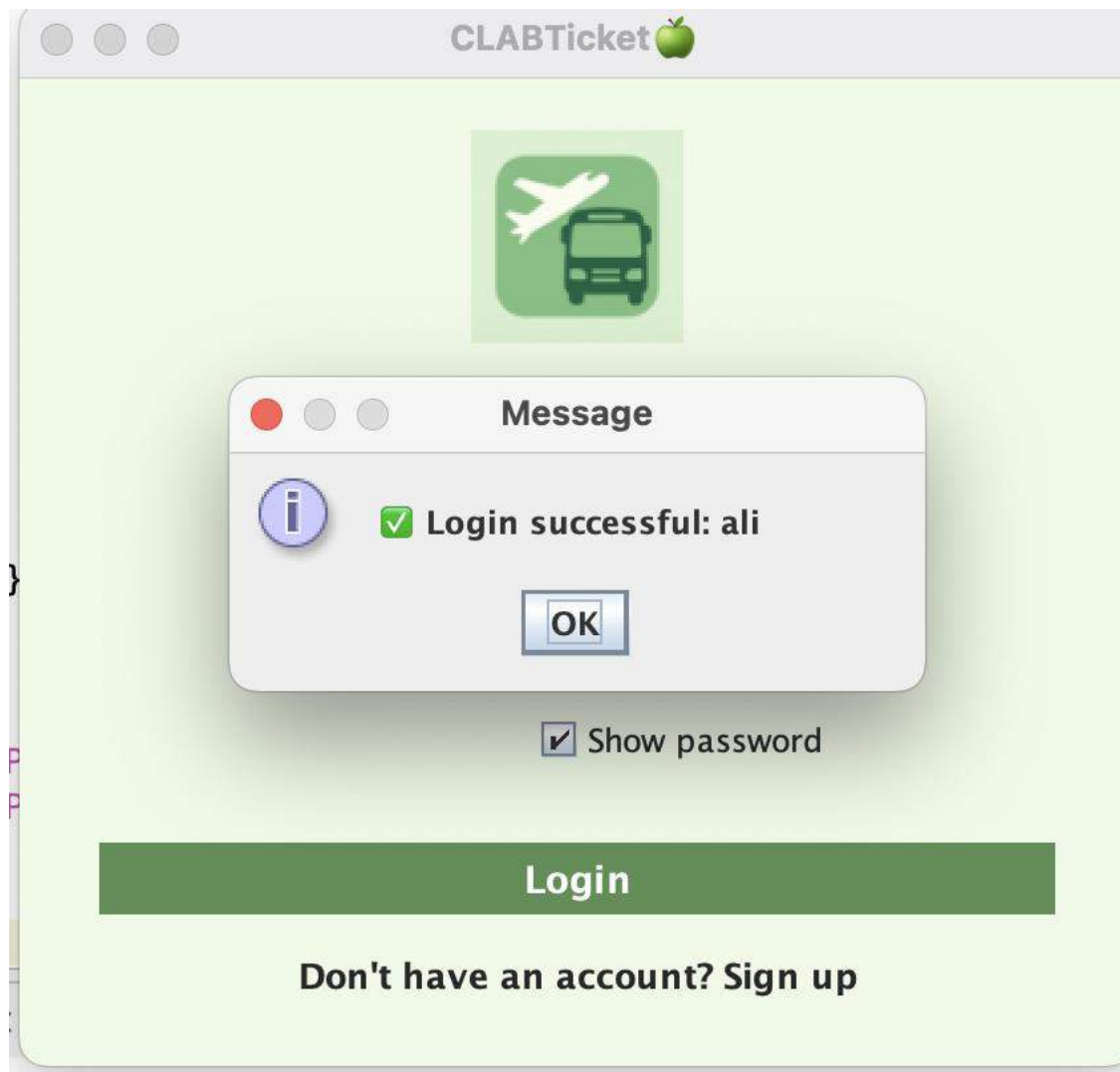
username:

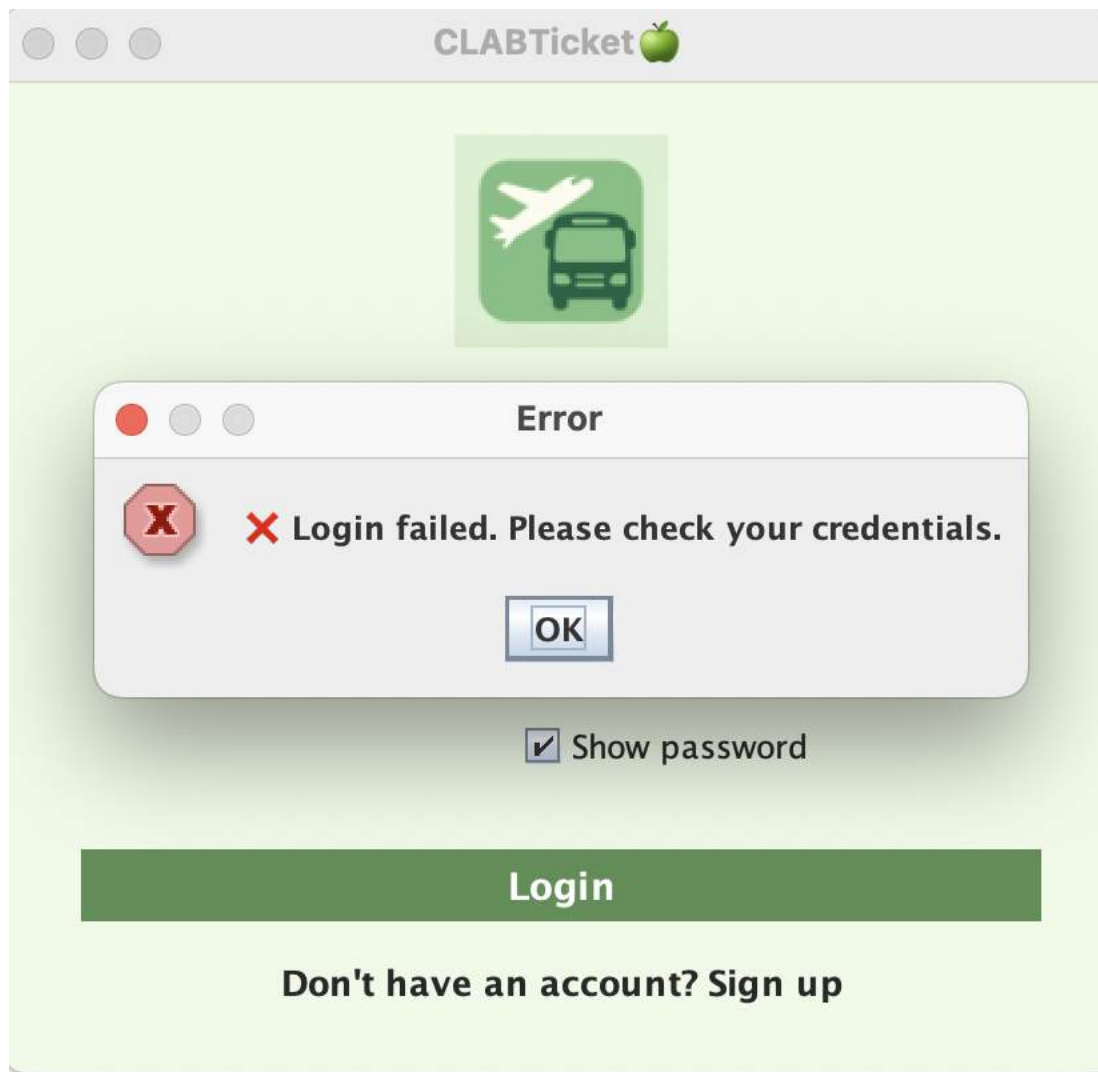
password:

☐ Show password

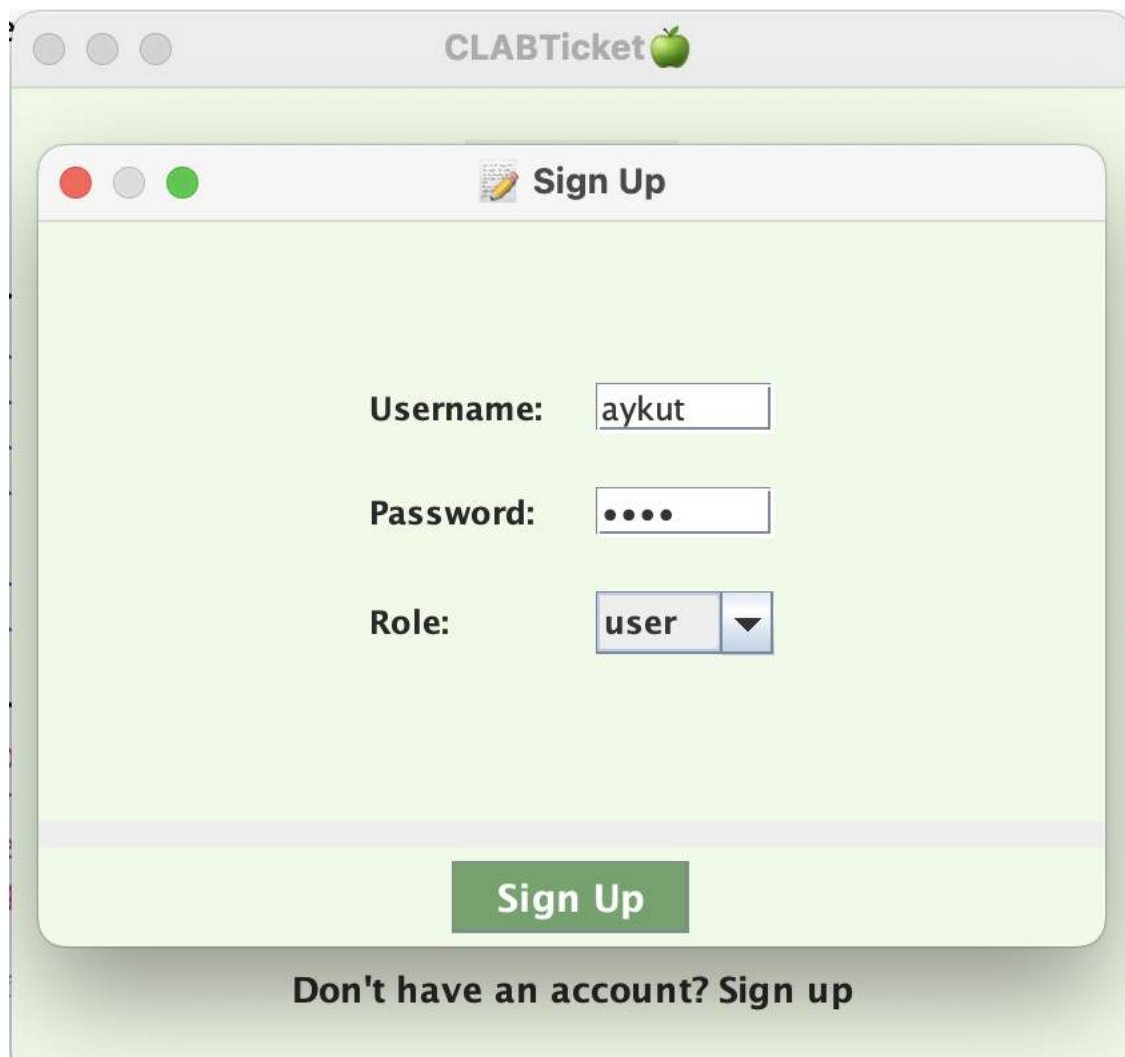
Login

Don't have an account? Sign up





5.2-Sign Up



The image shows a web browser window titled "CLABTicket" with a green apple icon. Inside the browser is a "Sign Up" form window. The form has three input fields: "Username:" with the value "aykut", "Password:" with four dots, and "Role:" with a dropdown menu showing "user". Below the form is a green "Sign Up" button. At the bottom of the browser window, there is a link that says "Don't have an account? Sign up".

CLABTicket 

Sign Up

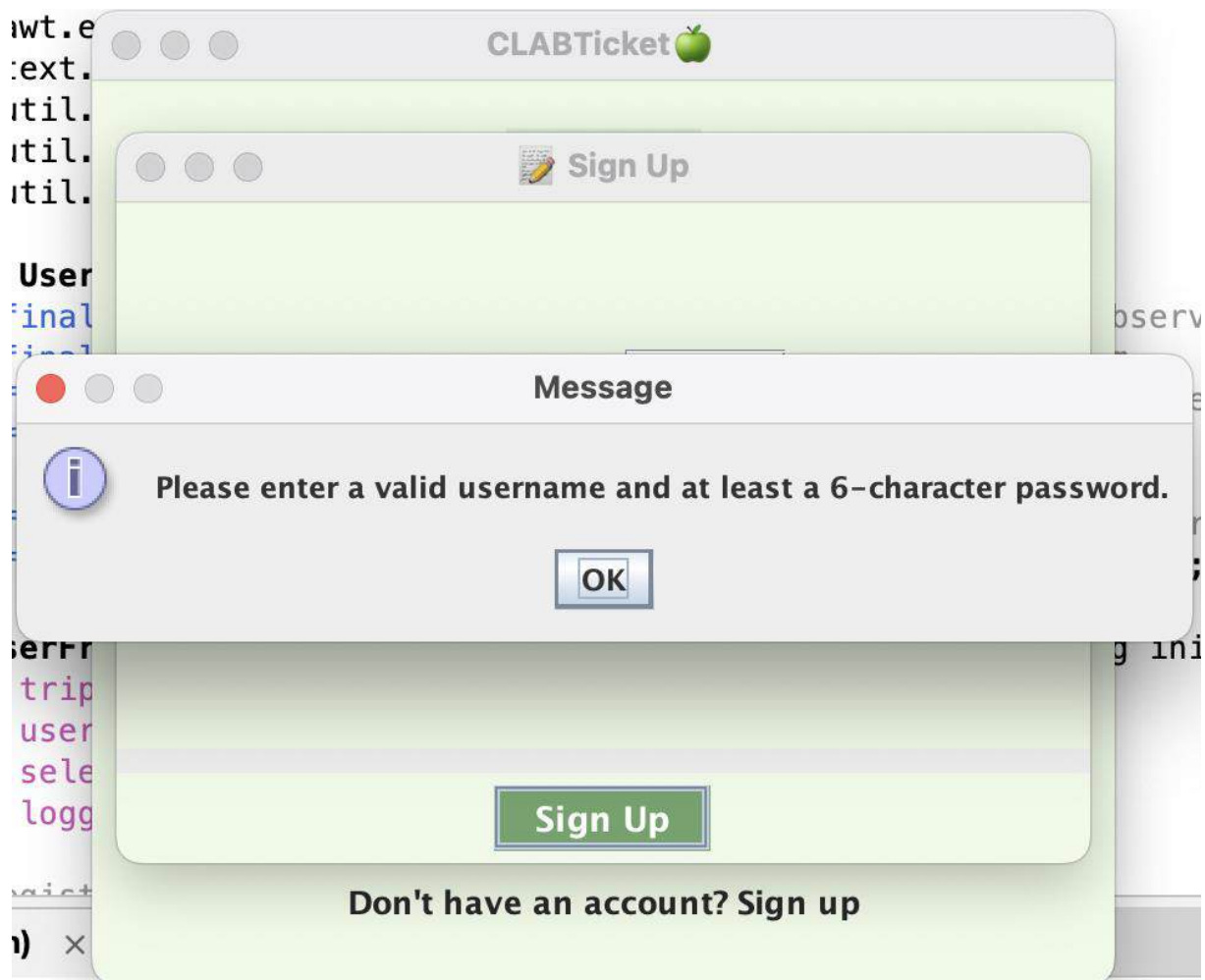
Username:

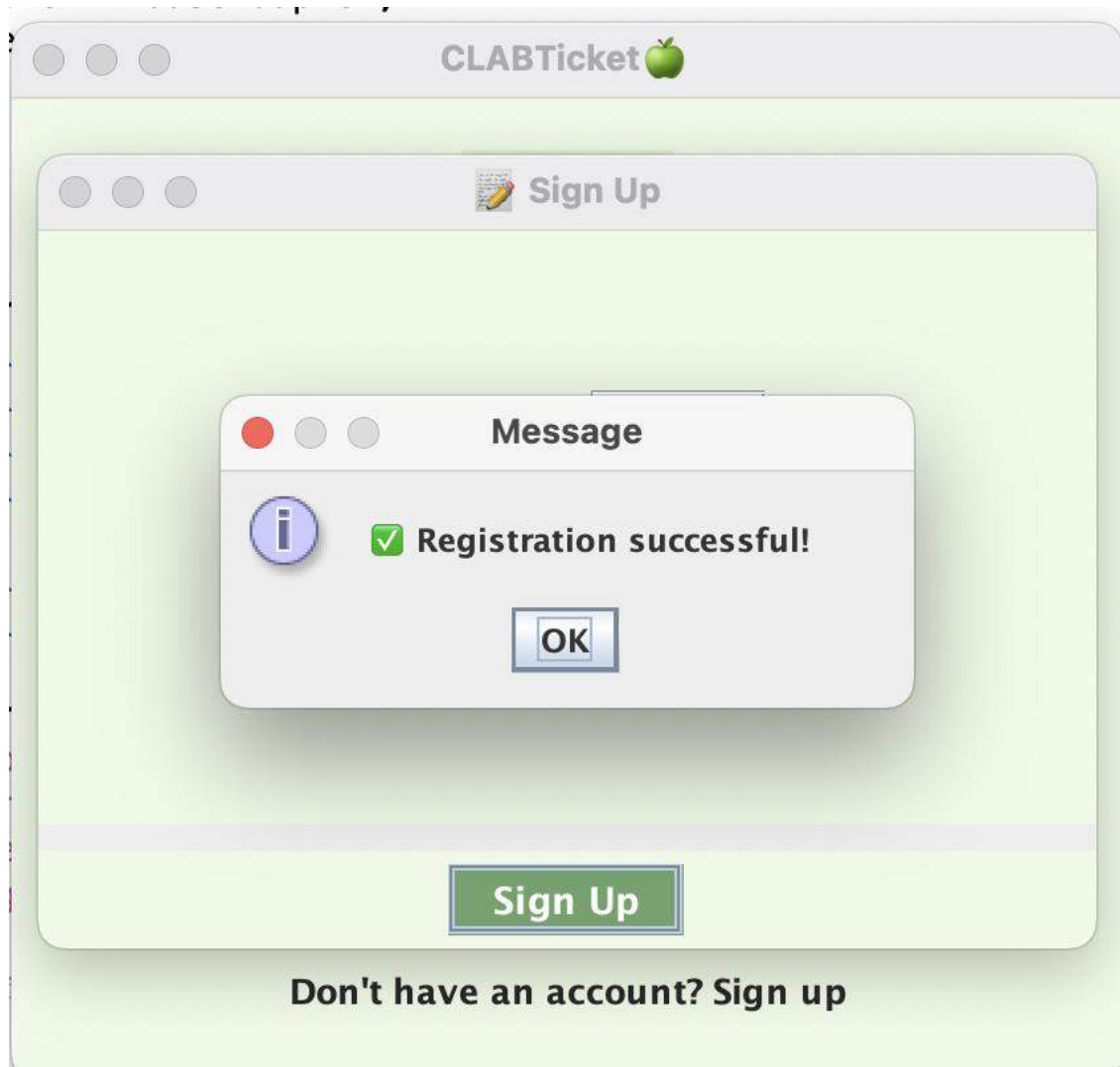
Password:

Role: ▼

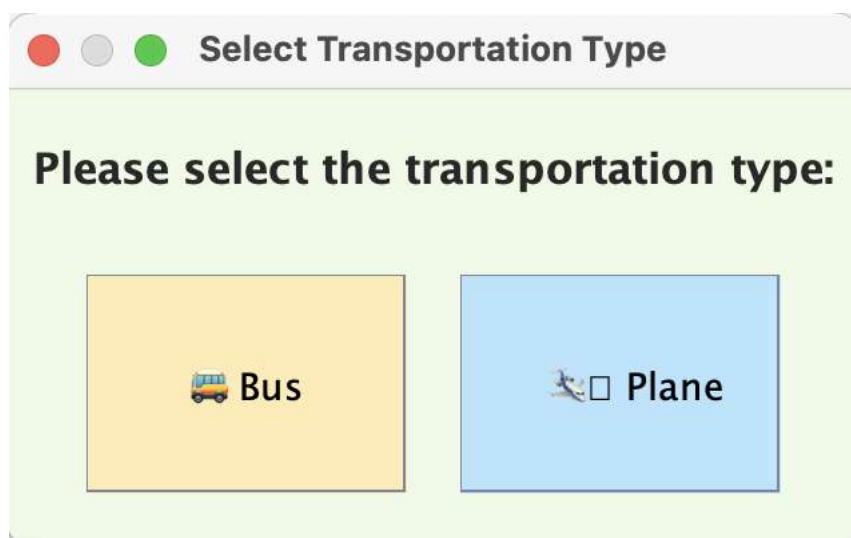
Sign Up

[Don't have an account? Sign up](#)

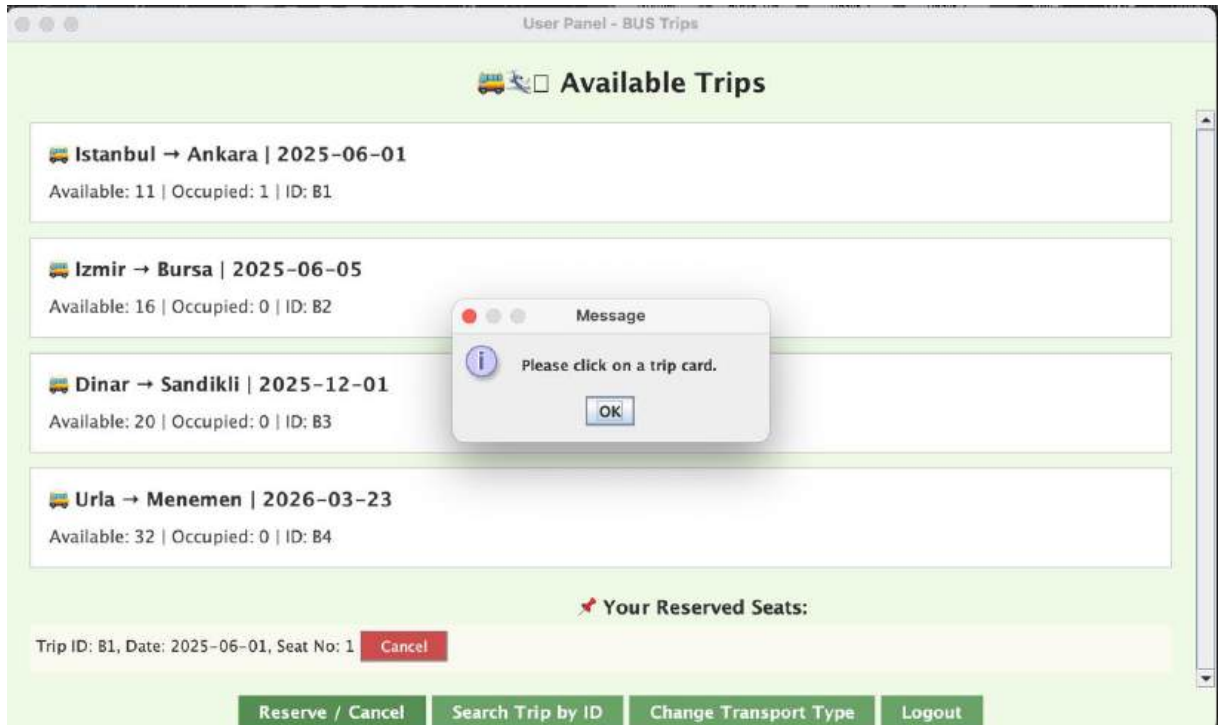
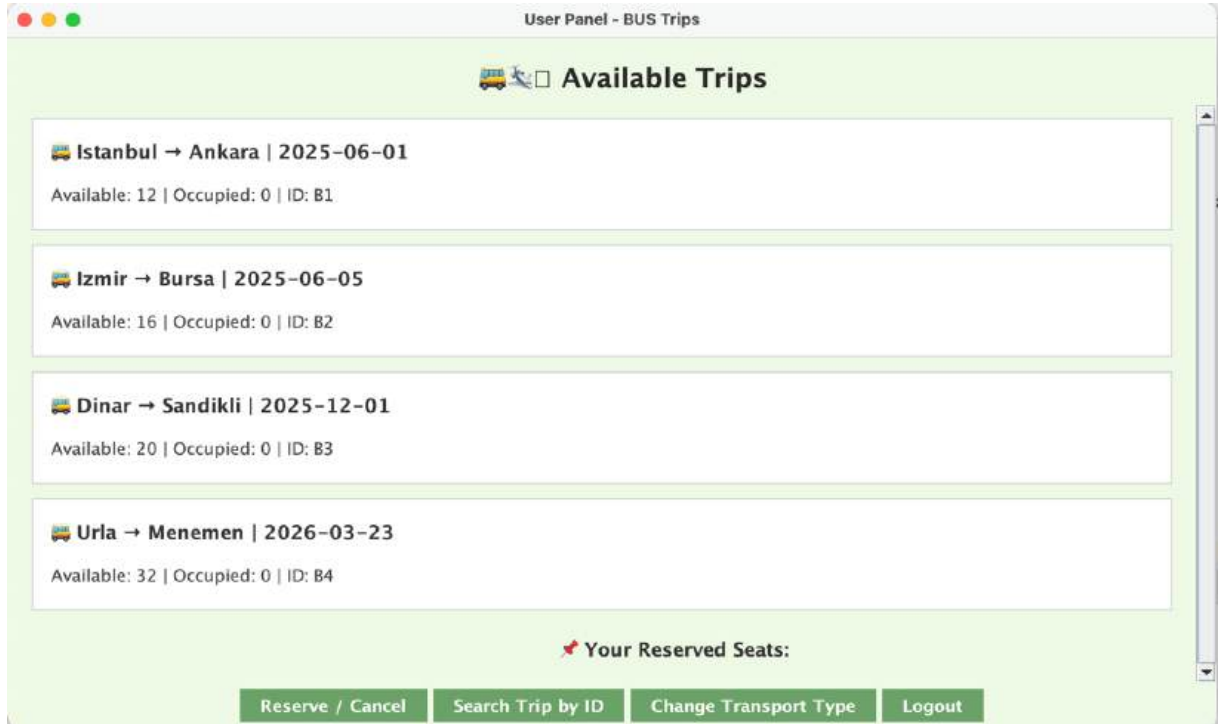




5.3-Select Transportation Type



5.4-User Panel-Bus Trips



User Panel - BUS Trips

  Available Trips

 **Istanbul → Ankara | 2025-06-01**
Available: 11 | Occupied: 1 | ID: B1

 **Izmir → Bursa | 2025-06-05**
Available: 16 | Occupied: 0 | ID: B2

 **Dinar → Sandikli | 2025-12-01**
Available: 20 | Occupied: 0 | ID: B3

 **Urla → Menemen | 2026-03-23**
Available: 32 | Occupied: 0 | ID: B4

 **Your Reserved Seats:**

Trip ID: B1, Date: 2025-06-01, Seat No: 1 Cancel

Reserve / Cancel

Search Trip by ID

Change Transport Type

Logout

Input

?

Enter Trip ID:

B2

OK

Cancel

User Panel - BUS Trips

  Available Trips

 **Istanbul → Ankara | 2025-06-01**
Available: 11 | Occupied: 1 | ID: B1

 **Izmir → Bursa | 2025-06-05**
Available: 16 | Occupied: 0 | ID: B2

 **Dinar → Sandikli | 2025-12-01**
Available: 20 | Occupied: 0 | ID: B3

 **Urla → Menemen | 2026-03-23**
Available: 32 | Occupied: 0 | ID: B4

 **Your Reserved Seats:**

Trip ID: B1, Date: 2025-06-01, Seat No: 1 Cancel

Reserve / Cancel

Search Trip by ID

Change Transport Type

Logout

Trip Info

i

Trip Detail:

ID: B2

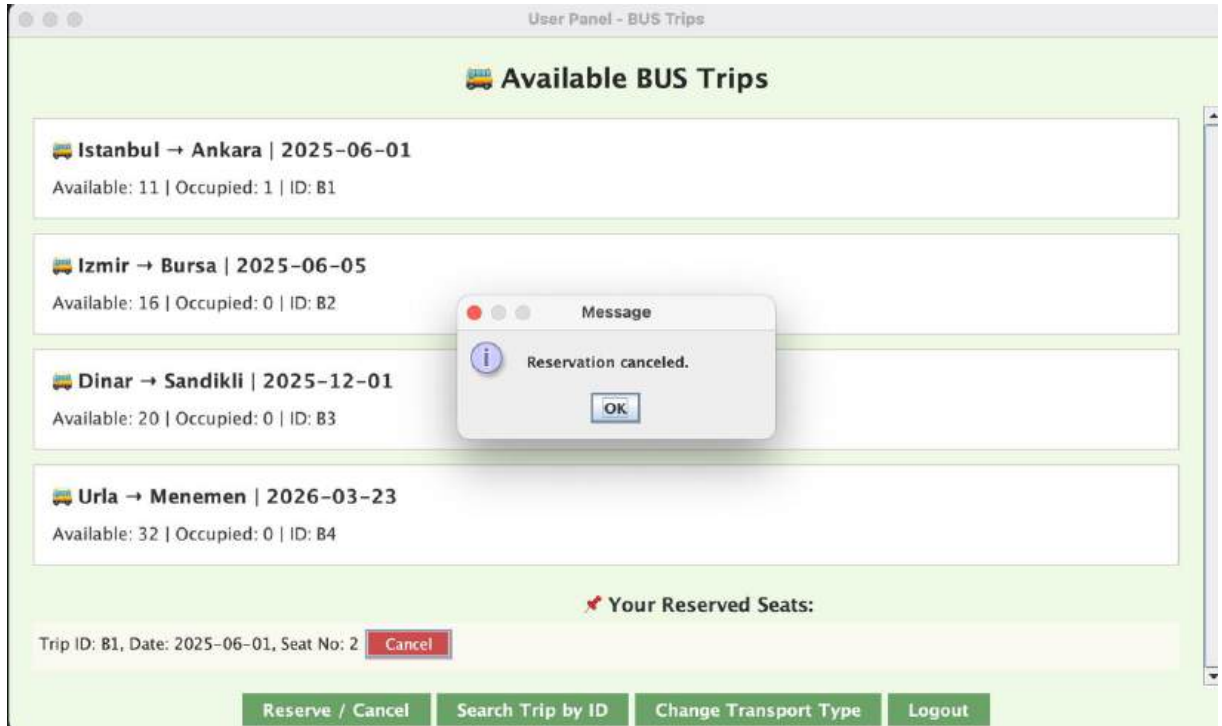
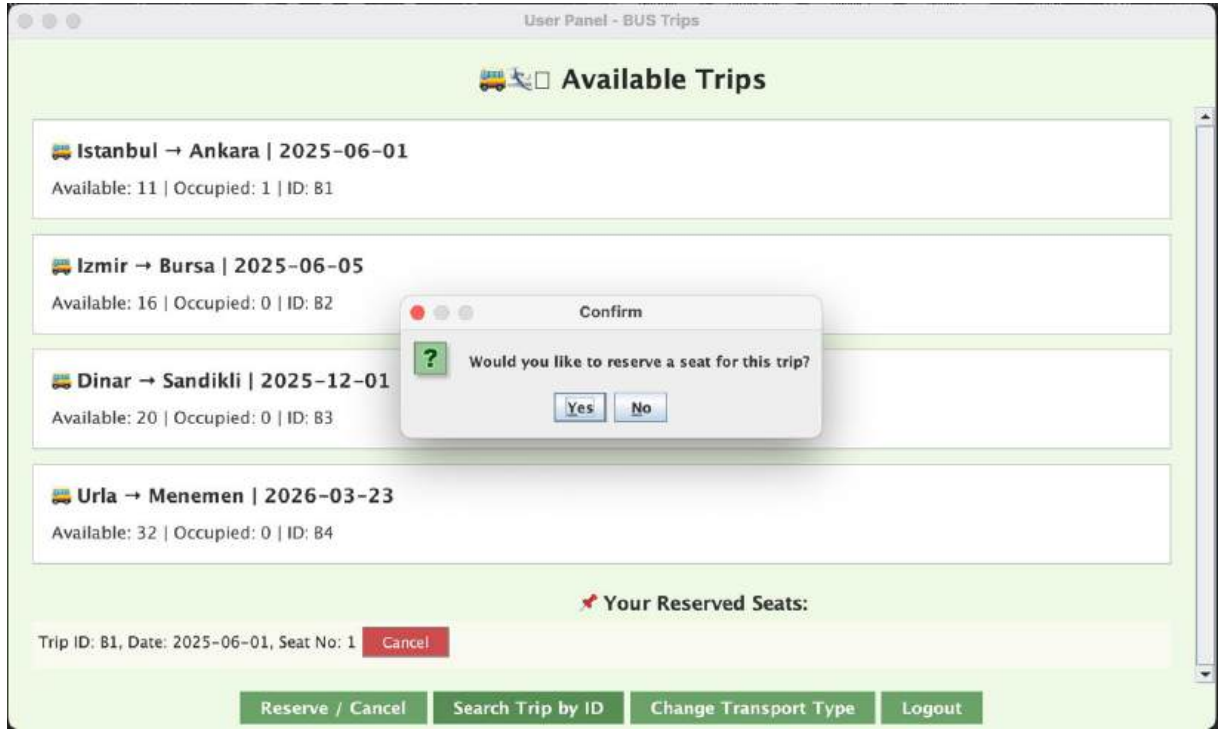
Departure: Izmir

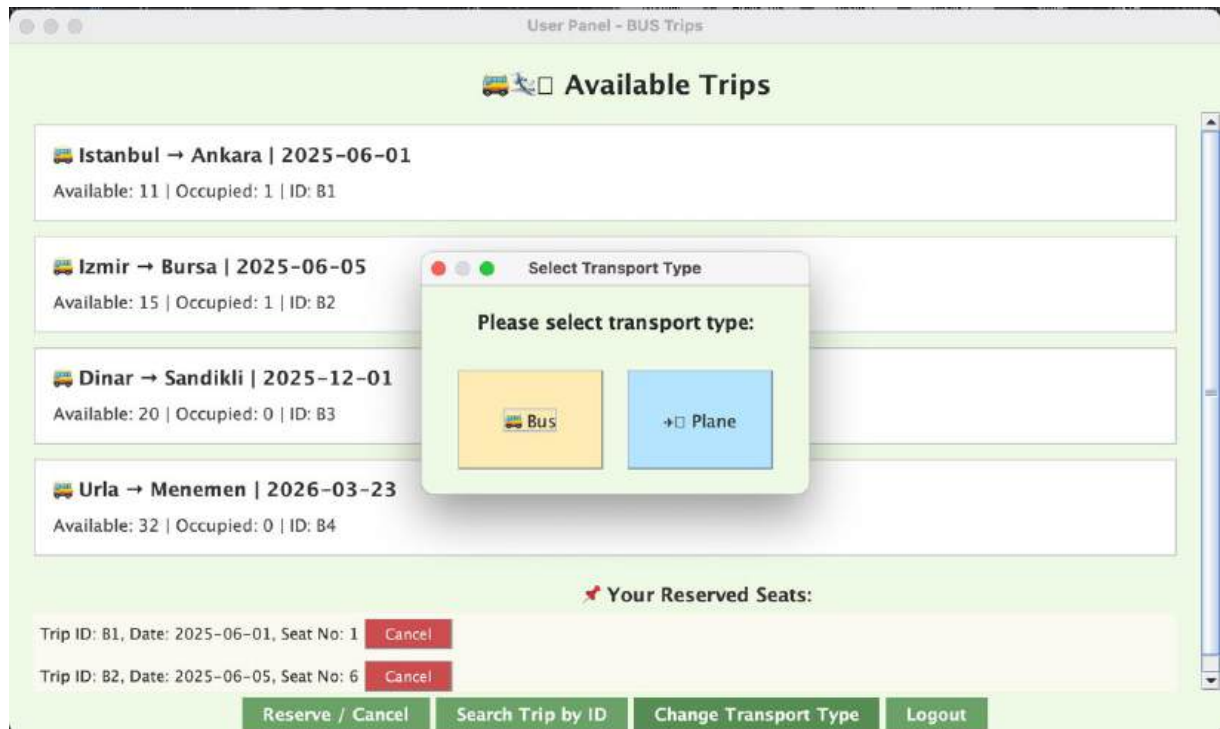
Arrival: Bursa

Date: 2025-06-05

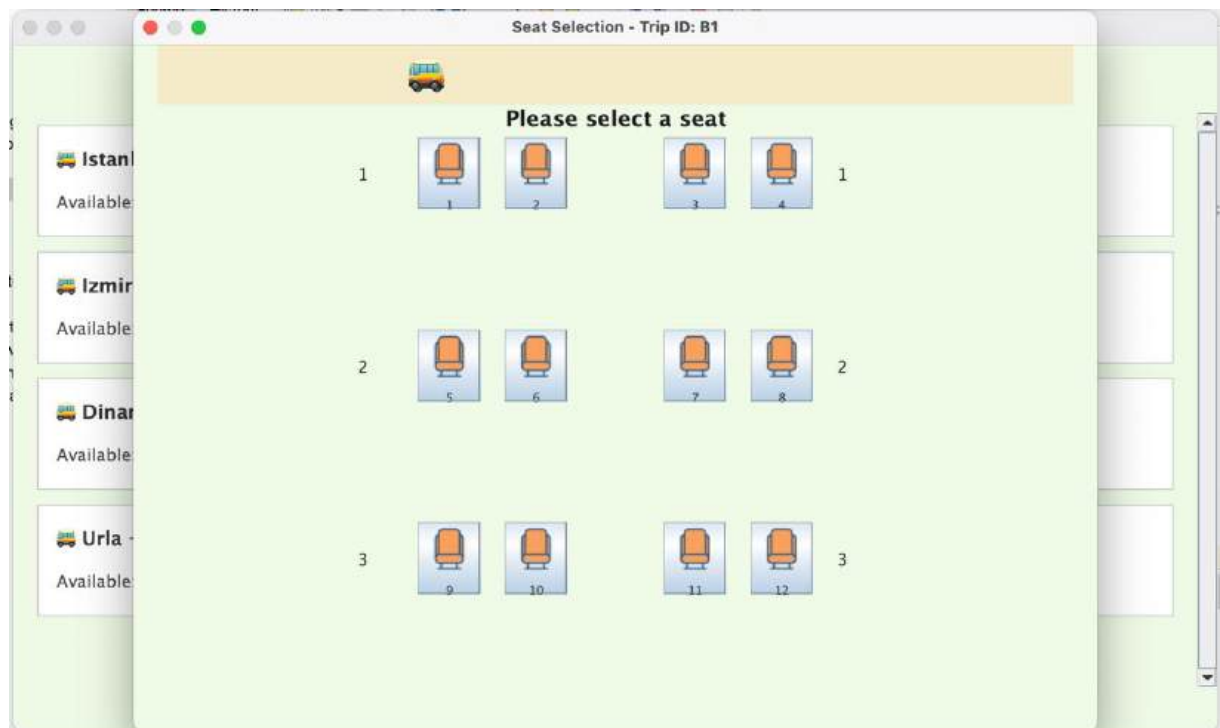
OK

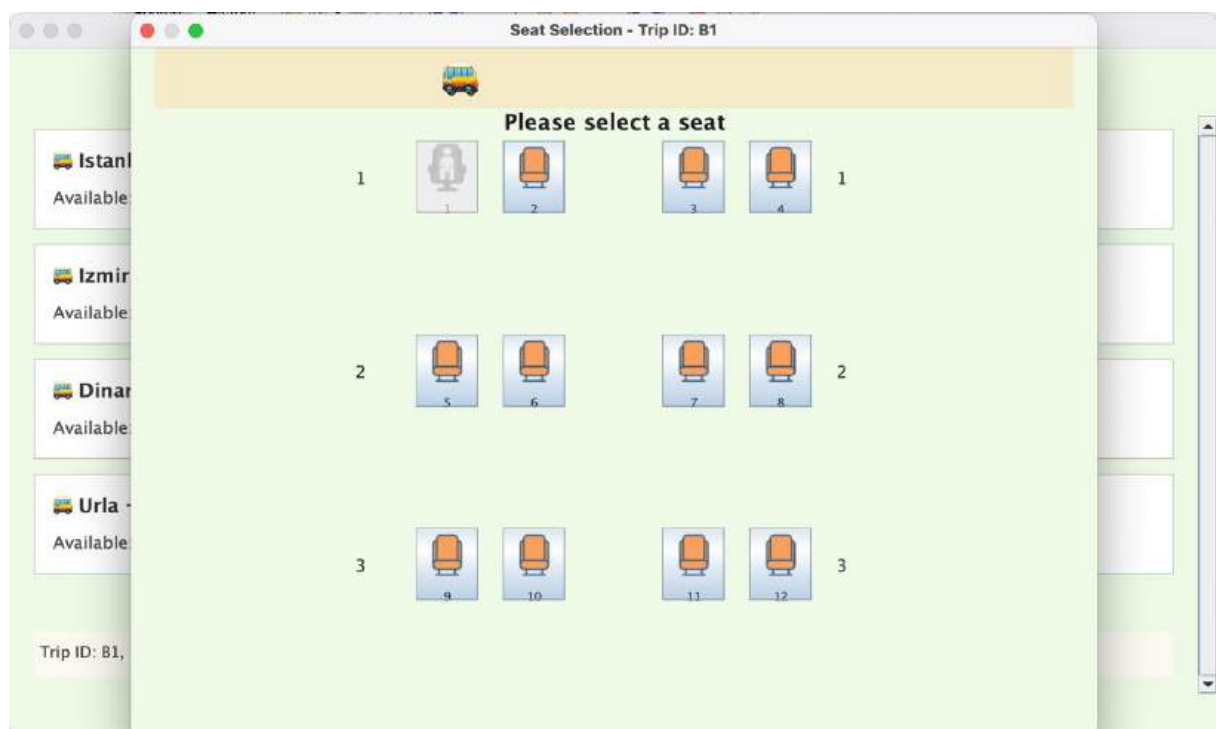
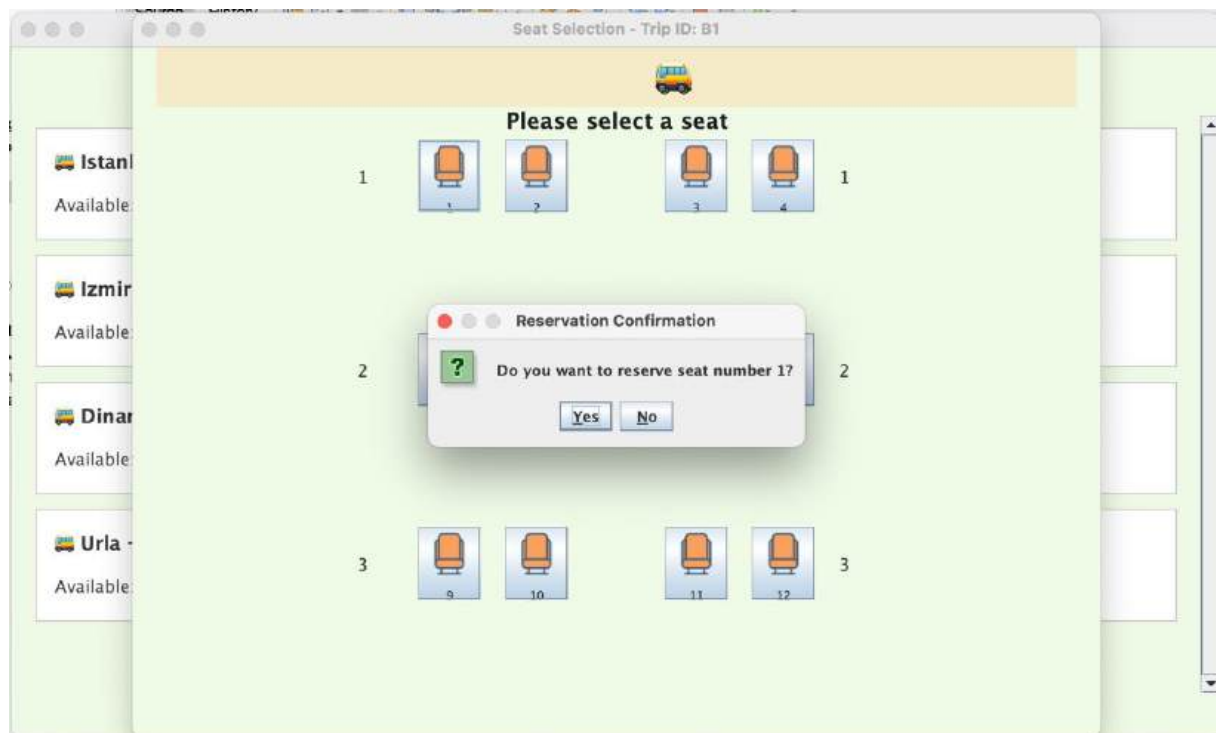
58



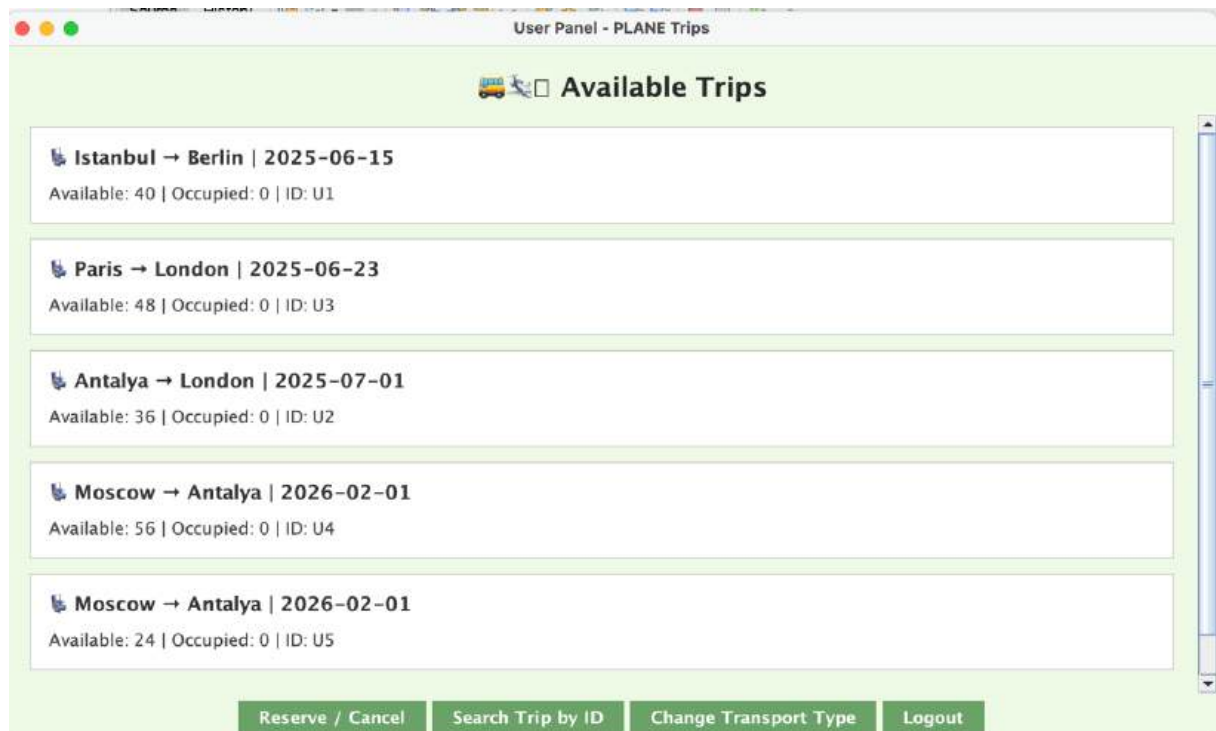


5.5-Seat Selection (Bus)



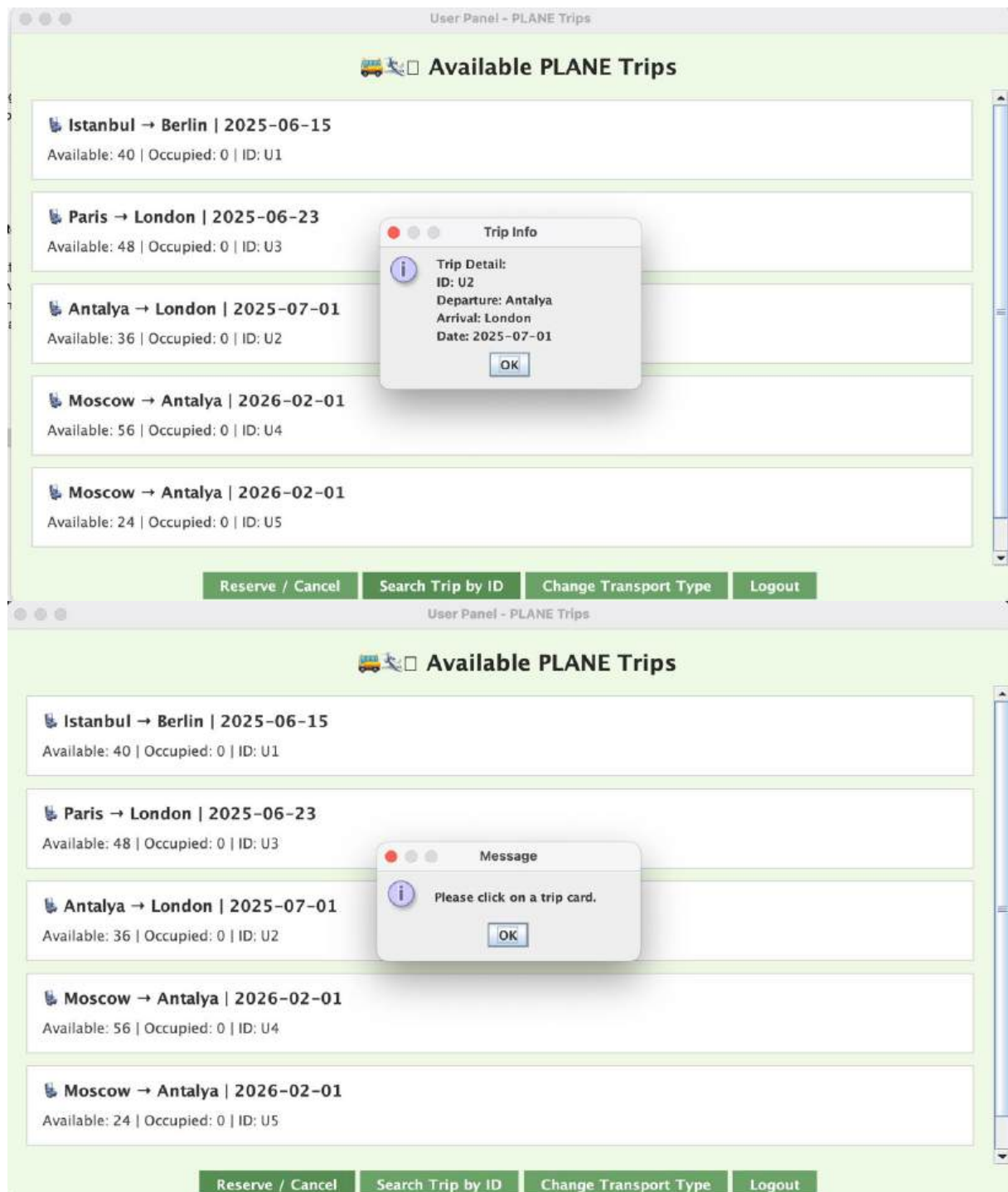


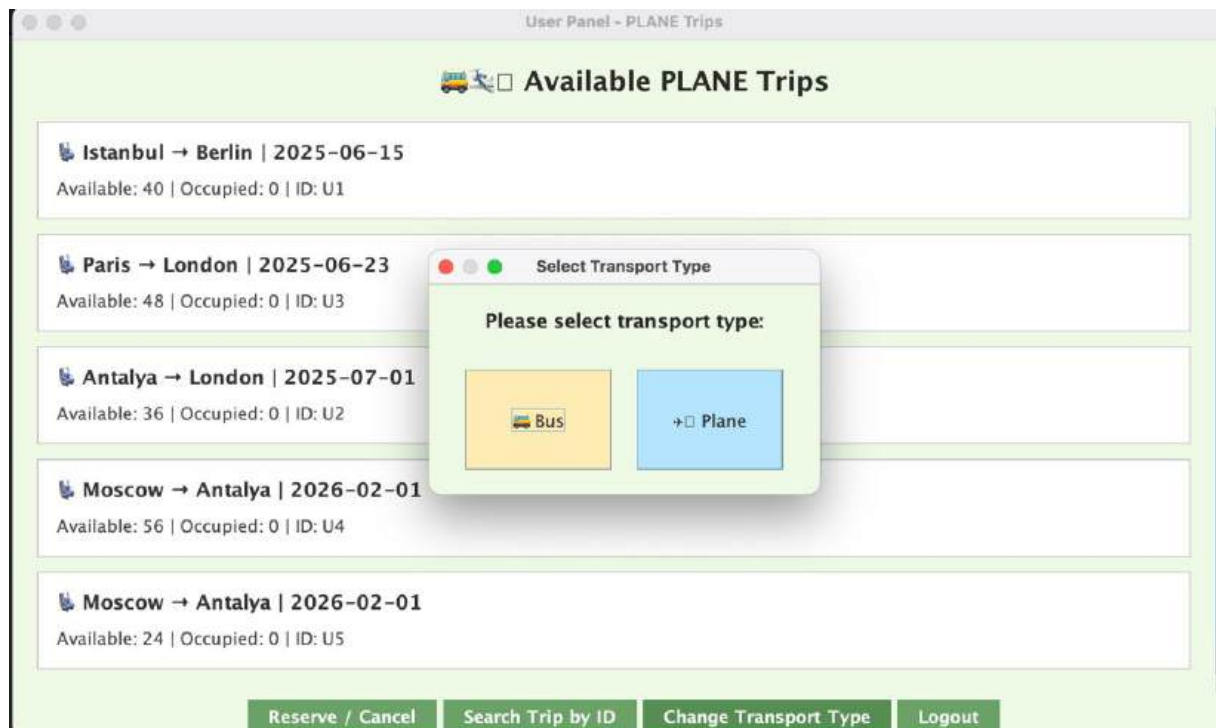
5.6-User Panel-Plane Trips



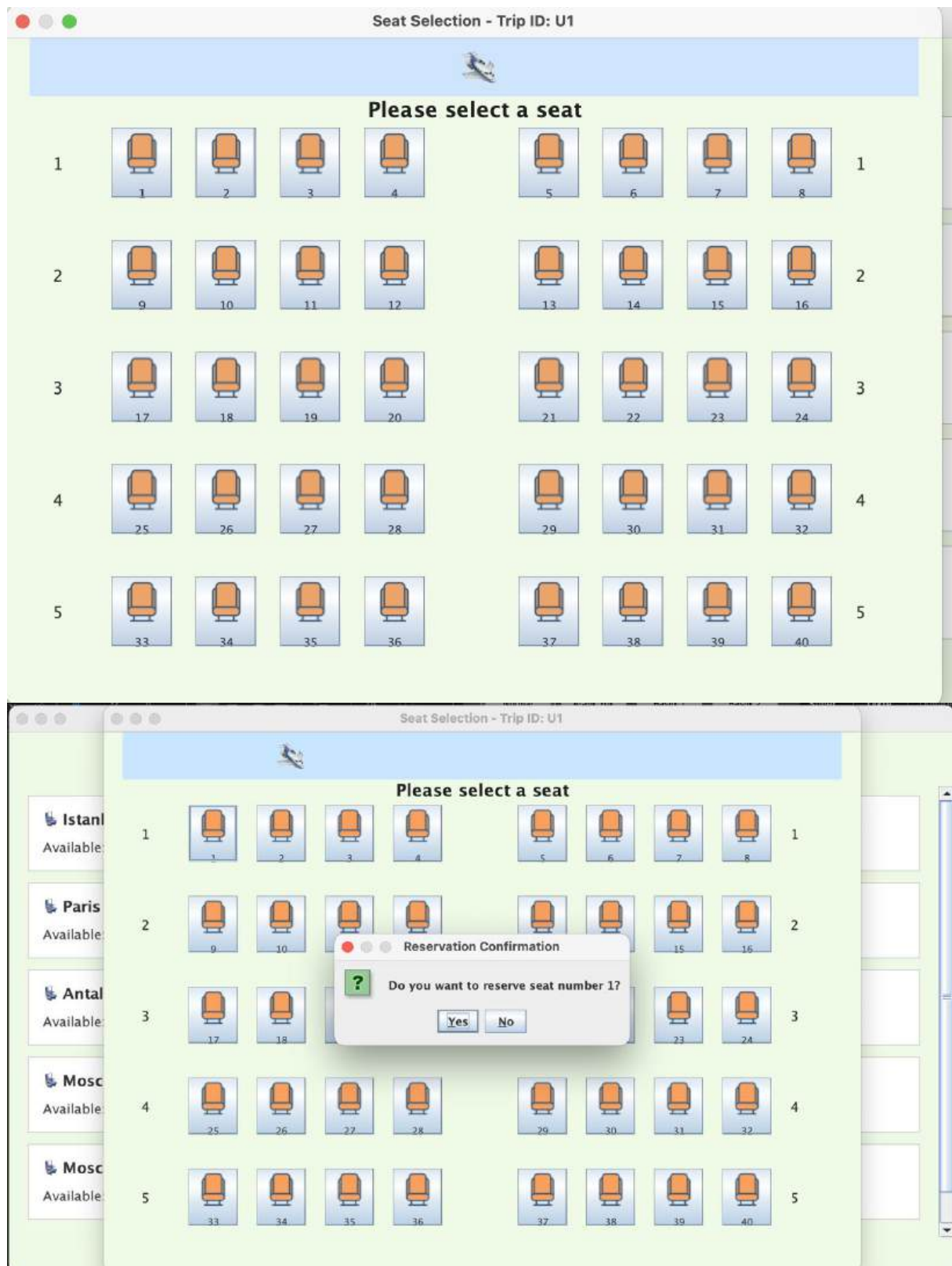
The screenshot displays a web application window titled "User Panel - PLANE Trips". The interface has a light green header and footer. The main content area is titled "Available Trips" with a small airplane icon. It lists five trips, each in a white box with a green border. Each trip entry includes a route, date, and availability status. The footer contains four green buttons: "Reserve / Cancel", "Search Trip by ID", "Change Transport Type", and "Logout".

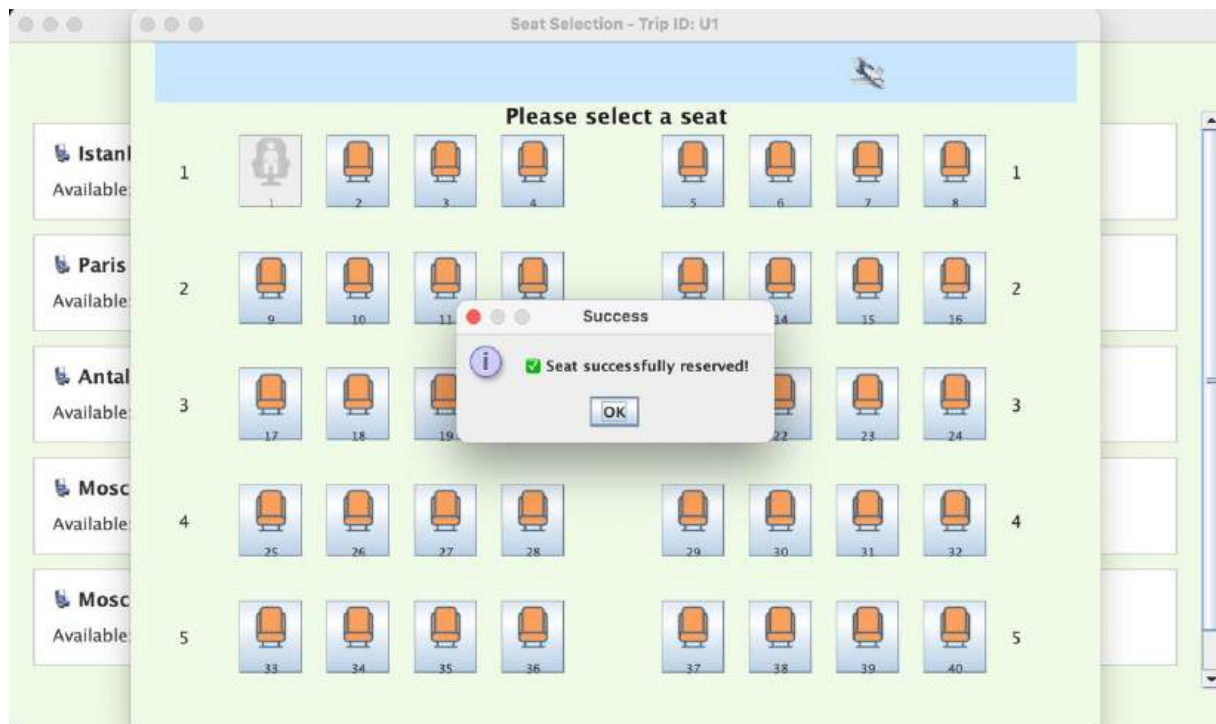
Route	Date	Available	Occupied	ID
Istanbul → Berlin	2025-06-15	40	0	U1
Paris → London	2025-06-23	48	0	U3
Antalya → London	2025-07-01	36	0	U2
Moscow → Antalya	2026-02-01	56	0	U4
Moscow → Antalya	2026-02-01	24	0	U5



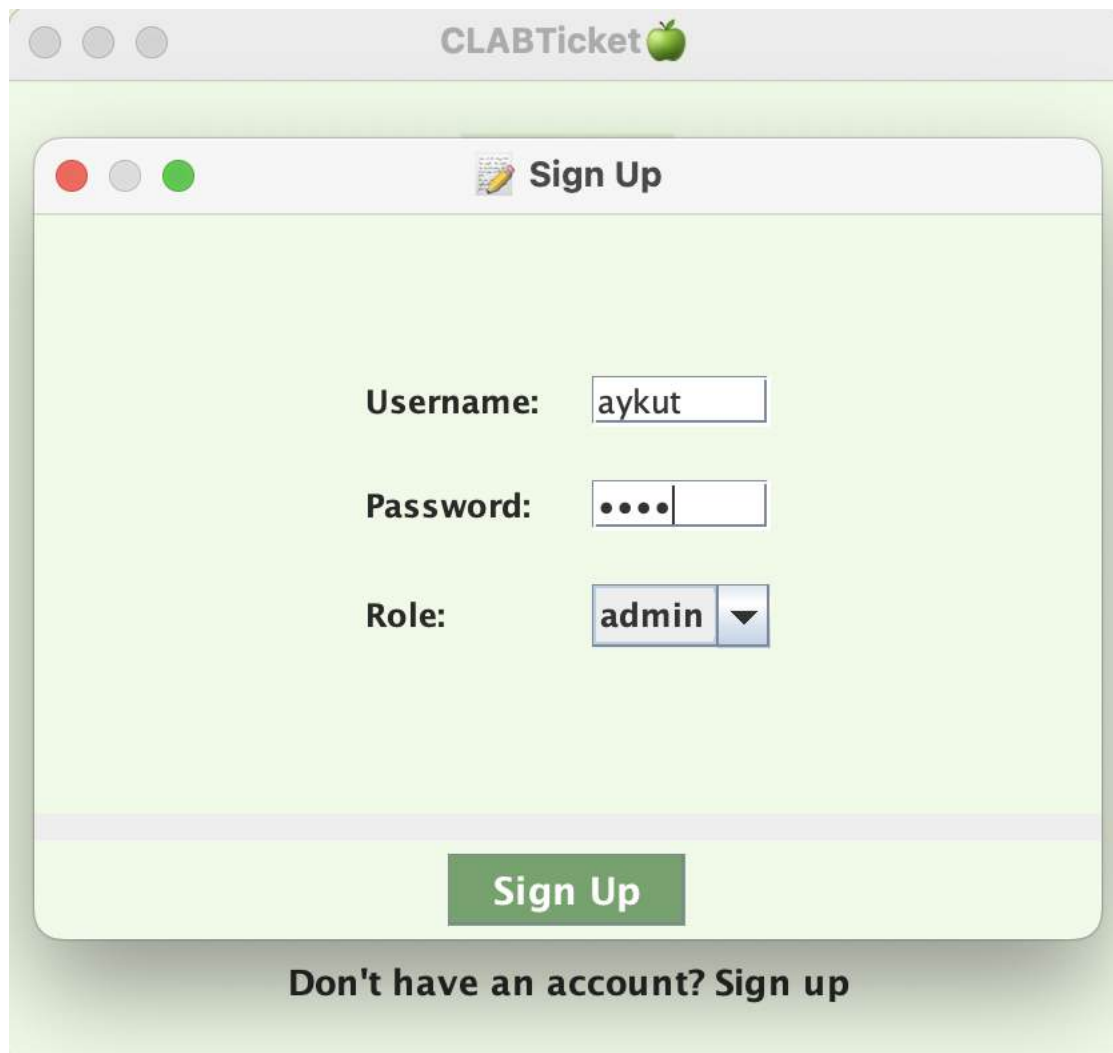


5.7-Seat Selection(Plane)





5.8-Admin Sign Up



The image shows a web application window titled "CLABTicket" with a green apple icon. Inside, there is a "Sign Up" dialog box with a light green background. The dialog box has a title bar with a pencil icon and the text "Sign Up". It contains three input fields: "Username:" with the value "aykut", "Password:" with four dots, and "Role:" with a dropdown menu showing "admin". A green "Sign Up" button is at the bottom of the dialog. Below the dialog, there is a link that says "Don't have an account? Sign up".

CLABTicket

Sign Up

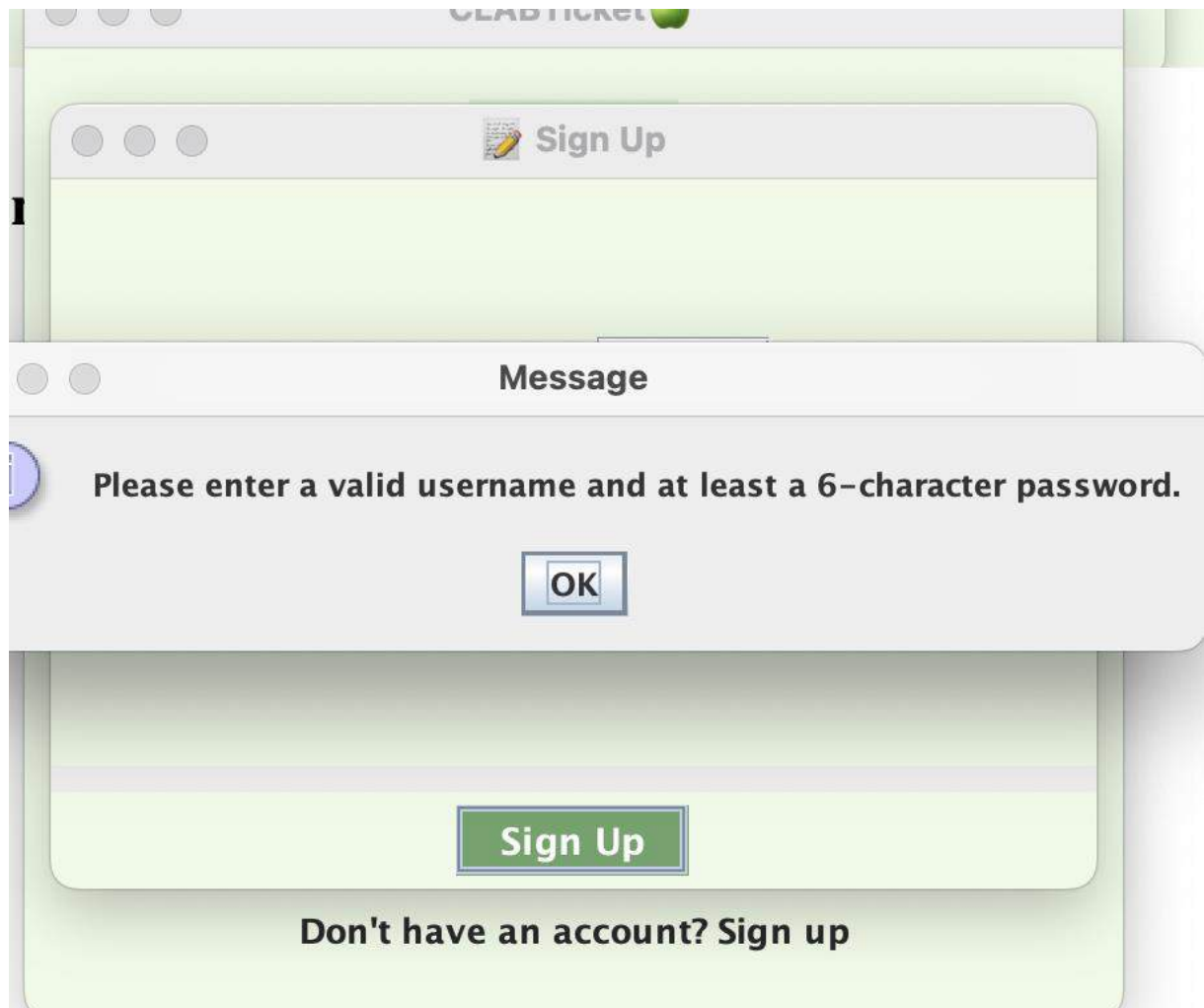
Username:

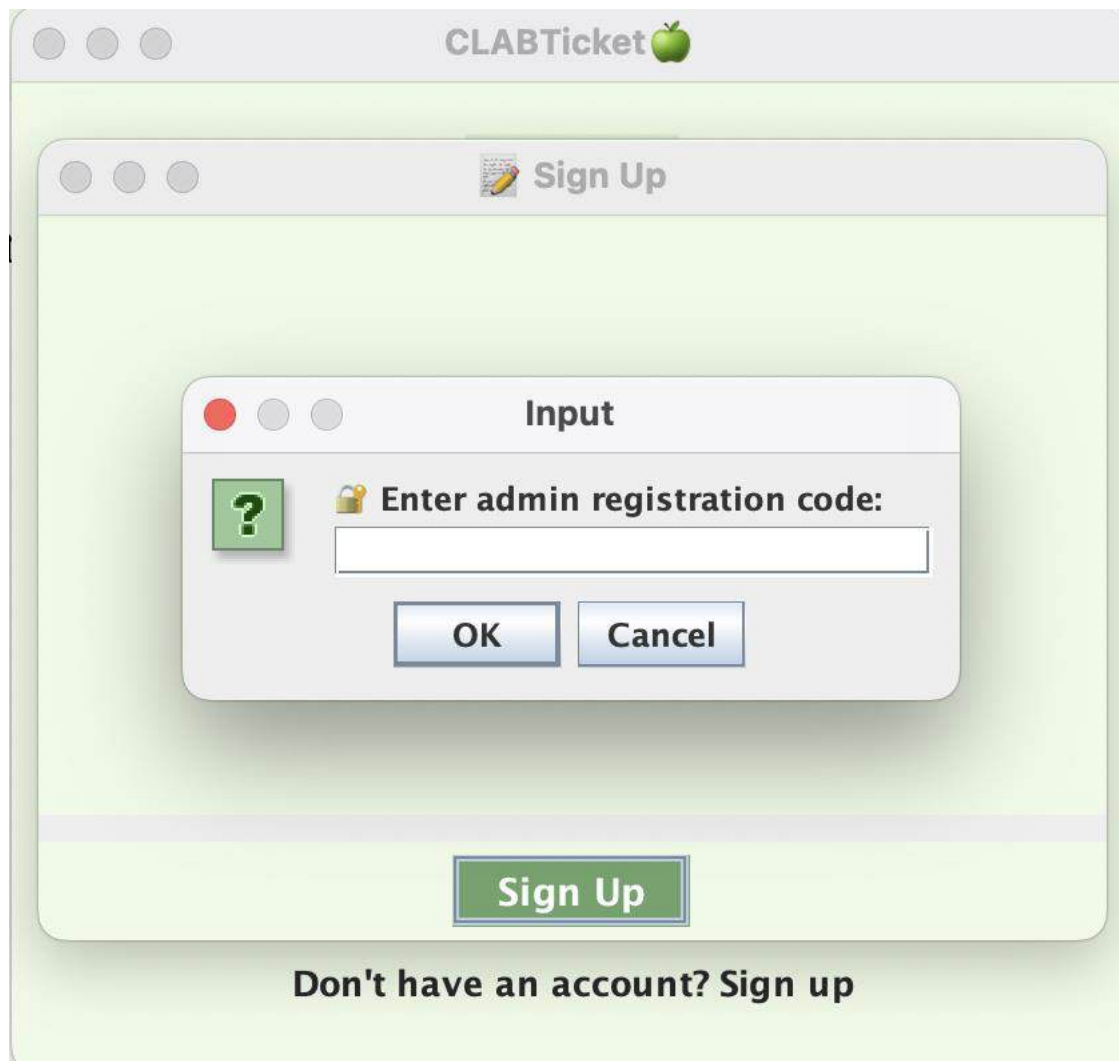
Password:

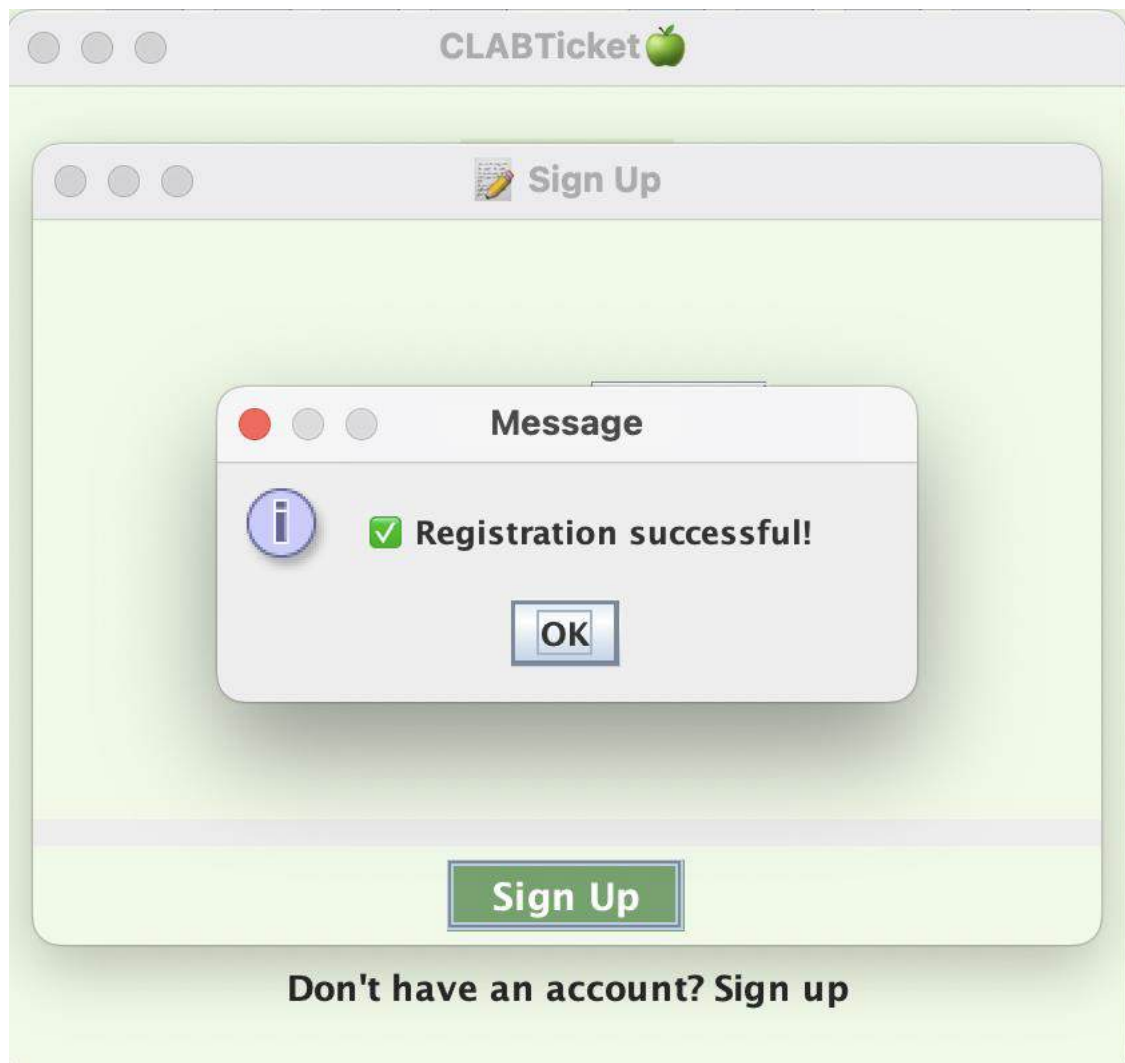
Role: ▼

Sign Up

Don't have an account? [Sign up](#)







5.9-Admin Panel

Admin Panel - Add Trip

+ Add Trip Panel

Trip ID:

S4

Departure:

Izmir

Arrival:

Kircaali

Date (yyyy-MM-dd):

2025-07-08

Seat Count:

16

Type:

plane

✓ Add Trip

Logout

View Trips

Admin Panel - Add Trip

+ Add Trip Panel

Trip ID:

S4

Departure:

Bornova

Arrival:

Urla

Date (yyyy-MM-dd):

2025-06-24

Seat Count:

12

Type:

bus

✓ Add Trip

Logout

View Trips

Admin Panel - Add Trip

+ Add Trip Panel

Trip ID:
 Departure:
 Arrival:
 Date:
 Seat Count:
 Type:

Message

✓ Trip added.

OK

5.10-Trip List (Admin Panel)

Trip List

All Trips

ID	Departure	Arrival	Date	Type	Available	Occupied
B1	Istanbul	Ankara	2025-06-01	bus	12	0
B2	Izmir	Bursa	2025-06-05	bus	16	0
U1	Istanbul	Berlin	2025-06-15	plane	40	0
U3	Paris	London	2025-06-23	plane	48	0
S4	Bornova	Urla	2025-06-24	bus	12	0
U2	Antalya	London	2025-07-01	plane	36	0
B3	Dinar	Sandikli	2025-12-01	bus	20	0
U4	Moscow	Antalya	2026-02-01	plane	56	0
U5	Moscow	Antalya	2026-02-01	plane	24	0
B4	Urla	Menemen	2026-03-23	bus	32	0

Trip List

All Trips

ID	Departure	Arrival	Date	Type	Available	Occupied
B1	Istanbul	Ankara	2025-06-01	bus	12	0
B2	Izmir	Bursa	2025-06-05	bus	16	0
U1	Istanbul	Berlin	2025-06-15	plane	40	0
U3	Paris	Lond			48	0
S4	Bornova	Urla			12	0
U2	Antalya	Lond			36	0
B3	Dinar	Sandi			20	0
U4	Moscow	Antalya			56	0
U5	Moscow	Antalya	2026-02-01	plane	24	0
B4	Urla	Menemen	2026-03-23	bus	32	0

Confirm

?

Delete trip 'B1'?

YesNo

Delete Trip

Trip List

All Trips

ID	Departure	Arrival	Date	Type	Available	Occupied
B1	Istanbul	Ankara	2025-06-01	bus	12	0
B2	Izmir	Bursa	2025-06-05	bus	16	0
U1	Istanbul	Berlin	2025-06-15	plane	40	0
U3	Paris	Lond			48	0
S4	Bornova	Urla			12	0
U2	Antalya	Lond			36	0
B3	Dinar	Sandi			20	0
U4	Moscow	Antalya			56	0
U5	Moscow	Antalya	2026-02-01	plane	24	0
B4	Urla	Menemen	2026-03-23	bus	32	0

Message

i

✓ Trip deleted.

OK

Delete Trip